جلسه دوم - ۱۳۹۹/۱۲/۵

برای استخراج دانش از دیتا این شرایط باید فراهم شود:

① دیتا باید ذخیره بشه ← Storage

② دیتا باید manage (مدیریت) بشه

③ دیتا باید تحلیل (آنالیز) بشه

approximation : ✗

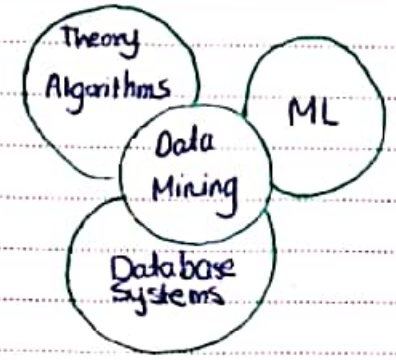
✗

الگوریتم های Data Mining :

① الگوریتم های توصیفی (descriptive) : متدهایی که در آن دانش یا Knowledge ای که به طور implicit در دیتا وجود دارد را استخراج می کنند. در مورد چیزی که در دیتا نیست حرفی نمی زنند. پیش بینی انجام نمی دهند. مثل خوشه بندی، استخراج الگو،

② الگوریتم های پیش بینی کننده (predictive) : با یک مدل یادی گیرید. یک متد توسعه می دهید که برای قسمتی از داده که وجود ندارد پیش بینی انجام می دهد

مثل Classification، الگوریتم های Recommender systems



سالروز عملیات افتخار آفرین مرصاد (۱۳۶۷ هـ ش)

قسمت های این درس:

Machine Learning, Data Mining, databases, statistics

اما تمرکز اصلی روی:

Scalability
Algorithms
Computing Architectures
Automation for handling large data

Scanned with CamScanner

دنیای کلان داده big Data داریم ۳ ویژگی دارد:

① Volume ← باید زیاد باشه

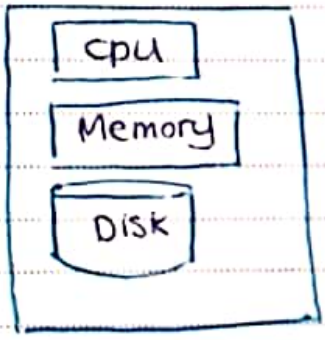
② Velocity ← باید سرعت تولید داده‌ها زیاد باشه

③ Variety ← تنوع داده ها باید زیاد باشه

مدل‌های محاسباتی که استفاده خواهیم کرد:

① MapReduce ② مدل پردازش جریان داده‌ها ③ Single Machine in memory

محاسبات توزیع شده برای کلان داده ها



معماری کامپیوترهای عادی ←

اگه بخواهیم دیتا رو بکوپی بگیریم با یه کامپیوتر عادی ۴ ماه طول می‌کشه!

شرکت های بزرگ به جای استفاده از سوپر کامپیوترها که هزینه زیادی دارند از یک معماری دیگر استفاده می‌کنند به نام Commodity Hardware (سخت افزار دم دستی)

تعداد زیادی کامپیوترهای عادی را باهم شبکه می‌کنند

مشکل دیگه: به کامپیوتری خواندن داده‌هایی پردازش انجام بدهد که در کامپیوتر دیگری است. خیلی هزینه بر است که داده‌ها منتقل شود از طریق network.

Scanned with CamScanner

① File system

② سبک برنامه نویسی

اگر بخواهیم با از دست رفتن یک نود، دیتاهای آن را از دست ندهیم چکار باید بکنیم ؟

باید به replication - global name space داشته باشیم

یه فایل بزرگ به یک سری chunk شکسته می‌شود و هر chunk روی یک سرور commodity قرار می‌گیرد. هر chunk دو یا سه بار تکرار می‌شه در کامپیوترهای مختلف و بهتره که این تکرارها از نظر جغرافیایی از هم دور باشند.

یک Master node داریم ← اطلاعات مربوطه chunk ها را ذخیره می‌کند

خود chunk server پردازش روی داده‌ها را هم انجام می‌دهد یعنی Comput server هم هست

باید سعی کنیم تا جایی که می‌شه ∝

مدل برنامه نویسی (programming Model)

به طور کلی Map_Reduce نگاشت_کاهش نامیده می‌شود. اهداف این مدل برنامه نویسی :

۱_ باید برنامه نویسی موازی راحت روش انجام بگیره

۲_ جزئیات های سخت افزاری و نرم افزاری باید از دید برنامه نویس مخفی باشه

۳_ باید قابلیت مدیریت داده های خیلی حجیم رو داشته باشه

شهادت امام محمد باقر (ع) (۱۱۴ هـ ق)

Scanned with CamScanner

گام‌های MapReduce :

① Map

یه سری Map Function کاربری می‌نویسه. ورودی این یه سری زوج کلید-مقدار است. خروجی این هم تعداد دل‌خواهی زوج کلید-مقدار است. (می‌تواند صفر باشد)

لزوما زوج کلید-مقدارهای ورودی و خروجی یکسان نیستند.

واحد پردازشی ما یک Map task است. تعریف ∝ Map task

② Group by key

بعد از مرحله Map، خروجی ما تعداد زیادی جفت کلید-مقدار است. در این مرحله گروه‌بندی براساس کلید انجام می‌شه. درواقع یک مرتب‌سازی براساس key داریم. به کلید داریم و یه لیستی از مقادیر

output : key - (list of values)

③ Reduce

بدنه‌ی این تابع را خود کاربر باید implement کند. ورودی این تابع همان خروجی مرحله قبل است.

∝ نکته: به ازای هر یک فراخوانی (call) از تابع انجام می‌گیرد.

جلسه سوم - ۱۳۹۹/۱۲/۱۰

مثال از MapReduce : مسئله word Counting

می‌خواهیم در یک فایل متنی بزرگ تعداد دفعات تکرار هر کلمه متفاوت را بشماریم

ورودی تابع Map باید زوج کلید-مقدار باشد، مثلا کلید می‌تونه اسم chunk (آدرس بیتی کامل یک chunk) باشد و value می‌تواند محتوای این chunk باشد.

Scanned with CamScanner

کار تابع Map: chunck را کلمه به کلمه بخواند و به ازای خواندن هر کلمه یک جفت (یک زوج مرتب) تولید می‌کند که کلید این جفت کلمه‌ای است که خوانده و مقدار این جفت، عدد ۱ است.

بعد تابع group by وارد عمل می‌شود و جفت‌هایی که توسط Call های مختلف تابع Map تولید شده اند را گروه‌بندی می‌کند به طوری که جفت‌هایی که کلیدهای (کلمه‌های) یکسان دارند پشت سر هم قرار بگیرند.

در نهایت تابع Reduce را داریم، ورودی‌اش، خروجی group by است. یک کلید و لیستی از مقادیر (ها) تابع Reduce کارش این است که مقادیر را با هم جمع می‌کند (ها را جمع می‌کند) به ازای هر کلید.

کار user ← تابع map و reduce را بنویسد و فایل ورودی را اجرا کند

کار سیستمی که از این سبک برنامه‌سازی پشتیبانی می‌کند:

روز عرفه (روز نیایش) - روز اهدای خون

۱۰ | مرداد | جمعه

۱۰ ذی الحجه ۱۴۴۱
31 July 2020

① دریافت کردن chunck ها، replication هم انجام دهد (partitioning)

② scheduling: اینکه map task ها، reduce task ها و حتی group by چگونه انجام بگیرد و در چه زمانی چه task ای به کدام یک از Commodity node ها assign بشه

③ group by ← مثل bottle neck می‌ماند است. چون روی کل دیتا انجام می‌گیرد. در مراحل دیگه مثل map یا reduce میایم دیتا رو chunck chunck می‌کنیم روی هر قسمت دیتا هر call مورد می‌گیرد اما در groupby میاد خروجی همه‌ی map call ها را می‌گیرد و بعد روی آن گروه‌بندی را انجام می‌دهد

④ تشخیص و هندل کردن خرابی‌ها (failures) ← تشخیص بدهد اگر نودی خراب شده باشد و اگه نمید نودی خراب شده بیاد اونو هندل کنه. اجازه ندهد دیتا از دست برود و اینکه برداشتی که روی آن نود در حال اجرا بوده را دوباره اجرا کند.

عید سعید قربان (تعطیل)

Scanned with CamScanner

۵) Communication ← ارتباط بین process های مختلف را انجام بدهد. هر task از reduce task ها یا map task ها خروجی خودش را دارد. مثلا نود master باید بداند خروجی هر کدام از ماشین‌ها نوشته شده اند. به طور خلاصه ارتباطات در این سبک از طریق نوشتن در فایل یا خواندن فایل صورت می‌گیرد.

Data Flow (جریان داده)

از فایل سیستم توزیع شده استفاده می‌کنیم (distributed File system)

محیط باید از یک فایل خیلی بزرگ یک فایل سیستم توزیع شده بسازد به صورت چانک چانک و اینکه هر چانک replicate بشه.

نتایج میانی روی local FS در کامپیوترهای commodity نوشته می‌شود و ارتباطات هم از طریق این نتایج میانی صورت می‌گیرد.

یک map task توسط یک کامپیوتر صورت می‌گیرد و خروجی اش را روی FS کامپیوتری می‌نویسد و آدرس خودش را به نود coordinator می‌فرستد. نود coordinator که حالا این اطلاعات را دارد در مرحله بعدی مثلا group by یا reduce task اگر این اطلاعات را نیاز داشته باشد در اختیار آن قرار می‌دهد.

حالت‌های task: idle ← هنوز اجرایش شروع نشده

in-progress ← اجرا شروع شده و به یکی از نودهای commodity assign شده اما هنوز اجرای آن تمام نشده

Completed ← اجرای آن map/reduce task تمام شده

و آن نود اطلاعات مربوطه خروجی را به نود coordinator فرستاده.

task های idle به محض اینکه یکی از نودها در دسترس باشند بنا به اولویت‌شان به آن ارسال می‌شوند

کشف کردن failure ها به این صورت است که نود Coordinator به طور مرتب نودها را ping می‌کند اگر جواب ندادند و دچار شکست شده یا شنگار و اگذار شده به آن‌ها در صف قرار می‌گیرد تا به نود دیگری داده شود.

Scanned with CamScanner

سوال: اندازه map task ها و reduce task ها

باید چقدر باشد؟ یعنی اینکه چندتا call از تابع map یا reduce را به عنوان یک task به یک کامپیوتر ارسال کنیم؟

فرض کنید M تعداد map task ها و R تعداد Reduce task هاست.

← در عمل M را یک عدد خیلی بزرگ در نظر می گیریم یعنی تعداد map call هایی که کنار هم قرار می گیرند تا یک map task بسازند یک عدد خیلی کوچکی در نظر می گیریم. تعداد map task ها (M) را خیلی بزرگتر از تعداد نودها در نظر می گیریم. معمولا هر chunck متناظر با یک map task در نظر گرفته می شود

← R را معمولا عدد کوچکی در نظر می گیرند. چون هر reduce task یک فایل خروجی تولید می کند اگر تعداد reduce task ها بزرگ باشد بعدا برای جمع آوری این فایل ها و تولید خروجی نهایی دردسر داریم.

فواید بزرگ در نظر گرفتن M:

: fine granularity (درجه بندی) برنامه ای موازی ما را خیلی ریز در نظر می گیریم. فواید granularity

① زمانی که برای بازیابی و recover کردن نود از خرابی صرف می کنیم کمتر می شود. زمانی که failure زیادی در سیستم داریم بهتر است تسک هایی که قرار است موازی اجرا شوند را کوچک در نظر بگیریم

② درجه ی موازی سازی را بالاتر می توانیم ببریم. مثلا می توانیم مرحله group by را موازی با map task ها انجام بدهیم. اگر map task خیلی بزرگ باشد و به تعداد از نودها assign شده اند و تعدادی از نودها خالی مانده اند بدون هیچ task ای که به آن ها assign شده باشد. باید خیلی صبر کنیم تا map task های بزرگ تقدم بشن زمانی که تعدادی از گره ها بیکار بوده اند. اما اگر map task ها را کوچک در نظر بگیریم اولا نودی خالی نمی ماند دوما اگر نودی خالی ماند map task های ناتمام بعدی می توانیم group by را شروع کنیم.

Scanned with CamScanner

۱۳۹۹

④ dynamic load balancing ← اگر بتوری کند شدنی توانیم load اش را پایین بیاوریم

map task بزرگتری به آن بدهیم یا اصلا به آن task ندهیم اما اگر خیلی بزرگ باشند map task

امکان انجام این کار نیست.

Refinements: Combiners

فرض کنیم می خواهیم یک لیست خیلی بزرگ را مرتب کنیم که از اجتماع یک سری زیرلیست ساخته شده

برای افزایش سرعت به ران ایده که هرکدوم از زیرلیست ها را مرتب کنیم و بعد آن ها را با هم ادغام کنیم.

در map reduce هم چنین کاری ممکن است:

هرکدام از map task های ما بیان group by بشن و بعد در یک فایل نوشته بشوند

یک مرحله جلوتر: نه تنها group by بشن بلکه تابع reduce هم روی خروجی خیلی خودشان

اعمال بشود (البته اگر تابع reduce این اجازه را به ما بدهد)

(مثال اسلاید هم)

مسائلی که مناسب هستند برای Map Reduce

مثال ۱: فرض کنید یک corpus خیلی بزرگ داریم،

URL, size, date, ...

↓

هر رکورد این log file از این اجزا تشکیل شده

می خواهیم تعداد دفعاتی که هر url فراخوانی شده را پیدا کنیم ← با این سبک قابل ادامه

هر دری که از اینترنت استفاده می کند به ازای فراخوانی هر url یک رکورد اضافه می شود

مثال 2: الگوریتم page rank ← یک پیاده سازی مشهورش با Map Reduce وجود دارد

مثال 3: در ML مثلا kmeans

مثال 4: در statistical ML مثلا در query processing، مثلا نیاز داریم همه توالی های n تایی

از کلمات را بشماریم.

Scanned with CamScanner

مثال 5: join کردن دیتا جدول

A	B
a_1	b_1
a_2	b_1
a_3	b_2
a_4	b_3

R

⋈

B	C
b_2	c_1
b_2	c_2
b_3	c_3

S

=

A	C
a_3	c_1
a_3	c_2
a_4	c_3

ورودی تابع map :

ورودی ما : یا یک ردیف از جدول R یا یک ردیف از جدول S

اگر یک ردیف از جدول R بود : $R(a,b) \xrightarrow[\text{key_value}]{\text{pair}} (b,(a,R))$

" " " S بود : $S(b,c) \longrightarrow (b,(c,S))$

بعد تابع group by صدا زده می‌شود و همه جفت‌هایی که کلید یکسان دارند کنار هم قرار می‌گیرند

ورودی تابع Reduce : کلید آن b است، مقدار آن (a,R) یا (c,S) می‌تواند باشد

تابع Reduce به ازای هر مقدار (a,R) یا (c,S) به دستگاهی مرتب (a,b,c) به عنوان یک ردیف از جدول نتیجه تولید می‌کند.

بین همه مسائل Map Reduce این ویژگی مشترک است که: دیتا را به صورت sequential (ترتیبی - رکورد به رکورد) می‌خوانیم.

اما جاهایی که این اتفاق نمی‌افتد مثلا مجبور باشیم interactive کوئری بزنیم، به قسمت از data را می‌خواهیم مستقیم دسترسی می‌شود برحسب آن قسمت بعدی دیتا را می‌خوانیم در این جا استفاده از Map Reduce جواب خوبی نمی‌دهد. یا مثلا خواندن دیتا و برداشتن به صورت real time

مثلا دسته‌ای از مسائل گرافی نودها به صورت ردیفی ذخیره می‌شود، مثلا page rank این خاصیت را دارد و می‌شود از map reduce استفاده کرد. اما خیلی از مسائل گرافی هستند که jump می‌کنید از یک نود به نود دیگر نمی‌توان از map Reduce بهره گرفت

Scanned with CamScanner

همین طور برای دیتاهایی که interdependency بینشان وجود دارد. رکورد بعدی که می خواهید بخوانید بستگی دارد به دیتایی که الان می خوانید. ← Map Reduce خوب نیست

هم چنین مثل هایی که لازمه زوج های زیادی را با هم مقایسه کنیم مثل Sort ← خوب جواب نمی دهد

تحلیل الگوریتم های Map Reduce

۴ معیار

① Communication cost : هزینه ارتباطات بین واحدهای پردازشی مختلف. از طریق I/O ارتباطات صورت می گیرد. یک task خروجی اش را در یک فایل می نویسد و به تسک دیگه میاد از روی فایل ورودی اش را می خواند. میزان I/O ای که انجام گرفته را محاسبه می کنیم در این معیار.

② Elapsed Communication cost :

یک execution path به ازای هر ورودی داریم. هر ورودی یک زوج مرتب کلید-مقدار است که توسط تابع map مورد استفاده قرار می گیرد و یک مسیر اجرایی برایش دنبال می شه تا در نهایت خروجی تولید بشه. در این معیار max I/O ای که بین ورودی های مختلف مورد استفاده قرار گرفته را محاسبه می کنیم.

③ (Elapsed) computation cost شبیه به قبلی ها فقط زمان رو حساب می کنیم در این جا

Communication cost :

(اندازه کل خروجی های map task ها) ۲x + اندازه خروجی های تمام reduced task + اندازه فایل ورودی

چون یک بار توسط map task ها در فایل نوشته می شوند و یک بار هم توسط reduce task ها از فایل خوانده می شوند. (مجموعا توسط دو نود مختلف)

Scanned with CamScanner

جفت های کلید-مقدار ورودی را در نظر می گیرید. این هزینه را برای هر map task محاسبه می کنیم:

① cost ای که باید آن جفت را بخوانید

② cost مربوط به I/O ضربدر ۲

③ هزینه reduce call که برای خروجی های map task مربوطه آن ورودی ایجاد می کند

↓

این ها را جمع می کنیم و بعد بین همه ورودی ها این مقدار جمع را max می گیریم.

Frequent pattern Mining & Association Rules

یک مثال: سبد خرید

هدف: آیتم هایی که باهم خریده می شوند را شناسایی کنیم

تعداد آیتم ها خیلی زیاده، تعداد خیلی زیادی ترانکشن داریم، تعداد آیتم هایی که در هر ترانکشن وجود دارد خیلی کم است.

تشکیل جهاد دانشگاهی

association rules:

{milk} → {coke}

{Diaper, Milk} → {Apple}

جلسه چهارم - ۱۳۹۹/۱۲/۱۲

یک کاربرد از large itemset ~~pattern mining~~:

آیتم های ما همان کالاها هستند

Basket های ما ← مجموعه کالاهایی که یک نفر باهم دیگه در یک بار مراجعه به فروشگاه خریداری می کند

این دیتا باهم جمع می شود و خیلی می شود و یک مجموعه داده بزرگ را تشکیل می دهند.

روز خبرنگار

Scanned with CamScanner

مثال دیگر: هر مجموعه یک Basket ، هر آیتم دیگر یک item باشد.

اگر دایرکتوری دارای آن N2 باشد مقدارش در ماتریس ۱ می‌شود در غیر اینصورت ۰.

یک large itemset در این مثال چه چیزی را نشان می‌دهد؟

یعنی گروه‌ای از دایرکتوری‌ها که تعداد زیادی از جلسات را همشون دارند ← برای کشف تقلب علمی

در واقع این گروه از دایرکتوری‌ها کپی همدیگه هستند.

کاربرد دیگه: بررسی اثرات دارو‌ها

Baskets ← هر تراکنش را یک بیمار در نظر بگیرید و هر item را یک دارو یا یک side-effect

(اثر جانبی دارو) در نظر بگیرید

بدان هر بیمار حتی اگر دارو را مصرف کرده ۱ می‌گذاریم در غیر اینصورت ۰، اگر یک اثر جانبی خاص

را دارد ۱ می‌گذاریم در غیر اینصورت ۰.

یک large itemset چه معنی‌ای می‌دهد؟ یعنی تعداد زیادی از بیمارها این دارو‌ها را مصرف

کرده اند این side effect ها را دارند. پس نتیجه می‌گیریم کدام دارو‌ها کدام عوارض را دارند.

به صورت کلی یک many-to-many mapping داریم چون یک آیتم می‌تونه در چند تراکنش باشه و یک تراکنش

می‌تونه شامل چند آیتم باشه. اما در نهایت ما می‌خواهیم ارتباط بین آیتم‌ها را پیدا کنیم.

تعریف دقیق مسئله:

به صورت خیلی پایه: یک مجموعه‌ای از تراکنش‌ها داده شده مسئله این است که مجموعه‌هایی از

آیتم‌ها که در این مجموعه تراکنش‌ها به صورت پرتکرار (Frequent) اتفاق افتاده اند پیدا کنیم.

به صورت پرتکرار اتفاق افتاده اند به چه معناست؟ یعنی ۲ تا کار باید انجام بدهیم

عید سعید غدیر خم (۱۰ هـ ق) (تعطیل)

① Frequency یا ساپورت یک آیتم ست را تعریف کنیم

② یک treshold تعریف کنیم و این مجموعه آیتم‌هایی که Frequency یا ساپورتشان از این threshold بالاتر

هست large itemset محسوب می‌شوند.

Scanned with CamScanner

برای یک itemset به روش های مختلفی می توان frequency یا ساپورت را تعریف کرد. ساده ترین: بیاییم تعداد تراکنش هایی که این itemset را در دارند به عنوان frequency یا ساپورتِ این itemset در نظر بگیریم.

گاهی اوقات به itemset ، pattern هم می گویند.

به itemset ای که frequency یا ساپورت آن از Threshold ما بزرگتر مساوی باشد large itemset می گوییم (frequent pattern هم گفته می شود)

خیلی از وقت ها frequency را به صورت تعداد بیان نمی کنیم بلکه به صورت یک نسبت بیان می کنیم به این صورت که تعداد تراکنش هایی که این itemset را دارند تقسیم می کنیم بر تعداد کل تراکنش ها

هر itemset در هر تراکنش حداکثر 1 بار می تواند اتفاق بیفتد. (مثال اسلاید ششم)

روابط وابستگی (Association Rules)

$$\{i_1, i_2, \dots, i_k\} \longrightarrow j$$

اگر Basket های ما آیتم های i_1 تا i_k را داشته باشند خیلی محتمل است که آیتم j را هم داشته باشند. ← نتیجه: آیتم j به آیتم های i_1 تا i_k وابستگی پیدا می کند

در عمل ممکن است روابط وابستگی خیلی زیادی را در یک مجموعه از داده ها پیدا کنیم ولی می خواهیم آن هایی را پیدا کنیم که برای ما مهم و جالب هستند و از یک درجه خاصی اهمیت بالاتری داشته باشند.

یک معیار خیلی رایج برای اهمیت association rule:

$$Conf(I \rightarrow j) = \frac{support(I \cup j)}{support(I)}$$

سوال: چرا اگر یک rule دارای confidence بالاتری باشد برای ما مهم تره؟ خاصیت monoton بودن: چون $I \subset I \cup j$ ، در هر تراکنش که $I \cup j$ اتفاق افتاده I هم اتفاق افتاده پس یعنی $support(I \cup j) \leq support(I)$

Scanned with CamScanner

پس این بست (conf) عواره کوچکتر مساوی ۱ است. هر چقدر این بست بزرگتر باشد یعنی حاهایی که I اتفاق افتاده اما j با I اتفاق نیفتاده کمتر است. یعنی j با آیتم هایی که در I هستند داره اتفاق می‌افته و j وابستگی قوی با آیتم هایی که در I هست دارد.

سوال: آیا همیشه conf خوب جواب می‌دهد؟ چه چیز وجود دارد که این معیار خیلی خوب وابستگی j به آیتم های موجود در I را نشان ندهد؟

حالتی که j ما آیتمی باشد که در خیلی از تراکنش ها وجود دارد و خیلی وابستگی به آیتم های I ندارد و یک آیتمی است که کلا در اکثر basket ها موجوده. در این صورت conf بالا به دست می‌آید درحالی که بین j و آیتم های I وابستگی وجود ندارد.

برای حل این مشکل یک معیاری تعریف کرده اند به نام Interest

$$Interest(I \to j) = conf(I \to j) - Pr[j]$$

$$Pr[j] = \frac{\text{تعداد تراکنش هایی که آیتم } j \text{ را دارند}}{\text{تعداد کل تراکنش ها}} = \text{احتمال ظاهر شدن } j \text{ در تراکنش های ما}$$

وقتی j آیتمی باشد که اکثر اوقات با I می‌آید ولی اینطور نیست که کلا خیلی پرتکرار باشد، در این حالت Pr[j] کوچک می‌شود در نتیجه معیار Interest مقدار بالایی دارد.

درحالتی که Interest خیلی منفی باشد به این معناست که j با I ارتباط منفی دارد

تعریف مسئله Association Rule Mining:

مجموعه‌ای از basket که شما داده اند و از شما خواسته اند rule هایی را پیدا کنید که:

① سابپورتشان از یک threshold ای بالاتر است. (s)

② conf آن ها از یک threshold دیگری بالاتر است.

ولادت امام موسی کاظم (ع) (۱۲۸ هـ ق)

Scanned with CamScanner

سوال: ساپورت یک association rule چیست؟

ساپورت احتماع عناصر سمت چپ و راست

الگوریتم‌های ما برای پیدا کردن frequent itemset به طور کلی این کارها را می‌کنند:

① مجموعه‌های نامزد frequent itemset ها را پیدا می‌کنند

② به ازای هر frequent itemset ای مثل I که پیدا شده تمام افرازهای I به دو مجموعه به طوری که هیچ‌کدام از این دو مجموعه‌ی تهی نباشند را در نظر می‌گیرند. به ازای هر افراز دو تا rule تشکیل می‌دهند یک بار افراز اول سمت چپ افراز دوم سمت راست و یک بار هم برعکس.

به ازای هر rule مین conf آن را در نظر می‌گیرند اگر conf آن بزرگتر c بود این rule یک rule دکوان است.

bottleneck ما مرحله اول است که قرار است large itemset ها را بیابیم.

برای مرحله دوم trick هایی هم وجود دارد. مثلا اگر یک rule به شکل زیر داشته باشیم:

$A, B, C \rightarrow D$

اگر conf برای این rule از threshold کمتر باشد هر rule دیگری که سمت راست آن superset ای از D باشد هم قطعا conf اش از threshold کمتر است

مثلا: $A, B \rightarrow C, D$

$I_1 \rightarrow I_2$ ، $I'_1 \rightarrow I'_2$

اگر $Conf(I_1 \rightarrow I_2)$ از threshold کوچکتر باشد و I'_2 ابر مجموعه‌ای از I_2 باشد در این حالت حتما $Conf(I'_1 \rightarrow I'_2)$ هم از threshold کمتر است.

threshold معمولا توسط کاربر تنظیم می‌شود.

نکته: با کاهش تدریجی threshold به نقطه‌ای می‌رسیم که تعداد large Itemset ها به صورت انفجاری افزایش پیدا کند. اصطلاحا گفته می‌شود که phase transition داریم. بنابراین به حالتی می‌رسیم

روز حمایت از صنایع کوچک

Scanned with CamScanner

۱۳۹۹

که روی دارد pattern ها خیلی زیاد می شود. هم خود الگوریتم و آن ها گیر می افتد و هم کاربری که می خواهد از این خروجی استفاده کند مشکل پیدا می کند و نمی تواند این حجم عظیم را پردازش کند. راه حل: یک compact representation از همه ی large Itemset ها داشته باشیم. یعنی مجموعه ای داشته باشیم که همه ی large Itemset ها را با حجم و تعداد خیلی کمتر نشان بدهد

large Itemset ها: ① Maximal ② closed

Maximal Itemset: آیتم ست هایی هستند که هیچ ابرمجموعه ای ندارند که پرتکرار باشد. یعنی هیچ مجموعه ای نیست که آن ها را در بر داشته باشد و آن هم پرتکرار باشد. انگار آیتم ست های Maximal یک مرزی ایجاد می کنند که در زیر آن آیتم ست های پرتکرار قرار دارند و در بالای آن آیتم ست های غیر پرتکرار و روی خود مرز هم که Maximal هستند.

نکته: مشکل آیتم ست های Maximal این است که ما frequency یا ساپورت آیتم ست ها را از دست می دهیم.

closed Itemset: یک آیتم ست، closed است اگر و تنها اگر هیچ ابرمجموعه ای نداشته باشد که دارای ساپورتی برابر با ساپورت آن آیتم ست باشد. بنابراین اگر I' ابرمجموعه ای از I باشد، وقتی از I به I' می رویم مقدار ساپورت کم می شود.

سوال: فرض کنید مجموعه ی تمام الگوهای closed به همراه frequency آن ها به ما داده شده، آیا از روی آن می توانیم تمام الگوهای پرتکرار را پیدا کنیم؟

نکته: در تعریف Maximal، آیتم ست باید پرتکرار باشد. در تعریف closed که در این جا مطرح شد شرط پرتکرار بودن ندارد اما یک تعریف دیگر از closed وجود دارد که شرط پرتکرار بودن هم دارد.

نکته: با داشتن آیتم ست های Maximal می توانیم تمام Fraquent Itemset ها را به دست آوریم ولی ساپورت آن ها را از دست می دهیم. در واقع هر Maximal Itemset ها و تمام زیرمجموعه های آن ها، الگوهای پرتکرار هستند.

سوال: آیا هر الگوی Maximal یک الگوی closed است؟

آیا هر الگوی closed یک الگوی Maximal است؟

Scanned with CamScanner

مثال:

	support	Maximal (S=3)	closed
A	4	NO	NO
B	5	NO	yes
C	3	NO	NO
AB	4	Yes	yes
AC	2	NO	NO
BC	3	yes	yes
ABC	2	NO	yes

اگه تعریف closed رو برعکس بگیریم، ABC، closed خواهد بود.

Maximal ها: AB، BC ← (تولید الگوهای پرتکرار) A، B، C، AB، BC

فرض کنید closed های پرتکرار به همراه ساپورت آن ها به ما داده شده:

AB: 4 B: 5 BC: 3

آیا می توانیم تمام الگوهای پرتکرار دیگر و ساپورت هرکدام از آن ها را پیدا کنیم؟

بله می توانیم. به ازای تک تک الگوهای بسته ی پرتکرار که داریم، همه ی زیرمجموعه های آن را تولید می کنیم تا حالی که به خود الگوی بسته برسیم. اگر این زیرمجموعه هایی که تولید شده اند خودشان closed (بسته) بودند که frequency شان را داریم. اگر بسته نبودند به اولین اَبَرمجموعه ای که closed هست آن را map می کنیم و ساپورتش را برابر با اولین اَبَرمجموعه بسته می شود. اولین یعنی نزدیکترین مثلا برای A ممکنه AB و ABC را داشته باشیم اما AB را در نظر می گیریم چون به آن نزدیکتر است.

نکته: ها از روی الگوهای پرتکرار بسته می توانیم ساپورت زیرمجموعه های کل الگوهای پرتکرار را پیدا کنیم در نتیجه اگر الگوهای بسته را همراه با frequency آن ها پیدا کنیم هیچ اطلاعاتی در مورد الگوهای پرتکرار از دست نمی دهیم.

به همین دلیل به مجموعه الگوهای closed، Complete یا کامل گفته می شود.

Scanned with CamScanner

Itemsets: Computation Model

چگونه large Itemset ها را ذخیره و محاسبه کنیم؟

حالت typical این است که تراکنش ها را در یک دیتابیس Relational ذخیره می کنیم

text file هایی داریم که اطلاعات در آن ذخیره می شود، در ابتدای هر فایل یک سری اطلاعات کلی

در مورد فایل داریم و بعد در هر line یک تراکنش ذخیره می شود. هر خط شامل یه سری اطلاعات

در مورد تراکنش مثل time و... است و بعد آیتم های آن تراکنش می آیند.

یک الگوریتم خیلی سطح بالا برای پیدا کردن large Itemset ها:

می توانیم سطح به سطح پیش برویم. سطح ۰ ام ← ϕ

سطح ۱ ام ← آیتم ست هایی که ۱ آیتم دارند

سطح 2 ام ← هر pattern ها از ۲ آیتم تشکیل شده

⋮

سطح K ام ← هر pattern از K تا آیتم تشکیل شده

در هر سطح کاندیدها را از روی مجموعه آیتم ست های پرتکرار سطح قبلی تولید می کنیم

جلسه ۵ - ۱۳۹۹/۱۲/۱۷

هر الگوی maximal یک الگوی closed است اما برعکس آن درست نیست چون ممکنه

یه الگو closed باشه و اثر pattern ای داشته باشه که frequency اش کمتر از خود الگو باشد اما

پرتکرار باشد.

محدودیت های ما در پیدا کردن large Itemset ها: ① می خواهیم تعداد I/O ها را کم کنیم، تعداد دفعاتی که فایل را

می خوانیم کم کنیم. به هر خواندن فایل در ورودی یک scan یا pass گفته می شود. بنابراین یکی از اهداف ما کم کردن تعداد pass ها است

روز خانواده و تکریم بازنشستگان برای مقایسه الگوریتم ها هم یکی از معیارها تعداد pass است. ② خیلی وقت ها حافظه کافی در دست

نیست که بتوانیم کل دیتا را در حافظه load کنیم.

در خیلی از داده های دنیای واقعی سخت ترین مرحله سطح دوم است که pattern ها با اندازه ۲ را پیدا می کنیم. چون تعداد پترن ها با اندازه ۲

خیلی زیاد است و یکی از چالش های اصلی است. اگر بتوانیم سطح ۲ را به خوبی هندل کنیم سطوح بالاتر را هم می توانیم هندل کنیم.

۲۶ | مرداد یکشنبه | ۲۶ ذی الحجه ۱۴۴۱ | 16 August 2020

جلسه پنجم - ۱۳۹۹/۱۲/۱۷

هر الگوی maximal یک الگوی closed هم هست اما هر الگوی closed لزوماً یک الگوی maximal نیست

دومین اصلی در large Itemset Mining : (برای هر Itemset دو تا کار انجام می دهیم)

① تولید کاندیدا ها برای pattern بودن در سطح k+1 (candidate generation)

② Frequency counting ← برای تمام کاندیدها support را حساب کنیم و آنهایی که از min support بیشتر مساوی باشن نگه میداریم.

مرحله اول به صورت efficient کار می کند و خیلی هزینه ای ندارد اما مرحله دوم هزینه بر است ↗ بایکل دیتاست را بگذریم

به کاری که برای اجرایش کار این می توان انجام داد: از تولید کاندیدهایی که حدس می زنیم پرتکرار نیستند پرهیز کنیم.

مثلا نیستی از آیتم ست های غیر پرتکرار نگه داریم و اگر کاندیدی تولید شد که یک آیتم پرتکرار در آن نبود قبل از فاز Frequency حذفش کنیم

Naive Algorithm (برای یافتن الگوهای پرتکرار با اندازه ۲)

فایل را می خوانیم، هر تراکنش به طول n را که می خواند $\frac{n(n-1)}{2}$ زوج آیتم دارد که count را برای هر کدام از این زوج ها یکی زیاد می کند.

$O(n^2)$ ← (n تعداد آیتم ها)

مشکل این روش: حافظه زیاد ← مقدار حافظه quadratic است برحسب تعداد آیتم ها

مزیت: یک بار اسکن انجام میدهید ← I/O یک بار pass را طی می کند

① به ازای برای ذخیره سازی زوج ها: روش ماتریسی ← n تای داریم، یه ماتریس بایین مثلثی می سازیم که فقط counter ها رو ذخیره می کنیم. خانه (و، ن) ماتریس آیتم ست دوتایی {و، ن} را راستان می دهد

② روش دیگه: نگهداری سه تایی (ج، و، ن) ← آیتم ست دوتایی {و، ن}، ج بار تکرار شده

مقایسه:

مزیت روش اول ← خود آیتم ست های دوتایی را ذخیره نمی کنیم

" " دوم ← برای دوتایی هایی که وجود ندارد حافظه ای اشغال نمی کند

روش اول برای زمانی مناسبه که دوتایی های زیادی داشته باشیم و روش دوم زمانی که دوتایی ها کم باشند.

آغاز بازگشت آزادگان به میهن اسلامی (۱۳۶۹ هـ ش)

Scanned with CamScanner

۱۳۹۹

روش ماتریسی: آدرس زوج $\{i,j\}$ در ماتریس: $j-1+\frac{(i-1)(n-i)}{2}$

مقدار کل خانه‌هایی که نیاز داریم: $2n^2$ (بایت) $\frac{n(n-1)}{2}\times 4$

روش سه‌تایی: ← هردوی این روش‌ها معروف حافظه quadratic دارند بحسب تعداد آیتم‌ها

زمانی خوب است که آیتم‌ست‌های دوتایی که حداقل ۱ بار در دیتابیس آمده‌اند، حداکثر $\frac{1}{3}$ برابر تعداد حالت‌های ممکن $\left(\frac{n(n-1)}{2}\right)$ باشد.

بهتر از روش اول

الگوریتم A_priori

مصرف حافظه را پایین می‌آورد اما هزینه اضافی‌اش این است که ۲ بار باید دیتابیس را scan کنیم برای یافتن حالت‌های پرتکرار و به طور کلی برای scan تا k برای آیتم‌ست k تایی پرتکرار.

ایده اصلی: خاصیت monoton بودن

فرض کنید یک پترن I داشته باشید، همه‌ی آیتم‌ست‌هایی که یک ابرپترن از I باشند ساپورتشان از I کم‌تر است ← خاصیت monoton بودن pattern ای

یا مسازی

لـ پس آیتم‌ست دوتایی اگر تراست پرتکرار باشد هردو عضویش باید پرتکرار باشند

① در pass اول دیتابیس را می‌خوانیم و frequency را برای آیتم‌ست‌های یه دونه‌ای حساب می‌کنیم

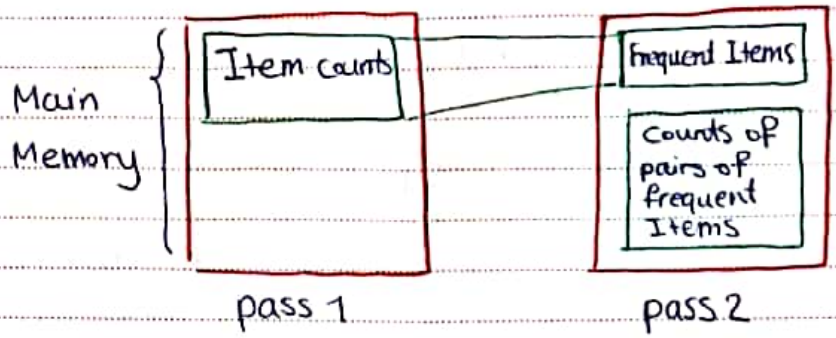
بعد میایم این آیتم‌های پرتکرار را نگه می‌داریم و بقیه را کنار می‌گذاریم

② در pass دوم، به ازای خواندن هر تراکنش آیتم‌ست‌های دوتایی‌اش را می‌یابیم و اگر هردو عضویش پرتکرار بودند counter آن زوج را یکی زیاد می‌کنیم.

اگر تعداد آیتم‌های تکی پرتکرار n' باشد آن‌گاه حافظه مصرفی n'^2 است.

چون تعداد آیتم‌های تکی پرتکرار (n') از تعداد کل آیتم‌ها (n) کمتر است حافظه کمتری مصرف می‌شود.

Scanned with CamScanner



حافظه معرفی شده سمت بالایی در pass2 بهتر از pass1 است چون فقط آیتم‌های پرتکرار را نگه می‌داریم و آیتم‌های غیر پرتکرار را حذف می‌کنیم. نکته دیگر هم اینکه در شکل سمت راست دیگر نیازی به نگه داشتن سایپورت یا counter مربوط به آیتم ست‌های تک‌آیتمی هم نداریم. کافیه فقط بدونیم کدام پرتکرار است.

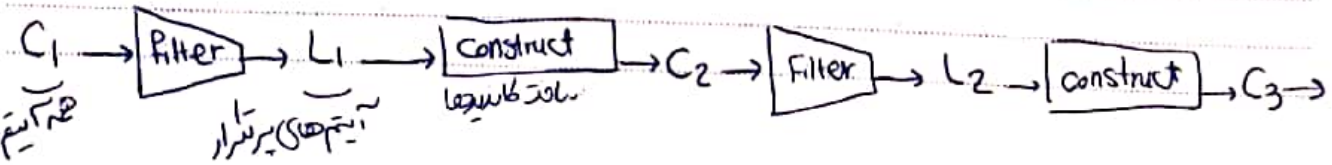
نکته: در pass1 از حافظه به خوبی استفاده نشده و این یک عیب برای این الگوریتم است

آیا از روش لیستی می‌توان استفاده کرد؟ بله. به هر ترانزکشن یک لیست آیتم‌های غیر پرتکرار را کندی کنیم و همه‌ی دوتایی‌های ممکن با آیتم‌های پرتکرار را می‌سازیم و count آن را زیاد می‌کنیم

آیا از روش ماتریسی می‌توان استفاده کرد؟ خیر چون فرض بر این بود که همه‌ی دوتایی‌های ممکن را داریم اما این جا آیتم‌های غیر پرتکرار را کنار می‌گذاریم اما اگه یک تغییر بدهیم می‌توان استفاده کرد. اینکه دوتایی آیتم‌های پرتکرار پیدا شدند آن‌ها را Renumber کنیم و ماتریس را با آن‌ها بسازیم. بین یک mapper داریم که رشته‌ی قدیم آیتم را به شماره‌ی جدید map می‌کند

الگوریتم فرآیند را ادامه بدهیم یک الگوریتم سطح به سطح خواهیم داشت. در سطح K+1 فرض می‌کنیم الگوهای پرتکرار سطح K ام را پیدا کرده‌ایم و از روی آن ابتدا خواهیم کاندیدهای سطح K+1 را تولید کنیم و بعد برای هرکدام از این کاندیدها frequency counting انجام بدهیم.

C_k ← لیست کاندیداهای سطح K ام
L_k ← کاندیداهایی که پرتکرار frequency counting معلوم شود پرتکرارند



Scanned with CamScanner

Candidate generation برای سطح k+1 :

هر الگوی پرتکراری که در سطح k ام است با هر الگوی پرتکراری که در سطح 1 است و داخل الگوی k تایی وجود ندارد باهم ترکیب می‌شوند.

مثال:

$$C_1 = \{\{b\}\{c\}\{j\}\{m\}\{n\}\{p\}\}$$

prune $\rightarrow \{b, c, j, m\}$

$$C_2 = \{\{b,c\}\{b,j\}\{b,m\}\{c,j\}\{c,m\}\{j,m\}\}$$

$$L_2 = \{\{b,m\}\{b,c\}\{c,m\}\{c,j\}\}$$

$$C_3 = \{\{b,c,m\}\{b,c,j\}\{b,m,j\}\{c,m,j\}\}$$

در این روشی که برای تولید کاندیداهای k+1 تایی گفته شد، کاندیداهای تکراری هم تولید می‌شوند. برای efficient شدن این راه حل این کار را این‌گونه انجام می‌دهیم: ① آیتم‌های درون هر آیتم ست را مرتب (sorted) تولید کنیم ② یک آیتم‌ست k تایی را با یک آیتم ادغام می‌کنیم اگر و تنها اگر آن آیتم، از بزرگترین آیتمی که در آیتم ست ما هست بزرگتر باشد. مثلا در مثال بالا $\{b,m\}$ را با c، join نمی‌کنیم چون c از m بزرگتر نیست.

این روش هیچ کاندیدای valid ای را از دست نمی‌دهد و هیچ کاندیدایی را دوبار نمی‌شمارد

یک روش بهتر برای تولید کاندیدا که از تولید کاندیدهایی که در آینده پرتکرار نخواهند بود جلوگیری می‌کند:

دو تا pattern پرتکرار با اندازه k را با هم ادغام می‌کنیم و یک pattern کاندید k+1 تایی تولید می‌کنیم

سوال: کدام دو تا pattern پرتکرار k تایی را می‌توان با هم ادغام کرد؟ هر دو تا pattern پرتکرار با اندازه k که k-1 تا آیتم مشترک دارند می‌توانند با هم ادغام شوند: $(k-1)+2=(k+1)$

Scanned with CamScanner

در حالت کلی الگوریتم های apriori که به صورت level wise کار می کنند برای هر سطح نیاز به یک اسکن دارند

extensions

۱- برای مقادیر عددی باز در نظر بگیریم. آیتم ها دوتایی نباشند. در این صورت categorical می کنیم. مثلا سن فرد ۶۰ تا ۷۰ را یکی در نظر بگیریم

numerical value داشته باشیم

۲- association rule هایی رو برای زیر مجموعه ها استخراج کنیم:

وقتی که آیتم ها در یک taxonomy هستند

Bread, Butler → FruitJam

استفاده می شود تا توانایی داشتن

BakedGoods, MilkProduct → PreservedGoods

چیزی را استخراج کنیم

۳- دیتاهای بالا min sup را کم کنیم، چون الگوهای پرتکرار همین دیتاهای بالا داشته باشیم که به خاطر بالا بودن threshold آن ها را از دست بدهیم. معمولا pattern های بزرگ جزئی معنیدار هستند.

Scanned with CamScanner

جلسه ششم ۱۳۹۹/۱۲/۱۹

الگوریتم a priori در scan اول خودش از حافظه به خوبی استفاده نمی کند و مقدار زیادی از حافظه بی استفاده می ماند چون تعداد آیتم های تک پرتکرار خیلی زیاد نیست.

الگوریتم PCY ← در اسکن اول سعی دارد که بهتر و مفیدتر از حافظه استفاده کند

در pass اول علاوه بر اینکه frequency هر آیتم را حساب می کند یک Hash Table هم تعریف می کند تعداد bucket ها یا خانه های این Hash Table تا جایی است که حافظه ما اجازه می دهد. با تعداد آیتم ها رای دادیم و اینکه یک counter برای هر آیتم لازم داریم پس می دونیم چقدر حافظه برای شمارش آیتم ها نیاز داریم. حافظه باقی مانده را هم برای Hash Table درست کنیم که به اندازه حافظه باقی مانده جا بگیرد.

این Hash Table یک آیتم ست دوتایی را می گیرد و آن را map می کند به یکی از باکت ها. در هر باکت خود جفت آیتمی که به این باکت map شده اند نگهداری نمی شود بلکه یک counter نگهداری می شود. این counter به ما می گوید که چند تا آیتم ست دوتایی به این باکت map شده اند. (شبیه کد در اسلایدها هست)

نکته: ما از قبل تعداد آیتم ها را می دانیم حتی قبل از اینکه یک اسکن اولی انجام بدهیم اما تعداد آیتم ست های دوتایی را نمی دانیم و اینکه ما در اینجا فقط آیتم ست هایی که دوتایی هستند حضور دارند را می خواهیم بلکه آن هایی که مقدار شمارشان از یک threshold بیشتر باشد را نیاز داریم.

فرض کنید که ما هر pattern پرتکرار را positive و هر چیز غیر پرتکرار را Negative در نظر بگیریم و فرض کنید الگوریتم ما به این شکل عمل کند بعد از اجرای یک pass، آیتم های به دونه ای رو شمردیم و برای آیتم ست های دوتایی Hash Table را ساخته ایم. بعد از این pass برای تشخیص پرتکرار بودن آیتم ست های دوتایی به این شکل عمل کنیم: به ازای هر باکت متناظر با آن دوتایی در Hash Table مراجعه کنیم و چک کنیم آیا counter اش از minSup بزرگتر مساوی هست یا نه. اگر بود آن را چون پرتکرار در نظر بگیریم و اگر نبود غیر پرتکرار.

روز بزرگداشت ابوعلی سینا - روز پزشک

Scanned with CamScanner

می خواهیم بررسی کنیم این الگوریتم ما آیا FP و FN دارد یا نه.

FP ← یعنی پترن ما پرتکرار نبوده اما توسط الگوریتم پرتکرار معرفی شده. FN یعنی پرتکرار بوده اما الگوریتم گفته غیر پرتکرار

سوال: آیا الگوریتم FN دارد؟ خیر. به ازای یک پترن خاص اگر آن پترن پرتکرار باشد حتما counter آن از minSup بزرگتر مساوی است

سوال: آیا الگوریتم FP دارد؟ بله. ممکنه چند تا پترن به یک بالت map شوند و به ازای حضور هرکدام از آن ها counter یکی زیاد می شود. بنابراین counter ما حاصل جمع حضور پترن های مختلف خواهد بود و اگر بزرگتر از minSup بشود همه ی آن ها را frequent در نظر می گیریم در حالی که ممکنه بعضی یا همه ی آن pattern ها پرتکرار نباشند.

سوال: مشکل FP را چگونه می توانیم حل کنیم؟ یک scan دوم اقدام میکنیم و در کل همه ی الگوهای دودویی که در اسکن اول پرتکرار شناخته شده اند به عنوان کاندیدا در نظر گرفته و frequency واقعی آن ها را شمارش می کنیم. اگر پرتکرار بودند به عنوان پرتکرار معرفی شان می کنیم و اگر frequency شان کمتر از minSup بود نیز پرتکرار ندونسته حذفشان می کنیم.

⇐ نتیجه: این الگوریتم اگر از یک pass استفاده کند یک الگوریتم approximate است که خطا دارد اما اگر از دو تا pass استفاده کند یک الگوریتم exact است و خطا ندارد.

نکته: در این الگوریتم در واقع تعداد کاندیدا ها را کم کردیم با استفاده از Hash کردن. تعداد کاندیداها pass دوم خیلی زیاد بود.

یک پترن دودویی برای pass دوم یک کاندیدا محسوب می شود اگر دو تا اگر: ① هر دو عضو آن frequent باشند

② بالت مربوط به آن پترن، یک بالت پرتکرار باشد.

نکته: یک improvment این است که وقتی می خواهیم از pass اول به pass دوم برویم counter های بالت ها را حذف می کنیم و فقط یک بیت نگه می داریم که مشخص می کند کانتر از minSup بیشتر بوده یا نه.

نکته: حداکثر مقداری که یک counter می تواند داشته باشد بستگی دارد تراکنش هاست. اما الان به بیت برای هر بالت ما به این تعداد بیت نگه داریم بلکه کافی است به اندازه minSup نگه داریم چون ما فقط لازم داریم که کانتر را تا minSup افزایش دهیم و بعد از آن دیگه می دانیم این بالت پرتکرار است.

Scanned with CamScanner

۱۳۹۹

سوال ۵: در این الگوریتم از روش ماتریسی و یا لیستی می توان استفاده کرد؟

روش لیست که همیشه قابل استفاده هست. روش ماتریسی قابل استفاده نمی باشد. چون همه ی زوج ها را نداریم. مثلا ممکنه (۱,۲) کاندیدای Valid باشه و بالکشن پرتکرار باشه اما (۱,۴) بالکشن پرتکرار نباشه و Valid نشه.

نکته: در scan اول این الگوریتم، مقدار زیادی از جفت آیتم ها به یک باکت Map می شود به همین دلیل است که حافظه مصرفی خیلی کمتر از حافظه لازم برای نگهداری تمام جفت آیتم هاست.

تا این جا الگوریتم هایی که گفته شده به ازای هر سطح نیاز به یک scan روی دیتاست داشتند. از این جا به بعد سعی الگوریتم هایی صحبت می کنیم که نیاز به اسکن در هر سطح ندارند. ویژگی مشترک این الگوریتم ها این است که با یک بار یا دو بار scan، همه ی الگوهای پرتکرار تا سطح k ام را پیدا می کنند اما در عوض هزینه ای که می دهیم randomness و دارای خطا بودن جواب است.

Scanned with CamScanner

تا این جا الگوریتم هایی که داشتیم به ازای هر سطح یک scan نیاز داشتیم. حالا سراغ الگوریتم هایی می رویم که این ویژگی را ندارند اما در عوض هزینه ای مانند randomness بودن و دارای خطا بودن است.

Random Sampling (1)

فرض کنید n ترانکنش داریم. k تا ترانکنش یک sample به طول k از داده مان می گیریم و به حافظه اصلی می آوریم. حالا سپس چند بار می خواهیم این k تا sample را scan کنیم اهمیتی ندارد چون در حافظه اصلی هستند و نیاز به خواندن نیست. در مورد انتخاب این k تا ترانکنش روش های مختلفی وجود دارد، توزیع های متقاوتی می توان استفاده کرد، با جایگذاری یا بدون جایگذاری و... حالت ساده: به هر ترانکنش شانس یکسان می دهیم و با جایگذاری

تغییر: min sup را کم می کنیم، ضرب در $\frac{k}{n}$ می کنیم، معادل آن این است که ساپورت هر آیتمست را در $\frac{n}{k}$ ضرب کنیم.

حالت دوم unbiased estimation است یعنی امید ریاضی ساپورت آن برابر با امید ریاضی این در دیتابیس اصلی است.

در این الگوریتم هر دو FN و FP را داریم. مثال از FN:

T_1: A
T_2: B minsup = 1 $\xrightarrow[k=1]{\text{Sample}}$ T_1 = A E_sup(A) = ۲
E_Sup(B) = ۰

پترن B به غلط Negative در نظر گرفته می شود در حالی که در دیتابیس اصلی پرتکرار است

مثال از FP:

T_1 = A
T_2 = B minsup = 2 $\xrightarrow[k=1]{\text{Sample}}$ T_1 = A E_sup(A) = ۲
sup(A) = 1
sup(B) = 1
E_sup(B) = ۰

الان پترن A به غلط پرتکرار در نظر گرفته می شود

برای حل مشکل FP می توان یک بار دیگر scan را انجام بدهیم. کاندید های ما آن پترن هایی هستند که Estimated support آن ها از threshold بیشتر باشد.

روز کارمند

اسکن دوم روی کل دیتاست انجام می‌شود. تعداد کل اسکن‌های ماهی ستور ۲ تا. عکسه در random Sample

براساس Sample هان k تا سطح بالا بردیم. یعنی الگوهای پرتکرار Sample هان را تا سطح k ام

پیدا کنیم و یکی یک اسکن دوم انجام بدهیم و ببینیم این الگوهایی که در سطح‌های مختلف پیدا شده اند کدام‌شان

پرتکرار هستند و اینا

Scanned with CamScanner

۱۳۹۹

برای حل مشکل FN باید threshold را کم کنیم مثلا بجای ضرب کردن در $\frac{k}{n}$، در $\frac{k}{۲n}$ یا $\frac{k}{۱.۵n}$ ضرب کنیم برای اینکه شانس اون patternهایی که در واقع پرتکرار بوده اند اما به اشتباه پرتکرار تشخیص داده نشده اند را افزایش بدهیم.

اگر Resampling داشته باشیم و مثلا ۲ بار sample تا k بگیریم معادل این است که ۲k تا سمپل را ۱ دفعه انتخاب کنیم. هرچقدر سمپل های شما افزایش پیدا کند واریانس به صورت خطی کم می شود.

مثلا با ۲k سمپل واریانس $\frac{1}{2}$ می شود.

SON Algorithm

دیتاست را به گونه ای افرازی کنیم به تکه هایی که هر تکه در حافظه اصلی جا بگیرد و هم چین پترن ها استخراج شده از آن تکه هم در حافظه اصلی جا بگیرد.

minsup هم $\frac{k}{n}$ برابر می شود

کاندیداهای شما اون پترن هایی هستند که حداقل در یکی از این تکه ها پرتکرار باشند ↑

خاصیت مونوتونی پترن: ساپورت یک پترن بزرگتر مساوی ساپورت ابر پترن هاشه

خاصیت مونوتونی دیتاسیی: اگر پترن داشته باشیم که در کل دیتاسیی پرتکرار باشند حداقل در یکی از افرازهای آن دیتاسیی پرتکرار است

اثبات database monotonicity:

فرض کنیم دیتاست را به k کسش شکانده ایم. روی هرکدام از این chunkها pattern extraction را انجام داده ایم و minSup هر chunk هم متناسب با اندازه آن chunk تنظیم شده است. این minSupهای تنظیم شده را $S_1, S_2, \dots, S_k$ می نامیم و minsup اصلی S باشد.

فرض کنید P یک پترن باشد و $sup_i(P)$، ساپورتی است که این پترن در هرکدام

Scanned with CamScanner

از این chunck ها پیدا می‌کند.

می‌دانیم که $sup(p)$ یعنی ساپورت پترن p در کل دیتاست برابر با مجموع ساپورت آن در chunck های مختلف است. یعنی داریم:

$$sup(p) = sup_1(p) + sup_2(p) + \dots + sup_k(p)$$

طبق فرض، p یک پترن پرتکرار در دیتاست اصلی است و می‌خواهیم نشان بدهیم p حداقل در یکی از chunck ها هم پرتکرار است. چون p پرتکرار (در دیتاست اصلی) است داریم:

$$sup(p) \geq s$$

می‌خواهیم نشان بدهیم:

$$\exists i \in \{1, \dots, l\} : sup_i(p) \geq s_i$$

$$\Rightarrow sup(p) \geq s \Rightarrow sup_1(p) + \dots + sup_k(p) \geq \underbrace{s_1 + \dots + s_k}_{\Downarrow} = s$$

$$s\left(\frac{k_1}{n} + \frac{k_2}{n} + \dots + \frac{k_k}{n}\right) = \frac{(k_1 + k_2 + \dots + k_k)}{n} = \frac{n}{n} \times s = s$$

۷ شهریور جمعه | ۸ محرم ۱۴۴۲ | 28 August 2020

اگر حداقل یک i وجود داشته باشد که $sup_i(p) \geq s_i$ آن‌گاه قضیه برقرار است.

برهان خلف: فرض کنیم هیچ iای وجود ندارد که $sup_i(p) \geq s_i$ بنابراین:

$$sup_1(p) < s_1 \;,\; sup_2(p) < s_2 \;,\; \dots \;,\; sup_k(p) < s_k$$

$$\Rightarrow sup(p) = \sum_i sup_i(p) < \sum_i s_i = s \quad \Rightarrow sup(p) < s \quad \text{تناقض} \;*$$

↳ چون پترن p پرتکرار است و $sup(p) \geq s$ ⇐ بنابراین خاصیت مونوتونی دیتاسیس برقرار است.

Scanned with CamScanner

جلسه هفتم _ ۱۳۹۹/۱۲/۲۴

الگوریتم Toivonen

همان الگوریتم Random sample است به اضافه‌ی یک Alarm، اگر FN دارید Alarm هشداری بدهد

→ Negative Border : پترن‌هایی که خودشان پرتکرار نیستند اما تمام sub pattern‌هایشان مستقیم آن‌ها پرتکرار هستند. (یعنی هر عضوی را حذف کنیم آیتم‌ست حاصل، پرتکرار است)

positive Border : مجموعه‌ی همه‌ی الگوهای Maximal

مثلا $\{A,B,C,D\}$ یک Negative Border است اگر و تنها اگر $\{A,B,C\}$، $\{A,B,D\}$، $\{B,C,D\}$ پرتکرار باشند و $\{A,B,C,D\}$ پرتکرار نباشد.

تشخیص FN: اگر یک مورد از پترن‌های (حداقل) Negative Border، FN باشد ما FN داریم.

قضیه: یک FN مثل S داریم اگر حداقل یکی از pattern‌های روی Negative Border (و تنها اگر) FN باشد.

Scanned with CamScanner

در فاز اول الگوریتم Toivonen، Random Sample رای گیریم و threshold را هم adjust می‌کنیم مثلا اگر k تا تراکنش از n تا تراکنش انتخاب کرده‌ایم در $\frac{k}{n}$ ضرب می‌کنیم بنده برای حل مشکل FN از چیزی که adjust شده یک مقدار کوچکتر ش می‌کنیم. علاوه بر این‌ها Alarm را هم تشکیل می‌دهیم. Alarm همان Negative Border ماست. (pass1)

pass2 ← برای تک‌تک الگوهایی که به مجموعه خروجی ما در pass1 اضافه شده‌اند، Frequency Counting انجام می‌دهیم. یعنی یک اسکن دیگر روی دیتابیس اصلی انجام می‌دهیم، الگوهایی که در Negative Border هستند درواقع توسط الگوریتم ما غیرپرتکرار تشخیص داده شده‌اند در اسکن دوم، پرتکرار بودن این الگوها را بررسی می‌کنیم، اگر غیرپرتکرار بودند مشکلی نیست ولی اگر پرتکرار باشند (احتمال یکی از آن‌ها) آن گاه FN داریم

جلسه هفتم - ۱۳۹۹/۱۲/۲۴

الگوریتم Toivonen

همان الگوریتم Random sample است به اضافه‌ی یک Alarm، اگر FN دارید Alarm هشداری دهد

Negative Border : پترن‌هایی که خودشان پرتکرار نیستند اما تمام sub pattern‌هایشان مستثنی هستند. (یعنی هر عضوی را حذف کنیم آیتم‌ست حاصل، پرتکرار است)

positive Border : مجموعه‌ی همه‌ی الگوهای Maximal

مثلا $\{A,B,C,D\}$ یک Negative Border است اگر و تنها اگر $\{A,B,C\}$، $\{A,B,D\}$، $\{B,C,D\}$ پرتکرار باشند و $\{A,B,C,D\}$ پرتکرار نباشد.

تشخیص FN: اگر یک مورد از پترن‌های حاشیه Negative Border، FN باشد یا FN داریم.

قضیه: یک FN مثل S داریم اگر حداقل یکی از pattern‌های روی Negative Border و تنها اگر FN باشد.

اثبات: فرض کنیم S یک پترن پرتکرار در دیتاست اصلی است اما توسط الگوریتم، غیرپرتکرار تشخیص داده شده.

چون S یک الگوی پرتکرار است پس همه‌ی زیرالگوهای آن هم پرتکرار هستند. از طرفی S در الگوریتم random Sampling ما غیرپرتکرار شده که نتیجه می‌گیریم زیرالگوهای آن تا یک جایی غیرپرتکرار اند و از جایی به بعد پرتکرار می‌شوند.

فرض کنید T اولین پترنی است که با حذف تعدادی از آیتم‌های S به دست آمده و این ویژگی‌ها را دارد:

① خود T در random Sample غیرپرتکرار است ② هرکدام از آیتم‌های T را که حذف کنیم آیتم‌ست باقی مانده در random Sample پرتکرار است. ← بنابراین $T \in$ Negative Border

پس ما یک پترن دیدیم که در random Sample ما غیرپرتکرار است اما در دیتاست اصلی پرتکرار است (چون TCS

Scanned with CamScanner

دیتایی که تا الان داشتیم دارای ساختار بود، هم چین تکرار نداشتیم و ترتیب هم مهم نبود.

اما درخت هایی دیتا بگونه ای است که ترتیب مهم مثلا بیوانفورماتیک. هم چین در بعضی داده ها تکرار مهم و می توانیم عناصر تکراری را حذف کنیم. مثلا sequence ها.

XML ← ذخیره سازی داده های Semi_structured

مسئله Tree Mining

یک collection از درخت ها داریم. یک minsup هم داریم. ← هردوی این ها ورودی هستند

هدف: پیدا کردن الگوهای درختی پرتکراری که حداقل به اندازه minSup در دیتاست ورودی ما آمده اند.

(به طور ساده ← تعداد درخت های دیتاستی که بیرون ما را در بر دارند)

تعاریف:

درخت: گراف متصلی که دور ندارد.

هر گراف متصلی که تعداد یال هاش برابر تعداد رأس هاش منهای یک باشد درخت است

هر گرافی که به شما بدهند (اگر تعداد یال ها و رأس ها را بدانیم) در O(1) می توانیم بفهمیم درخت است یا خیر.

درخت ریشه دار: درختی که در آن یک نود خاص به عنوان ریشه جدا شده و جهت یال ها از ریشه به سمت نودهای دورتر از ریشه است. همه ی درخت های دنیای واقعی ریشه دارند.

درخت مرتب: درختی که در آن بین فرزندان هر نود ترتیب داریم (و می توان جای آن ها را تغییر کرد)

درخت برچسب دار: هر نود درخت یک برچسب دارد.

در این جا از درخت های ریشه دار مرتب برچسب دار استفاده می کنیم.

گام های pattern extraction:

① candidate generation

② Frequency counting

Scanned with CamScanner

سه شنبه ۱۱ Sep 2020

Subtree homeomorphism

یکی از matching operator ها برای شمردن frequency یک پترن

به این صورت هست که یک بال از pattern مان را به یک مسیر در گراف map می کند

Subtree Isomorphism ← matching operator خلی استاندارد

Database Tree ← T گراف C

هر رئوس C را به یک رئوس از T نسبت می دهد. $\varphi : V(C) \rightarrow V(T)$

سه ویژگی دارد: ① برچسب نگهدار است. یعنی یک رئوس و تصویر آن رئوس T باید برچسب یکسان

داشته باشند: $\forall v \in V(C) : \lambda_C(v) = \lambda_T(\varphi(v))$

② اگر یالی مثل (u,v) در C داشته باشیم از $\varphi(u)$ به $\varphi(v)$ هم باید یال داشته باشیم در T.

$\forall u,v \in V(C) : (u,v) \in E(C) \Rightarrow (\varphi(u), \varphi(v)) \in E(T)$

③ ترتیب نگه دار است. اگر u از نظر قبل تر باشد نسبت به v، $\varphi(u)$ هم باید قبل تر باشد نسبت به $\varphi(v)$

$\forall u,v \in V(C) : u < v \iff \varphi(u) < \varphi(v)$

Subtree Homeomophism → $\varphi : V(C) \rightarrow V(T)$

① همان شرط اول Isomorphism

② از u به v در یک یال وجود دارد اگر و تنها اگر $\varphi(u)$ جد $\varphi(v)$ باشد در T (یال به مسیر مپ می شود)

$\forall u,v \in V(C) : u$ is a parent of v iff $\varphi(u)$ is an ancestor of $\varphi(v)$

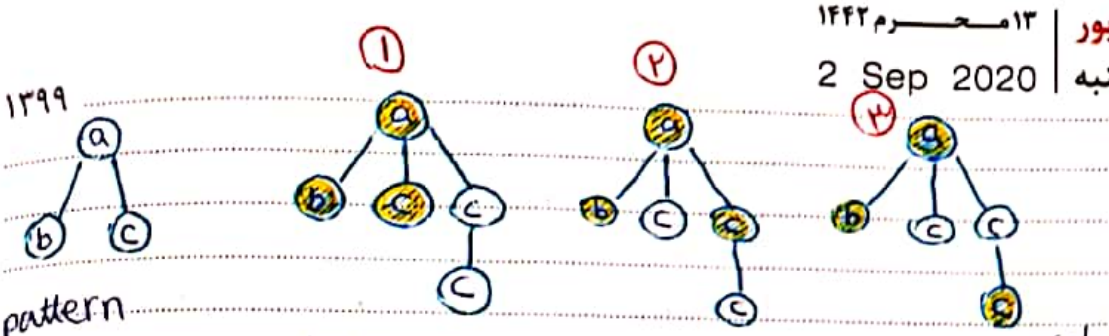
③ مثل شرط سوم Isomorphism

هر occurence یا embedding یعنی یک Subtree Homeomorphism از pattern با رابطه تناسبی وجود دارد.

Tree

Scanned with CamScanner

مثال:

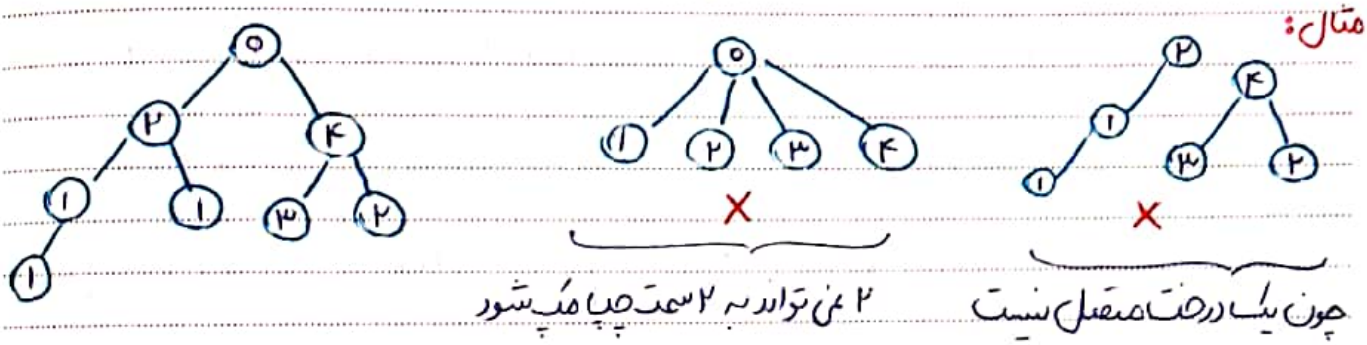


بین homeomorphism داریم

فقط اولی دومی isomorphism هم هستند (منظور از ترتیب، پیمایش preorder است)

هر isomorphism ، homeomorphism هم هست اما در کلیشن برقرار نیست.

مثال:



چون یک درخت متصل نیست

۲ می تواند به ۲ سمت چپ مپ شود

Frequent Tree pattern Mining

ما در تعریف از ساپورت یا frequency در دیتابیس هایی که structure دارند، داریم:

① per Tree Support ← تعداد درخت های دیتابیسی که پترن ما را در بردارند یعنی حداقل یک subtree Homeomorphism از پترن به دیتابیس ما وجود دارد.

در این تعریف تعداد subtree Homemorphism هایی که بین پترن و یک database Tree وجود دارد اصلا مهم نیست.

② per occurrence ← تعداد subtree Homemorphism هایی که از پترن ما به هر یک از database Tree های ما وجود دارد

Scanned with CamScanner

مسئله ← یک دیتابیس از درخت های ریشه دار مرتب برچسب دار به ما داده اند و یک minSup هم به ما داده اند

ما می خواهیم همه الگوهای در این دیتابیس را پیدا کنیم که ساپورتشان از minSup بزرگتر باشد.

- به هر نود یک id نسبت می دهیم. طبق pre_order پیمایش می کنیم و id ی دهیم. هم چنین هر نود دارای یک برچسب است که برچسب می تواند تکراری باشد اما id تکراری نیست.

Dietz numbering scheme (شمای شماره گذاری دایتز)

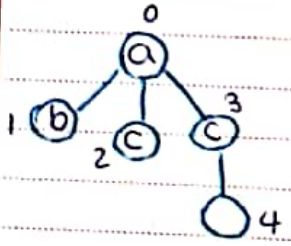
به هر نود یک زوج مرتب (مشتکل از دو عدد) اختصاص می دهیم.

lower band ← ⟨l , u⟩ → upper band

به کل این جفت scope می گوییم

pre_order number : upper band بیشترین عدد pre_order در زیر درخت آن نود

pre_order number : lower band خود آن نود



مثال:

root scope : ⟨0,4⟩

C scope : ⟨2,2⟩

C scope : ⟨3,4⟩

دو حوزه مقایسه روی اسکوپ هایی می دهیم که برگ رو هستند

فرض کنیم نودهای X و Y دارای اسکوپ های $S_x = [i_x, r_x]$ ، $S_y = [i_y, r_y]$ هستند آنگاه:

① S_x is strictly less than (<) S_y iff $r_x < l_y$

② S_x Contains S_y iff $l_x <= l_y$ and $r_x >= r_y$

① می گوییم S_x کاملا کوچکتر از S_y است اگر و تنها اگر حد پایین S_y کوچکتر از حد بالای S_x باشد [4,4] [5,5]

② می گوییم S_x ، S_y را در بر دارد اگر و تنها اگر حد پایین S_x کوچکتر مساوی حد پایین S_y باشد و حد بالای S_x بزرگتر مساوی حد بالای S_y باشد. [1,4] [3,3]

اگر S_x ، S_y را در برداشته باشد x یک جد برای y است (رابطه جد و فرزندی یا مسیر)

اگر $S_x < S_y$ اصطلاحا می گوییم y یک right sibling از x است ← y برادر x ی آید و y نوه ی x نیست

Scanned with CamScanner

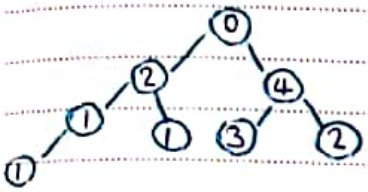
یک راه ذخیره درخت: ماتریس مجاورت ۱۳۹۹

لیست مجاورت: ~ ~ ~ ~

اما چون درخت‌ها sparse هستند یک سری روش‌های دیگر برای ذخیره‌سازی درخت‌ها استفاده می‌شود.

String encoding (کدگذاری رشته‌ای):

به صورت pre order درخت را پیمایش می‌کنیم و هر برچسبی را دیدیم وارد رشته می‌کنیم، هرجا backtrack داشتیم یک کاراکتر خاص (مثلاً ^) را به رشته اضافه می‌کنیم.



مثال: 1 -1 -1 2 -1 4 3 -1 -1 -1 -1 -1 1 1 2 0

بعضی جاها عکسه این backtrack های آخر را نگذارند یعنی بعضی جاها در بهترین حالت پیمایش عمل می‌شود در بعضی جاها در بیشینه.

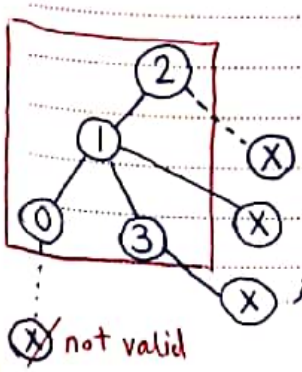
درخت‌های دیتاست ما به این شکل ذخیره می‌شوند در فایل، در حافظه هم وقتی candidate pattern می‌سازیم به این فرمت می‌سازیم و ذخیره‌اش می‌کنیم

جلسه هشتم - ۱۳۹۹/۱۲/۲۶

کلاس هم ارزی (Equivalence class)

فرض کنید تا سطح K ام الگوهای پرتکرار را پیدا کرده ایم و می‌خواهیم وارد سطح K+1 ام شویم. باید برای سطح K+1، کاندیدا پیدا کنیم. در سطح K ام، الگوهایی که تا الان پیدا کرده ایم را به چند کلاس هم ارزی افراز می‌کنیم

دو تا مجموعه در سطح K ام هستند به یک کلاس هم ارزی تعلق دارند اگر در تنها اگر K-1 تا نود اول آن‌ها در دو درخت یکسانی را بسازند یا به عبارت دیگه زیردرخت K-1 تایی اول آن‌ها یکسان باشد.



منظورمون از ترتیب نودها، ترتیب pre_order است

۳ تا نود سمت راستی هرکدامشان به این درخت اضافه شوند، باز هم ۲ تا نود اول درخت همان نودهای ۲، ۱، ۰، ۳ هستند. پس این ۳ تا درخت به یک کلاس هم ارزی تعلق دارند

Scanned with CamScanner

اما درخت حاصل از افزودن بزر X پایین به این کلاس هم ارزی تعلق خواهد داشت چون با افزودن این بزر، k تا بزر اول درخت ۲ را ۰ و X خواهد شد.

prefix sorting: 2 1 0 -1 3

رتبگی بدیهی درخت هایی که به یک کلاس هم ارزی تعلق دارند این است که اگر از انتهای جایشان

equivalence classes:

2 1 0 -1 3	-1 -1 X -1
2 1 0 -1 3	-1 X -1 -1
2 1 0 -1 3	X -1 -1 -1

رشته‌ای شان ۱- ها و لیبل آخر را حذف کنیم تا زمانی که به دومین لیبل از آخر برسیم، جایشان رشته‌ای باقی بماند که همه‌ی اعضای کلاس هم ارزی یکسان خواهد بود.

سوال: اعضای یک کلاس هم ارزی را چگونه میتوانیم به صورت efficient تولید و ذخیره کنیم؟ آیا نیاز داریم که هرکدام از اعضای کلاس هم ارزی را به صورت مجزا ذخیره کنیم در حافظه؟ خیر، چون در سطح k ام که هستیم k-1 تا بزر اول این ها مشترک هستند پس میایم در سطح k ام که هستیم، این سطح را به چند کلاس هم ارزی افراز می‌کنیم، برای هر کلاس هم ارزی core (زیردرخت k-1 تای مشترک) را جایشان رشته‌ای برایش تولید می‌کنیم و ذخیره می‌کنیم، برای هر کدام از آخرین بزرها فقط لیبل اش و pre_order number بزر پدرش را نگه میداریم هر element list یک درخت در سطح k ام را برای ما مشخص می‌کند و هر element list یک آرایه است که عضو اول: لیبل عضو دوم: pre_order number پدر بعدی راست ترین بزر در core

الگوریتم تولید کانونیکال

تنها اعضایی که به یک کلاس هم ارزی تعلق دارند می‌توانند با هم join شوند و یک درخت کانونیکال در سطح k+1 بسازند.

هر عضو را با هر عضو دیگر از جمله خودش join می‌کنیم، ۳ حالت پیش می‌آید برای join: ۲ یا ۱ یا ۰ درخت جدید تولید شود

بررسی حالت های join کردن (x,i) با (y,j):

① اگر $j=i$ ← ۲ درخت جدید سطح k+1 تولید می‌کنیم ← درخت اول از اضافه کردن (y,j) به P_x

۱۳۹۹

به دست می آید. P_x همان درخت core ماست که بود (i, x) به آن اضافه شده است.

درخت دوم ← (y, n) را به P_x اضافه می کنیم که n برابر با pre_order number نود x است و

نود y میاید به نود x اضافه می شود

② اگر $y > i$ ← یک درخت تولید می شود ← (y, y) به P_x اضافه می شود. یعنی y به نودی که در P_x

pre_order number اش برابر y است اضافه می شود

③ اگر $y < i$ ← هیچ درخت جدیدی تولید نمی شود

← مثال از کلاس هم ارزی در اسلایدها

الگوریتم candidate generation ابتدا الگوهای با اندازه ۱ را تولید می کند ← frequent label ها

می شن (در واقع لیبل های پرتکرار) ، بعد میریم سراغ تولید کاندیدهای با اندازه ۲ . هر نود پرتکرار را با هر

نود پرتکرار دیگر جوین می کنیم از جمله هر نود با خودش ← به این میگن large itemset mining

تکرار داریم و ترتیب هم مهمه. بعد از این که ریشه شون یکی باشه (یک کلاس هم ارزی تراز می گیرند و دوباره

طبق الگوریتمی که توضیح داده شد کاندیدها تولید می شن و به سطح های بالاتری رویم.

"Frequency Counting"

ما هر occurance یا هر رویداد از پترن مان در درخت دیتا بیس را بر حسب Subtree Homeomorphism

محاسبه می کنیم.

از یک data structure به نام Scope list برای نگه داشتن اثر Subtree Homeomorphism

هر عضو Scope List یک سه تایی به نام scope است

Scope هم برای نگه داشتن upper band ، lower band استفاده می شود و هم برای

نگه داشتن یک سه تایی هر سه تایی یک Subtree Homeomorphism را

Scanned with CamScanner

به صورت فشرده نگهداری می کند.

مثلا پترن های ۱ یا ۲ تایی

استراتژی کلی ما به این صورت است که برای هر پترن یک scope List با اسکن دیتابیس ورودی می سازیم.

بعد برای پترن های بزرگتر candidate generation می کنیم و چون نمی خواهیم برای بیشتر شدن پترن های بزرگ مجددا دیتابیس را اسکن کنیم، از روی scope List پترن های کوچکتر scope List پترن بزرگ را می سازیم

هر scope یک دسته ای است: (t, m, S)

t: آیدی database Tree ای که subtree Homeomorphism ما در آن اتفاق افتاده است

m: match lable ← آیدی نقطه $k-1$ تا نود اول پترن ما که با این subtree Homeomorphism را نگه می دارد

ما یک پترن k تایی داریم. این پترن k تایی با subtree Homeomorphism ی map می شود به database Tree.

$k-1$ تا نود از این k تا نود را در نظر بگیرید. آیدی این ها (یا pre_order number این ها) در m نگهداری می شود.

S: scope نقطه سمت راست ترین نود. یک پترن k تایی داریم، سمت راست ترین نود آن را در نظر بگیرید،

این نود یک SH با map می شود به یک نود در Database Tree که این نود یک scope دارد و همان scope است.

هر ردی scope List ما (x, i) با (y, z) جوین می شود، ما (x, i) ها را داریم می خواهیم از روی آن scope list درخت های جدید حاصل از جوین شان را بسازیم.

دقت (x, i) را با (y, z) در حالت اتفاق می افتد: ① آخرین نود (y, z) ی ستون بچه ی آخرین نود (x, i) ② آخرین نود (y, z) ی ستون برادر زاده ی سمت راست آخرین نود (x, i)

در حالت ① از روش In_Scope test استفاده می کنیم تا scope List ترکیبشان را بسازیم:

هر triple که در Scope List مربوط به (x, i) هست را با هر تریپل در scope List مربوط به (y, z) مقایسه می کنیم

فرض کنید triple اول را با (t_x, m_x, s_x) و triple دوم را با (t_y, m_y, s_y) شان بدهیم، ۳ تا شرط را چک می کنیم:

① $t_y = t_x$ (یعنی دیگ درخت دیتابیس باشند)

② $m_y = m_z$ ($k-1$ نود اول هر دو مجموعه دقیقا هم باید برابر باشد)

③ s_y در برداشته باشد s_x را (نقطه آخرین نود (x, i) باید یا بزرگ نقطه آخرین نود (y, z) باشد.)

مثال از این مبحث در اسلایدها آمده است

۱۳۹۹

اگر این سه شرط برقرار باشد بین (x,i) و (y,j) یک Homeomorphism حاصل از جفت‌شدن به این صورت ساخته می‌شود:

$t=t_y$, $m=m_y \cup \{l_x\}$, $S=S_y$

S_x مربوط به lower band یعنی l_x

در حالت ② از روش out_scope test استفاده می‌کنیم

شرط‌ها: ① $t_y=t_x$ ② $m_y=m_x$ ③ $S_x \subset S_y$

آخرین نود (x,i) در ستون sibling سمت چپ آخرین نود (y,j)

triple برای کاندیدای حاصل به این صورت ساخته می‌شود: $t=t_y$

سوال: چرا در هر دو حالت $S=S_y$؟ $m=m_y \cup \{l_x\}$, $S=S_y$

وقتی کاندیدای جدید تولید می‌کنیم همیشه آخرین نود (سمت راست‌ترین نود)، آخرین نود درخت دوم است. در نتیجه همین آخرین نود درخت جدید هم باید همین آخرین نود درخت دوم باشد و ما هم برای S همین آخرین نود را نگه می‌داریم و آن را با S_y مقداردهی می‌کنیم

← مثال از TreeMiner Algorithm در اسلاید ۴

مثال:

List 1

0,0,[1,1]
1,1,[2,2]
2,0,[2,2]
2,0,[5,5]
2,4,[5,5]

0,0,[3,3]
1,2,[3,3]
2,02,[7,7]
2,05,[7,7]
2,45,[7,7]

List 2

↓ یکی از درخت‌هایی که از ترکیبشان تولیدی می‌شود این است

فرض کنیم scope List دو زیردرخت سمت چپ را داریم و می‌خواهیم scope List درخت حاصل از join آن‌ها را بیابیم. باید از out_scope استفاده کنیم. هر سطر از list 1 را باید با هر سطر از list 2 مقایسه کنیم و این ۲ شرط را چک کنیم

0,01,[3,3]
1,12,[3,3]
2,02,[7,7]
2,05,[7,7]
2,45,[7,7]

0,0,[1,1] , 0,0,[3,3]

✓ $0=0$ ✓ $0=0$ $[1,1] \subset [3,3]$ ✓

$t=t_y=0$ $m=m \cup \{l_x\}=01$ $S=S_y=[3,3] \Rightarrow 0,01,[3,3]$

Scanned with CamScanner

پنجشنبه | 10 Sep 2020

Locality Sensitive Hashing (درهم‌سازی حساس به مکان)

یه task پایه داریم که در بسیاری از مسائل کاربرد دارد ← یافتن object های مشابه در یک collection

خیلی بزرگ از اشیاء

یه task پایه دیگه که خیلی ارتباط داره با task بالا ← یه collection خیلی بزرگ از اشیاء داریم و یک شیء دیگه به عنوان query به شما داده می‌شود و از شما خواسته می‌شود اشیاء مشابه آن را گزارش کنید

مثال از Similarity Search ← در Pinterest

pinterest تصویر مورد سوال را به یک بردار map می‌کند، تمام عکس‌های دیتابیس هم به شکل بردار ذخیره شده‌اند و ما بردار تصویر کوئری را با آن‌ها مقایسه می‌کنیم و بردارهای مشابه را پیدا می‌کنیم

در Similarity Search ما مقدار خیلی زیادی object داریم و هر object هم ویژگی‌های زیادی دارد. این دو مسئله Similarity Search را بسیار کندی کرده و به دنبال تکنیک‌هایی برای سریع‌تر

انجام دادن این فرآیند هستیم.

شهریور ۲۱ جمعه

۲۲ محرم ۱۴۴۲

11 Sep 2020

← یکی از تکنیک‌ها ← LSH

تعریف رسمی Similarity Search:

مقدار خیلی زیادی object در دیتابیس داریم، هر object از یک بردار تشکیل شده که مقدار خانه‌های این بردار خیلی زیاد است. می‌خواهیم object هایی خیلی شبیه هم (یا شباهتشان از یک threshold که ما کند کار set کنه، بیشتر هست) رو پیدا کنیم.

در تشخیص دالیوفت‌های شراری هم کاربرد دارد (در سرقت علمی داریم) ← به حالت ساده: هر کلمه را یک نیچر در نظر گیریم و اگر در اون doc آمده بود جای منتظر آن را ۱ در نظر گیریم و می‌گذاریم. بردارهای doc ها را با هم مقایسه می‌کنیم اگر شبیه باشند doc شراری داریم.

روز سینما

Scanned with CamScanner

۱۳۹۹

کاربرد دیگر ← خوشه‌بندی مشتریان ← مشتری‌هایی که آیتم‌های یکسان خریداری کرده‌اند را می‌خواهیم در یک خوشه قرار بدهیم. می‌توانیم برای هر مشتری یک بردار داشته باشیم که هر خانه آن متناظر یک کالاست که اگر مشتری آن را خریداری کرده 1 می‌گذاریم.

کاربرد دیگر ← pinterest ← برای خوشه‌بندی تصویرها

کاربرد دیگر ← Recommendation ← مثلا آمازون می‌خواهد به مشتریان خود کالاهایی را پیشنهاد دهد برای این کار باید مشتری‌های شبیه مشتری مورد نظر را بیابد و کالاهایی که آن‌ها خریده‌اند را به مشتری مورد نظر پیشنهاد دهد. یک راه استاندارد برای یافتن مشتری‌های شبیه به هم ← بردارهای خرید مشتری‌ها را در نظر بگیریم، شباهت این بردارها را حساب کنیم و شباهت مشتری‌ها را برحسب شباهت بین بردارهای خرید آن‌ها در نظر بگیریم.

Scanned with CamScanner

13 Sep 2020

جلسه ششم - ۱۴۰۰/۱/۱۵

Similarity search ← مقدار زیادی آیتم داریم و می خواهیم مشابه ها را پیدا کنیم آن هایی که شباهتشان از یک threshold ای بیشتر است)

ورودی: ① یک threshold بین ۰ تا ۱ (min Similarity) ② یه سری data point (که مقادیر زیادی بُعد دارند ← high dimensional فضای)

③ یک distance function (که دو تا دیتاپوینت را می گیرد و آن ها را به یک عدد بین ۰ و ۱ مَپ می کند)

خروجی: آن دیتاپوینت هایی که شباهتشان از threshold بیشتر است (یا اینکه distance آن از threshold کمتر است)

similarity = 1 - distance

راه حل ساده: عَلَم دیتاپوینت ها را ۲ به ۲ مقایسه کنیم ← $O(N^2)$ هزینه بالاست

به دنبال خانواده ای از سولوشن های linear هستیم، هزینه ای که ذهنم یا دقیق بودن جواب است.

LSH (Local Sensitive Hashing)

از تعدادی Hash function استفاده می کنیم که دیتاپوینت های ما را به تعدادی bucket مَپ می کنند و collision برخورد داریم. Hash function ها به گونه ای طراحی می شوند که دیتاپوینت هایی که بهم شبیه هستند به احتمال زیاد در یک bucket قرار می گیرند. بنابراین به جای مقایسه همه دیتاپوینت ها با هم، فقط آن هایی که در یک باکت هستند را مقایسه کنیم. اگر Hash function به خوبی طراحی شود در هر باکت تعداد ثابتی دیتاپوینت داریم که هزینه مقایسه آن ها constant است پس هزینه کل هم $O(cn) = O(n)$ می شود ← مزیت این روش

عیب: False Negative که داشته باشیم یعنی دو تا دیتاپوینت شبیه را عمدا در دو تا باکت مختلف بندازه.

۳ مرحله اصلی در الگوریتم:

① Shingling ← یک doc را به یک set representation (بازنمایی مجموعه ای) تبدیل می کند

(بردار ۰ و ۱ به طول زیاد) ← با تعداد ابعاد زیاد.

Scanned with CamScanner

② Min_Hashing ← بردارهای ۰و۱ تولیدشده در گام اول خیلی طولانی هستند. در این مرحله این بردارهای طولانی به بردارهای خیلی کوچکی تبدیل می‌شوند به طوری که شباهت بردارها حفظ می‌شود. بصورت Expected value، خیلی احتمال دارد که شباهت دو بردار کوتاه برابر شباهت حالت طولانی آن دو بردار باشد. (در واقع در این مرحله مستقل تعداد زیاد اعداد را حل می‌کنیم)

③ Locality_Sensitive Hashing ← در این مرحله مستقل تعداد زیاد دیتا پوینت‌ها را حل می‌کنیم از یک یا چند تابع Hash استفاده می‌کنیم و آن‌ها را روی signature (بردارهای کوچک) تولیدشده در مرحله قبل اعمال می‌کنیم و مپ می‌شوند به یک سری باکت.

بررسی دقیق‌تر این سه مرحله:

① shingling ← باید یک vector از داکیومنت‌مان استخراج کنیم.

یک مجموعه از K_shingle (K-gram) ها از داکیومنت ما استخراج می‌کند. (یک توالی K تایی از توکن‌ها، توکن میتونه یه کاراکتر باشه یا یه کلمه باشه ← بستگی به تعریف داره)

تمامی توالی‌های K تایی از توکن‌ها را از doc استخراج می‌کنیم و doc خودرا به صورت مجموعه‌ای از {k-gram} نمایش می‌دهیم. اگه k-gram ها خیلی طولانی بودن می‌توانیم آن‌ها را map کنیم به عدد و اون عدد رو نگه داریم.

مثال: $D_1 = abcab$ $\quad S(D_1) = \{ab, bc, ca\}$ $\quad h(D_1) = \{1, 5, 7\}$

$K = 2$

معمولا K های بالاتر همیشه جواب می‌دهد. (K های ۸ و ۹ و ۱۰ رایج است) چون ساختار محلی را در نظر می‌گیرند

مزایای shingle ها: ① داکیومنت‌هایی که شبیه‌ترند، shingle های زیادی را به صورت مشترک دارند ② تغییر یک توکن فقط k-1 همسایه‌ی سمت چپ و راست آن را تغییر می‌دهد و نیازی به بررسی کل doc نیست.

معیار شباهت برای shingle ها:

شهادت امام سجاد (ع) (۹۵ هـ ق) به روایتی

شباهت جاکارد (jaccard Similarity)

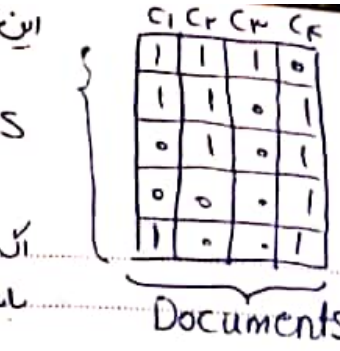
$$Sim(D_1, D_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$$

$$\text{jaccard distance}: d(C_1, C_2) = 1 - \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$$

Scanned with CamScanner

این ماتریس خیلی sparse است

shingles



اگر shingle ای در doc ای باشد خانه متناظر آن برابر ۱ می‌شود در غیر اینصورت ۰.

شباهت jaccard بین ۲ تا doc روی توپیم مستقیم از روی این جدول حساب کنیم:

$Sim(C_1, C_2) = \frac{۲}{۴}$ ← مثال: $\frac{\text{تعداد خانه‌هایی که هر دو doc در آن ۱ هستند}}{\text{تعداد خانه‌هایی که حداقل یکی از آن‌ها ۱ است}}$

② Min Hashing

هر ستون را به یک یا چند Hash function می‌دهیم. هر Hash function با دریافت یک ستون، یک عدد به عنوان خروجی می‌دهد. اگر از m تا Hash function استفاده کنیم برای هر ستون m تا عدد خروجی داریم که با کنار هم قرار دادن آن‌ها، یک Signature تولید می‌شود برای آن ستون.

Hash function باید به گونه‌ای باشد که:

① اگر $Sim(C_1, C_2)$ بالا باشد آن‌گاه با احتمال بالایی $h(C_1)=h(C_2)$

② اگر $Sim(C_1, C_2)$ پایین باشد آن‌گاه با احتمال بالایی $h(C_1) \neq h(C_2)$

برای شباهت jaccard می‌توانیم Hash function ای تعریف کنیم که این ویژگی را داشته باشد

Random permutation: یک درهم‌سازی تصادفی از ردیف‌های جدول shingle/doc

هر permutation تصادفی یک Hash function به ما می‌دهد.

هر ستون را تحت یک permutation جابجایی می‌کنیم و به اولین خونه‌ای که مقدارش ۱ هست و تا رسیدیم، index آن خانه تحت آن permutation را برمی‌گردانیم. (فرض می‌کنیم که index های آن را برگردانیم یا index آن تحت permutation، اما قرارداد می‌کنیم که همیشه دومی را برگردانیم.)

نکته: تحت یک permutation تصادفی، هرکدام از ۱ های موجود در یک ستون با احتمال یکسانی انتخاب می‌شوند.

قضیه: $Pr[h_\pi(C_1) = h_\pi(C_2)] = Sim(C_1, C_2)$ ← اثبات در اسلاید

Scanned with CamScanner

چرا شباهت دو تا Signature را به صورت Expected value سنجیدیم؟

چون این حالت برآیند تصادفی داریم. هرکدام از Hash function های ما تصادفی هستند چون نسبت به یک permutation تصادفی تولید می‌شوند، بنابراین خروجی ای که تولید می‌کند یک متغیر تصادفی است و signature های ما برداری از متغیرهای تصادفی هستند.

$$E[Sim(S_1,S_2)] = E[\#\text{permutations } \pi : h_\pi(C_1)=h_\pi(C_2)]/m$$

$$= m * Pr[h_\pi(C_1)=h_\pi(C_2)]/m = Pr[h_\pi(C_1)=h_\pi(C_2)] = Sim(C_1,C_2)$$

سوال: برای کاهش خطا چه کار کنیم؟ یعنی برای افزایش شباهت دو تا signature به شباهت دو تا ستون ها signature یک تقریب می‌زند از شباهت بین دو تا ستون و این کار را به صورت unbiased انجام می‌دهد و ما می‌توانیم با افزایش تعداد sample ها (تعداد permutation ها) خطا را به صورت خطی کاهش بدهیم.

نکته: در عمل تولید permutation ها هم با استفاده از یک hash function تعریف می‌شود.

تعریف hash function برای تولید permutation:

$$h_{a,b}(X) = ((aX+b) \bmod p) \bmod N \qquad (p>N)$$

a و b دو عدد صحیح اند که به صورت تصادفی تولید می‌شوند. p یک عدد اول است که خودمان انتخاب می‌کنیم اگر در جدول اصلی N تا ردیف داشته باشیم و روی permutation یک عدد بین ۱ تا N و خروجی این هم یک عدد بین ۱ تا N است. (نکته: ما می‌گوییم هر ردیف به چه ردیفی map می‌شود)

نحوه تولید ماتریس M:

هر ردیف آن متناظر با یکی از permutation ها و هر ستون معادل یک doc است. ابتدا تمام خانه‌ها را با ∞ مقداردهی اولیه می‌کنیم. بعد می‌آییم جدول را از بالا ردیف به ردیف scan می‌کنیم. برای هر خانه‌ای که ۱ است تک‌تک hash function ها را روی آن شماره ردیف (مثلا j) اعمال می‌کنیم. اگر $h_i(j) < M(i,c)$ باشد آن گاه $M(i,c) \leftarrow h_i(j)$

Scanned with CamScanner

جلسه دهم - ۱۴۰۰/۱/۱۷

Locality Sensitive Hashing (هش‌سازی حساس به مکان)

۲۷ | شهریور | پنجشنبه | ۲۸ محرم ۱۴۴۲ | 17 Sep 2020

هدف کلی ما similarity search است.

ایده کلی LSH این بود که برای حل این مسئله از Hash Function استفاده بکنیم. اون هایی که در یک پیمانه (باکت) می افتند شبیه به هم درنظر گرفته شوند (یا اینکه کاندیدای شبیه بودن باشند و با یک scan دوم چک شود که آیا واقعا شبیه اند یا نه)

ماتریس M ← کاهش بعد زیادی می دهد، similarity preserving است، باید بیایم Hash Function را روی ماتریس M اعمال کنیم و signature های مختلف را به پیمانه‌ی مربوطه map کنیم.

در ماتریس M هر ردیف متناظر با یک Min Hash است

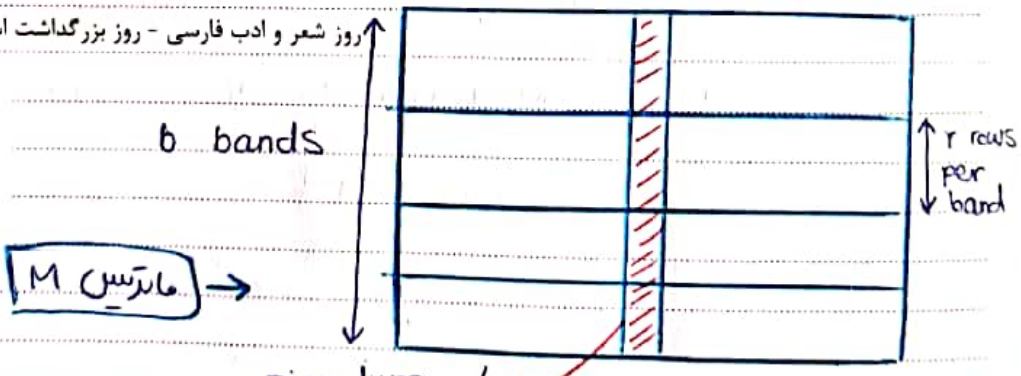
" " ستون " " " data point است

ماتریس M را به صورت افقی افرازی می کنیم، به هرکدام از پارتیشن ها یک band می گوییم. هر band شامل تعدادی ردیف است. فرض می کنیم تعداد ردیف های band های مختلف با هم مساوی است.

تعداد band ها : b داخل هر band تعداد ردیف ها : r

روز شعر و ادب فارسی - روز بزرگداشت استاد سید محمد حسین شهریار

۲۸ | شهریور | جمعه | ۲۹ محرم ۱۴۴۲ | 18 Sep 2020



گام اول: band band کردن

گام دوم: هر band از هر signature را توسط یک Hash Function، map می کنیم به یک پیمانه

برای تک تک signature ها این کار را انجام می دهیم. دوتا signature را مشابه (یا کاندیدای مشابه) درنظر می گیریم اگر دست کم داخل یک band وجود داشته باشد که همه‌ی آن ها به یک پیمانه map شده اند

در ادامه برای ساده سازی تحلیل ها، در مورد hash function این فرض را می کنیم:

hash function ها روی band دوتا band را به یک bucket، مپ می کند اگر و تنها اگر این دوتا band دقیقاً عین هم باشند. به عبارت دیگه collision یا برخورد نداریم.

Scanned with CamScanner

مثال: فرض کنید ماتریس M دارای ۱۰۰،۰۰۰ ستون است. تعداد ردیف‌های آن ۱۰۰ است. می‌خواهیم جفت doc هایی را بیابیم که similarity آن‌ها از ۰/۸ بیشتر باشد. فرض کنید $b=20$ و $r=5$ باشد.

۲۹ | شهریور / شنبه | ۱ صفر ۱۴۴۲ / 19 Sep 2020

۱۳۹۹

positive بودن در این مسئله به شباهت داشتن

دنبال false ها می‌گردیم: با چه احتمالی false (شامل FP و FN) اتفاق می‌افتد؟

FP ← الگوریتم به غلط دو doc ای که تشبیه نیستند را تشبیه معرفی کند

FN ← دو doc که تشبیه هستند را به غلط غیر تشبیه معرفی کند.

* بررسی FN: فرض کنید شباهت ۲ تا doc برابر ۰/۸ باشد. می‌خواهیم احتمال این را حساب کنیم که الگوریتم ما این دو doc را غیرتشبیه معرفی کند.

به ازای هر ۲ بند c_1 و c_2 باید مقادیر مختلف داشته باشند. احتمال اینکه به ازای یک ردیف خاص c_1 و c_2 عین هم باشند ۰/۸ است. به چه احتمالی c_1 و c_2 دارای یک band یکسان هستند؟

با چه احتمالی یک بند خاصشان عین هم می‌شود؟ $1-(0.8)^5$ و $(0.8)^5 = 0.328$

با چه احتمالی هر ۲۰ بندشان متفاوت است؟ $(1-(0.8)^5)^{20} = 0.00035$

احتمال اینکه c_1 و c_2 هیچ بندِ مشترکی نداشته باشند (احتمال FN) ← ۰/۰۰۰۳۵

* بررسی FP: فرض کنیم شباهت c_1 و c_2 = ۰/۳، برای اینکه c_1 و c_2 تشبیه معرفی بشوند باید حداقل یک band مشترک داشته باشند.

احتمال اینکه ۱ بند مشترک داشته باشند: $(0.3)^5$

" " بند خاص در عین هم نباشند: $1-(0.3)^5$

احتمال اینکه در هر ۲۰ بند مشترک نباشند: $(1-(0.3)^5)^{20}$

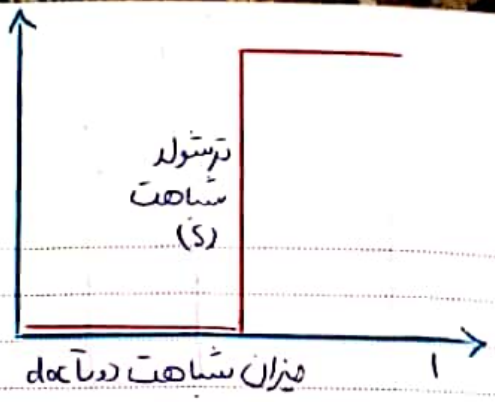
احتمال اینکه حداقل در ۱ بند مشترک باشند: $1-(1-(0.3)^5)^{20} = 0.047$

نکته: تعداد سطرهای ماتریس M دست خود ماست، تعداد b و r هم دست خود ماست.

این رابطه همیشه برقرار است: $M = b * r$

با کم و زیاد کردن b و r می‌توانیم احتمال‌های FP و FN را جابه‌جا کنیم. مثلا در مثال قبل اگه ۱۰ تا و ۱۰ تا بود FN بیشتر و FP کمتر می‌شد.

Scanned with CamScanner



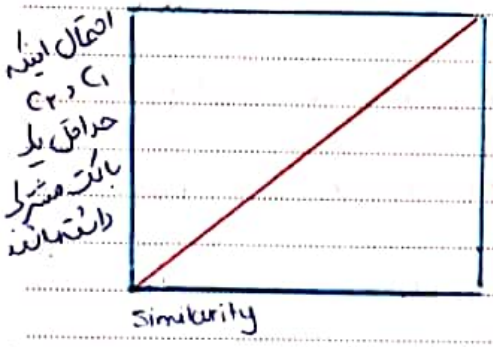
احتمال اینکه ۲ تا doc حداقل یک بند مشترک داشته باشند

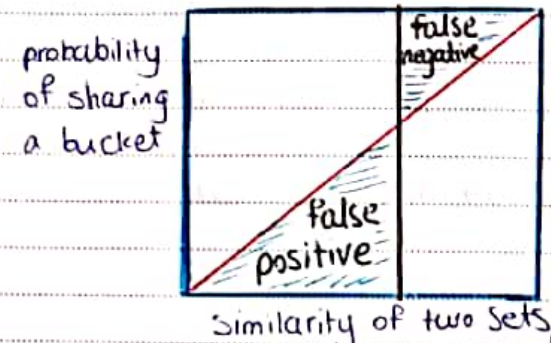
نمودار قرمز ← حالت ایده آل

وقتی $Sim(C_1,C_2)$ از threshold کوچکتر است احتمال اینکه C_1 و C_2 یک بند مشترک داشته باشن صفر است ← در نتیجه FP نداریم

روی threshold یک گام از ۰ به ۱ داریم.

وقتی $Sim(C_1,C_2) > S$ احتمال اینکه C_1 و C_2 بند مشترک داشته باشن ۱ است ← FN نداریم





فرض کنید فقط ۷ ردیف داشته باشیم، در این حالت دیاگرام به این شکل است ↑ آیا این نمودار خوب است؟ خیر

میزان قابل توجهی FP و FN داریم

احتمال اینکه C_1 و C_2 به یک باکت مشترک بیفتند برابر $Sim(C_1,C_2)$ است.

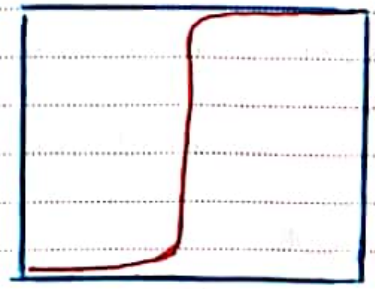
فرض کنید شباهت C_1 و C_2 برابر t باشد، تعداد ردیف های هر band : r ، تعداد bandها : b

احتمال اینکه همه ی ردیف های داخل یک band عین هم باشند : t^r

احتمال اینکه دو بند حداقل در یک ردیف با هم متفاوت باشند : $1-t^r$

احتمال اینکه برای تمام بندها C_1 و C_2 عین هم نباشند : $(1-t^r)^b$

احتمال اینکه برای حداقل یک band ، C_1 و C_2 عین هم باشند : $1-(1-t^r)^b$



نمودار این تابع :

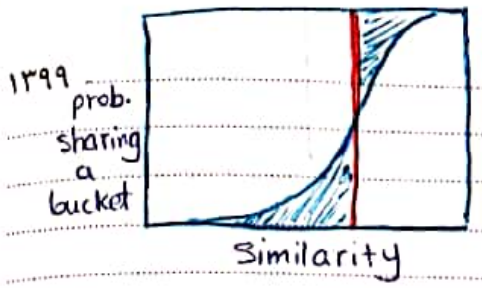
نمودار S شکل دارد. یک افزایش خیلی کند

داریم بعد یک گام داریم دوباره یک افزایش کند

استفاده از LSH باعث می شود دیاگرام از حالت خطی به شکل S دربیاید.

Scanned with CamScanner

نکته: اگر b و r را طوری تنظیم کنیم که threshold وسط step (گام) بیفتد خوب است. حد قابل قبولی از FP و FN خواهیم داشت.



نکته: برای FP ها همیشه سولوشن داریم ← اسکن دوم

بیایم اعضایی که در یک باکت افتاده اند را واقعا با Similarity شان حساب کنیم دقیق کنیم از threshold بیشتر هست یا نه.

به همین دلیل حساسیت ما روی FN ها بیشتر است.

فواید تکنیک Min Hash: ① به صورت Similarity preserving کاهش بُعد انجام می‌شود

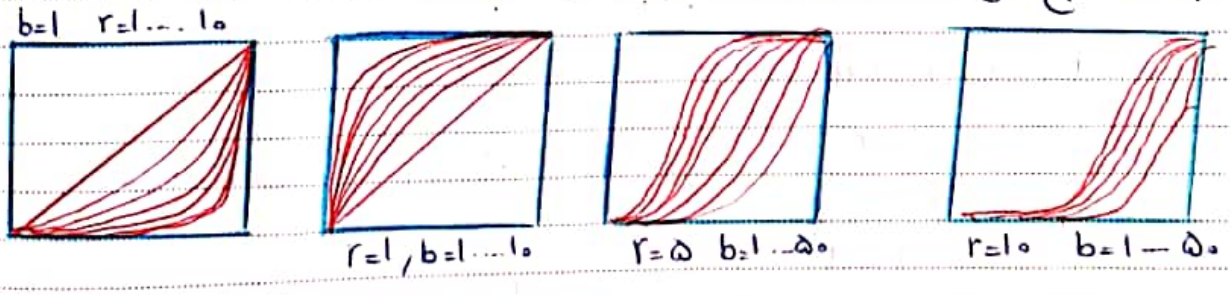
② با time Complexity پایین (خطی برحسب تعداد داده‌ها) این کار را انجام می‌دهد.

سوال: time Complexity برای تولید جدول Min Hash چقدر است؟

بسته به در نظر گرفتن بقیه عامل‌ها: $O(n)$، با در نظر گرفتن بقیه عامل‌ها مثلا تعداد dimension ها و تعداد ردیف‌های ماتریس M، متفاوت می‌شود.

با تغییر مقدار b و r: ① شکل تابع را عوض کنیم ② تعیین کنیم step تابع کجا اتفاق بیفتد

برای اینکه تابع شکل S داشته باشد b و r باید بزرگتر از ۱ باشند.



تعمیم LSH:

distance measure ← برای فضایی مثل S اعمال می‌شود (ممکن است فضای برداری باشد یا نه)

دوتا data point از فضا را به یک عدد map می‌کند. (عدد مثبت)

اگر اونها normalize می‌شدند این عدد مثبت بین ۰ و ۱ قرار می‌گیرد.

آغاز جنگ تحمیلی (۱۳۵۹ هـ ش) - آغاز هفته دفاع مقدس

خاصیت جابه‌جایی دارد ③ $d(x,y) = d(y,x)$

نامساوی مثلث ④ $d(x,y) \leq d(x,z) + d(z,y)$

distance measure این دو ویژگی را باید داشته باشد:

① $d(x,y) \geqslant 0$

② $d(x,y) = 0$ iff $x = y$

Scanned with CamScanner

هر دو تا که حکت همایز باید distance بزرگتر از صفر داشته باشند.

cosine distance (فاصله کسینوسی) ← زاویه بین دو بردار

$\frac{x \cdot y}{\|x\| \|y\|}$

Jaccard distance

L_2 norm
L_1 norm
} Euclidean distances

← تفاضل دو بردار را حساب می‌کنیم، از بردار تفاضل اگر نرم ۱ بگیریم L_1 norm بدست می‌آید و اگر نرم ۲ بگیریم L_2 norm بدست می‌آید

L_1 norm: جمع قدر مطلق درایه‌ها

L_2 norm: جذر جمع توان دو اختلاف‌ها

خانواده‌ای از Hash function ها

دیترکس hash function ← اگر دو عضر x و y دارای دیترکس یکسان باشند (شبیه باشند) به احتمال زیاد آن‌ها را به یک باکت map می‌کند.

خانواده Hash function ها ← مجموعه‌ای از Hash function ها به گونه‌ای که بتوانیم یک HF خاص از این مجموعه را به صورت تصادفی به صورت کارا دربیاوریم (زمان چند جمله‌ای) انتخاب کنیم.

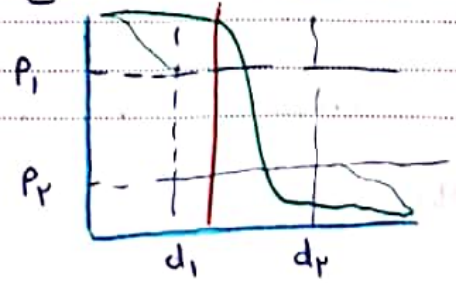
مثال: مجموعه تمامی Min Hash ها که روی ردیف‌ها تعریف می‌شوند ← متناظر با هر permutation یک Min Hash داشتیم و چون permutation ها را تصادفی تولید می‌کردیم انتخاب Min Hash هم تصادفی بود.

H ← خانواده از hash function ها h ← یک hash function

یک خانواده از hash function ها مثل H، حساس به (d_1, d_2, P_1, P_2) هست اگر به ازای هر x و y در فضای داده‌ها به ازای هر h داخل این خانواده این دو شرط برقرار باشد:

① اگر x و y فاصله‌شان (distance) از d_1 کمتر است $(d(x,y) \leq d_1)$ با احتمال حداقل P_1 توابع hash موجود در این خانواده، x و y را در یک پیمانه بیندازند.

② اگر x و y به هم شبیه نیستند $(d(x,y) \geq d_2)$ با احتمال خیلی کمی (حداکثر P_2) توابع h این خانواده x و y را داخل یک پیمانه می‌اندازند.



Scanned with CamScanner

مثال: S ← مجموعه تمام بردارهای ۰ و ۱
Jaccard distance ← d
H ← خانواده تمام Min Hash هایی که توسط تمام permutation هایی که روی ردیف‌های بردارها صفر و ۱ ها تعریف می‌شوند اعمال می‌شود.

۲ | مهر | ۵ صفر ۱۴۴۲
چهارشنبه | 23 Sep 2020

۱۳۹۹

می‌دانیم که: $\Pr[h(x)=h(y)] = 1 - d(x,y)$

خانواده Min Hash ها یک خانواده حساس به $(\frac{1}{3}, \frac{2}{3}, \frac{2}{3}, \frac{1}{3})$ است.

چون اگر $d \leq \frac{1}{3}$ پس $similarity \geq \frac{2}{3}$ است

و اگر $d \geq \frac{2}{3}$ پس $similarity \leq \frac{1}{3}$ است و دقت $Sim(x,y)$ حداکثر $\frac{1}{3}$ باشد با احتمال حداکثر $\frac{1}{3}$ داخل یک باکت می‌افتند.

نکته: به طور کلی خانواده Min Hash ها حساس به $(d_1, d_2, (1-d_1), (1-d_2))$ است

به این علت که Min Hash، Similarity preserving است ← اینکه اگر شباهت x,y بزرگتر مساوی $1-d_1$ باشد احتمال share کردن باکت در تابع Min Hash بزرگتر مساوی با $1-d_1$ است. (احتمال اینکه دقتا آبجکت را در یک باکت بیندازد برابر میزان شباهت آن‌هاست)

جلسه یازدهم - ۱۴۰۰/۱/۲۲

Amplify (تقویت) کردن یک LS_family ←

روشی که برای تقویت کردن تا این جا گفتیم:

روی ردیف‌های یک band باید and اعمال کنیم و روی bandهای مختلف or اعمال می‌کنیم ← نمودار S شکل به ما می‌دهد. آیا این تنها راه ممکن برای تولید نمودار S شکل است؟ خیر

فرض کنید یک خانواده از Hash Function ها به نام H داریم. روی این خانواده می‌توانیم خانواده H' را بسازیم که از اعمال and روی H ایجاد شده. r تا HF از H به طور تصادفی انتخاب می‌کنیم و می‌گوییم $h=[h_1, \dots, h_r]$ اگر و تنها اگر $h(x)=h(y)$ برای همه $1 \leq i \leq r$ $h_i(x)=h_i(y)$

عکس AND

قضیه: اگر خانواده H حساس به (d_1, d_2, P_1, P_2) باشد آن گاه H' حساس به $(d_1, d_2, (P_1)^r, (P_2)^r)$ است.

چون r تا event داریم که هرکدام حداقل با احتمال P_1 اتفاق می‌افتند و از هم مستقل اند پس احتمال‌هاشون ضرب می‌شه

Scanned with CamScanner

سوال: آیا این عملگر and خوب است؟

احتمال P_1 هرچقدر بیشتر باشد بهتر است و احتمال P_2 هرچقدر کمتر باشد بهتر است.

۳ | مهر | پنجشنبه | ۶ صفر ۱۴۴۲ | 24 Sep 2020

حالا در H'، P_1 به $(P_1)^r$ و P_2 به $(P_2)^r$ تبدیل شده و چون هر دوی P_1 و P_2 بین ۰ و ۱ هستند $(P_1)^r$ و $(P_2)^r$ هر دو کوچکتر شده اند نسبت به P_1 و P_2. پس برای P_1 بد بوده و برای P_2 خوب شده. از طرفی P_2 با شدت بیشتری کاهش پیدا می کند چون عدد کوچک تری است. پس این هم خوب است در مورد and.

عملگر OR: فرض کنید خانواده H را داریم، H' را از H اینگونه می سازیم: تا hash function از H را به طور تصادفی انتخاب می کنیم و می گوییم $h(x)=h(y)$ اگر و تنها اگر حداقل برای یک i داشته باشیم: $h_i(x)=h_i(y)$

$h=[h_1,\dots,h_b]$ (یعنی به ازای هر x و y بین hi های مختلف OR داریم)

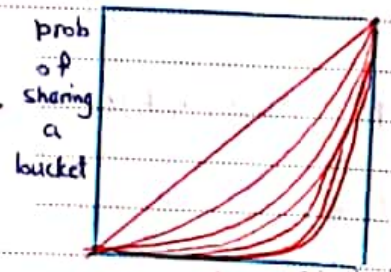
قضیه: اگر H حساس به (d_1, d_2, P_1, P_2) باشد آن گاه H' حساس به $(d_1, d_2, 1-(1-P_1^b), 1-(1-P_2^b))$

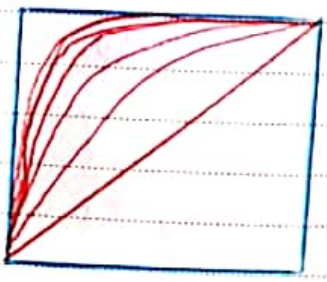
($1-(1-P_1^b)$ و $1-(1-P_2^b)$: اثر عملگر OR روی P_1 و P_2)

خواهد بود.

نتیجه: هر دوی P_1 و P_2 بزرگ شده اند پس برای P_1 خوب است و برای P_2 بد است اما P_1 را با شدت بیشتری زیاد می کند نسبت به P_2. در نتیجه استفاده از عمل and_or (در حالت کلی) مفید است.

۴ | مهر | جمعه | ۷ صفر ۱۴۴۲ | 25 Sep 2020





بررسی روزه که همانا عملگر and را استفاده کنیم و بعد عملگر or. می توانیم برعکس عمل کنیم (اول or بعد and)

نکته: به ازای s های کم با احتمال خیلی کمی c_1, c_2 را در دون یک باکت می گذاریم و به ازای s های بزرگ با احتمال خیلی زیادی c_1 و c_2 را در یک باکت می گذاریم.

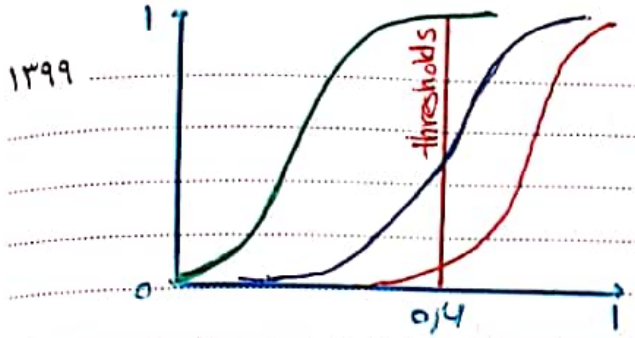
$r=۴, b=۴$

یک خانواده حساس به $(۰.۸, ۰.۲, ۰.۸۸, ۰.۲۰۴)$

را تبدیل کرده به یک خانواده حساس به

Scanned with CamScanner

انتخاب r و b بستگی دارد به threshold ما و اینکه چقدری خواهیم FP و FN داشته باشیم.



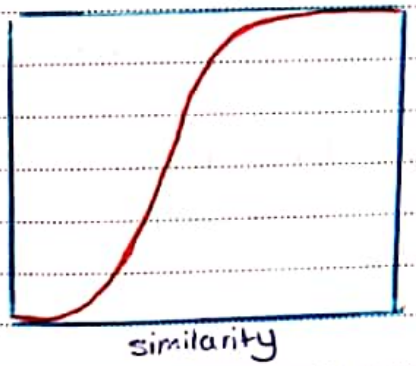
فرض Linear بودن LSH:

اینکه داخل هر باکت تعداد ثابتی object داشته باشیم

با توجه به اینکه گفتیم مشکل FP را می‌توان با اسکن دوم حل کرد ولی FN را نه، آیا انتخاب r و b به گونه‌ای که FN را کاهش دهد مناسب است؟

نباید FP آنقدر زیاد باشد که LSH را از حالت خطی خارج کند

راه حل دیگر این است که or را داخل band ها اعمال کنیم و بین band های مختلف and اعمال کنیم. رابطه آن به این شکل می‌شود: $(1-(1-s)^b)^r$



دارای هم S_curve است:

تبدیل شده ← حساس به (۰/۲ , ۰/۸ , ۰/۸ , ۰/۲) (۰/۲ , ۰/۸ , ۰/۸ , ۰/۲)

حساس به (۰/۱۲ , ۰/۹۹ , ۰/۸ , ۰/۲)

مثال: ابتدا یک (4,4)OR_AND و بعد یک (4,4)AND_OR اعمال کنیم

(.2, .8, .8, .2) ⟶ (.2, .8, .9999996, .0008715) ↓

همانطور که مشاهده می‌شود بسیار خیلی خوبی داده، P_1 را به ۱ نزدیک کرده و P_2 را به صفر.

کون ایجام دادن:

Scanned with CamScanner

جریان داده‌ها (Data Streams)

در خیلی از کاربردها در ابتدا کل Data را در اختیار نداریم.

به چند دلیل: یک دلیل غالب این است که یک سیستم خارجی این دیتا را تولید می‌کند و در هر زمان قسمتی از دیتا را تولید می‌کند و ممکن است تا ابد در حال تولید دیتا باشد. اصطلاحاً گفته می‌شود دنیای ما infinite یا never ending است. مثلاً ترانزکشن‌های بانکی همیشه ادامه دارند. یا مثلاً موتور جستجوی گوگل، همواره کاربران پرس و جوهای خود را برای آن می‌فرستند.

یک دلیل دیگری نتواند حجم بزرگ دیتا باشد. در هیچ storage ای به صورت یکجایی نتوانید داشته باشید کل داده را.

در همه‌ی این موارد فرض بر این است که دیتا را تنها یک بار می‌توانید اسکن کنید. به این setting ها اجازه می‌دهند داده را $\log n$ بار اسکن کنید که به آن‌ها semi-streaming می‌گویند.

چالش‌های data streaming

① تعداد آیتم‌ها خیلی زیاد است، never ending است.

② بین دو تا آیتم زمان خیلی کمی برای پردازش آیتم قبلی داریم

③ resource های خیلی کمی در مقایسه با تعداد آیتم‌ها داریم

④ توزیعی که داده‌ها بر اساس آن تولید می‌شوند در طول زمان ممکن است تغییر کند

در big Data سه تا V داریم یک مجموعه داده big یا کلان به حساب می‌آید اگر این ۳ تا V را داشته باشد: variety (تنوع) / velocity (سرعت) / volume (حجم)

جریان داده مادر data streaming چون never ending است volume زیادی دارد

ویژگی velocity هم دارد، داده‌ها با نرخ بالایی وارد می‌شوند

ویژگی variety هم داریم به دلیل تولید دیتا با توزیع‌های مختلف

Scanned with CamScanner

جلسه دوازدهم - ۲۴ / ۱ / ۱۴۰۰

۷ | مهر | ۱۰ صفر ۱۴۴۲
دوشنبه | 28 Sep 2020

ادامه مبحث جریان داده‌ها:

معمولا یک فرن آرشیوی داریم. یک سری کوئری ۱۳۹۹ برحسب نیاز کاربر وارد می‌شوند.

به طور کلی محدودیت حافظه داریم. الگوریتمی که $O(n)$ فضا مصرف کند به حساب نمی‌شه در این جا.

الگوریتمی خوبه که حداکثر $\log n$ فضا مصرف بکنه (n حجم داده‌هاست)

یکی از task های مهم برای جریان داده‌ها: Sampling

حجم داده‌ها خیلی زیاد است پس یک sample از داده‌ها را مورد مطالعه قرار می‌دهیم

یک پنجره لغزان در نظر می‌گیریم و همیشه K تا عضر آخر داده‌ها را بررسی می‌کنیم (کاربر K را وارد می‌کند)

مسائل در جریان داده‌ها

← Filtering: می‌خواهیم یک آیتم را با تمام آیتم‌های مجموعه داده مقایسه کنیم

← شمارش تعداد آیتم‌های متمایز یک جریان داده

← محاسبه moment های مختلف یک جریان داده ← moment یه سری اطلاعات در مورد جریان داده‌ی ما هست مثلا توزیع frequency آیتم‌ها. چقدر تعداد تکرار آیتم‌های مختلف uniform است یا غیر uniform.

← پیدا کردن الگوهای پرتکرار در جریان داده

applications (کاربردها)

مثلا گوگل از تحلیل جریان داده‌ها خیلی استفاده می‌کند. مثلا یافتن پرس و جوهای پرتکرار، کمک می‌کند که مثلا کوئری‌های پرتکرار را index کند تا جواب را سریعتر برگرداند.

یا مثلا تحلیل کلیک‌ها ← مثلا کلیک‌هایی که در صفحه‌های مختلف ولی پیاپی اتفاق می‌افتد

یا مثلا در social Media ← هشتگ‌ها کمک می‌کند trend ها را پیدا کنیم

مثلا در sensor network ها ← اینکه بفهمیم یک داده شبیه‌ی زیاد شده

شبکه‌های تلفن ← داده‌های آن جمع‌آوری می‌شود و مورد بررسی قرار می‌گیرد، مثلا چه شماره‌های زیاد مورد تماس بوده‌اند.

تشخیص حملات DOS ← اگر Frequency یک IP مقدار زیاد باشد می‌تواند نشانه‌ای از حملات DOS باشد

Scanned with CamScanner

Sampling : یک نمونه تصادفی از جریان داده‌ی خواهیم بگیریم

دو دسته اصلی :

① با دنبست ثابت نمونه برداری انجام می دهیم : یعنی هر آیتم مجموعه داده را با مقدار احتمال ثابتی تصمیم می گیریم که آن را نگه داریم یا دور بیندازیم . تعداد سمپل ها متناسب با سایز جریان داده است .

② Sampling با اندازه ثابت ← تعداد سمپل ما ثابته و به اندازه جریان داده بستگی ندارد . به هر آیتم صرف نظر از اینکه قدیمی یا جدید است شانس یکسانی برای sample ما باشد .

حالت اول : سناریو ← جریان داده یک موتور جستجو (کوئری ها) — که در sample ما باشد .

هر آیتم یک سه تایی است : user , query , time

هر آیتم را با احتمال ۱/۱۰ نگه داریم و با احتمال ۹/۱۰ دور بیندازیم ← متناسب با ای هر آیتم یک عدد تصادفی از بازه [۰...۹] تولید کنیم و مثلا اگر ۰ بود نگه داریم و اگر ۰ نبود نگه نداریم . روش دیگه ← یک Hash Function تعریف کنیم که آیتم های ما را به صورت uniformly at random به یکی از n تا سطل ، مپ می کند . اگر آیتم به ۱ سطل خاص مپ شد آن را نگه میداریم .

فرض کنیم x تا آیتم داریم که ۱ بار آمده و d تا آیتم ۲ بار آمده اند پس به طور کلی $x+2d$ آیتم در جریان داده داریم

نسبت تعداد آیتم های که ۲ بار آمده به تعداد آیتم های متمایز : $d/(x+d)$

← حالا همین سوال را می خواهیم روی سمپل اعمال کنیم .

از آیتم هایی که یکبار آمده $x/10$ در سمپل می آیند . احتمال اینکه از یک جفت خاص هر دو در سمپل بیایند : $\frac{1}{10}\times\frac{1}{10}=\frac{1}{100}$

d تا جفت داریم و هر جفت با احتمال $\frac{1}{100}$ هر دوشان می آیند . پس تعداد آیتم هایی که ۲ بار در سمپل می آیند : $\frac{d}{100}$

احتمال اینکه از یک جفت فقط یکیشون بیاد : $d\left(\frac{1}{10}\times\frac{9}{10}+\frac{9}{10}\times\frac{1}{10}\right)=\frac{18}{100}d$

تعداد آیتم های جفت تقسیم بر آیتم های متمایز :

$$\frac{d/100}{\frac{x}{10}+\frac{d}{100}+\frac{18d}{100}}=\frac{d}{10x+19d}$$

جواب برس دو حالت داده اصلی و دل سمپل باهم متفاوت است .

یکی از مشکلات این Sampling این است که برای بعضی برس دوها خوب جواب می دهد

یک راه حل : user ها را سمپل کنیم . اگر user ای انتخاب شد همه‌ی کوئری های آن user انتخاب می شود

یک HF داریم که user ها را به اعداد ۱ تا a مپ می کند . میایم برای آیتم هایی triple بودند یک کلید در نظر می گیریم و در اینجا کلید را user قرار می دهیم . کلید به صورت یونیک یک آیتم را می دهد بلکه استاطین آیتم های مختلف را پیدا می کند . آیتم هایی که کلید یکسان دارند هم مربوط هستند و باهم سمپل می شوند . HF دارای a خانه است ، برای هر کلید حتی میایم Hash می کنیم اگر به b تا خانه خاص مپ شد انتخابش می کنیم .

مثلا اگر بخواهیم ۳۰ درصد جریان داده را مپ کنیم یک HF با ۱۰۰ خانه درست می کنیم ، اونایی که کلیدشان به ۳۰ خانه اول مپ می شود انتخاب می کنیم

Scanned with CamScanner

در روش دوم اجازه می دهیم اندازه سمپل رشد کند متناسب با اندازه جریان داده. روش دوم معمولا مفیدتر در عین حال که پاسخ به برخی کوئری های خاص در روش اول ساده تره. چون ویژگی حفظ داریم.

۹ | مهر ۱۲ صفر ۱۴۴۲ | چهارشنبه 30 Sep 2020

روش دوم Sampling:

فرض کنید n تا آیتم داریم در جریان داده و s تا آیتم را می خواهیم انتخاب کنیم به طوری که هر آیتم جریان داده با $\frac{s}{n}$ شانس انتخاب شود. مثال:

هر کدام از این ۵ تا با احتمال $\frac{2}{5}$ شانس انتخاب داشته باشند ۱۳۹۹

stream: a x c y z k c d e g ...

هر کدام از این ۷ تا باید احتمال $\frac{2}{7}$ داشته باشند که در سمپل بیایند

الگوریتم Reservoir Sampling

هدف الگوریتم: اینکه در هر زمان یک سمپل با اندازه s از جریان داده داشته باشیم به طوری که آیتم های ۱ تا این زمان همه شانس یکسانی داشته باشند که در سمپل بیایند

s عضو اول که می آیند همه آن ها را در sample set قرار می دهیم.

فرض کنیم n-1 عضو تا الان آمده اند. می خواهیم با ورود آیتم nام، دوباره به همه عناصر ۱ تا n شانس یکسانی بدهیم.

اگر با احتمال $\frac{s}{n}$ قرار شد آیتم جدید انتخاب کنیم باید یکی از آیتم های انتخاب شده قبلی را با احتمال uniform انتخاب کنیم و حذف کنیم و آیتم جدید را به جای آن در سمپل قرار می دهیم. (اگر با احتمال $1-\frac{s}{n}$ قرار شد آیتم جدید را دور بیندازیم هیچ کاری لازم نیست انجام بدهیم)

اثبات: با استقرا.

حالت پایه ← فرض کنیم s=n، هر کدام از آیتم های ۱ تا n با احتمال $\frac{s}{n}=1$ داخل sample set هستند.

فرض کنیم برای n تا آیتم این ویژگی برقرار است، هر کدام از این n تا با احتمال $\frac{s}{n}$ داخل sample set قرار دارند

می خواهیم ثابت کنیم برای n+1 آیتم هم این ویژگی برقرار است. فرض کنیم آیتم n+1 ام وارد شود، ۲ تا چیز را باید نشان بدهیم: ①

اینکه این آیتم با احتمال $\frac{s}{n+1}$ در sample set قرار می گیرد ② هر کدام از اعضای sample set با احتمال $\frac{s}{n+1}$ داخل سمپل باقی می ماند.

حالت اول اثباتش بدیهی است چون خود الگوریتم با همین احتمال $\frac{s}{n+1}$ تصمیم می گیرد آیتم n+1 ام را نگه دارد.

حالت دوم: با احتمال $1-\frac{s}{n+1}$ آیتم جدید به سمپل اضافه نمی شود که در این حالت هر آیتم موجود در سمپل با احتمال ۱ در سمپل باقی می ماند. اگر آیتم n+1 ام با احتمال $\frac{s}{n+1}$ تصمیم گرفته شود که به سمپل اضافه شود، هر آیتم موجود در سمپل با احتمال $\frac{1}{s}$ حذف می شود و با احتمال $1-\frac{1}{s}=\frac{s-1}{s}$ در سمپل باقی می ماند. پس احتمال باقی ماندن یک آیتم موجود در sample set برابر است با:

$$\left(1-\frac{s}{n+1}\right)+\left(\frac{s}{n+1}\right)\left(\frac{s-1}{s}\right)=\frac{n}{n+1}$$

آیتم قدیمی ما که در sample set هست با احتمال $\frac{s}{n}$ انتخاب شده در سمپل قرار گرفته. پس احتمال اینکه بعد از ورود آیتم n+1 آیتم قدیمی هم چنان در سمپل باقی بماند:

$$\frac{s}{n}\times\frac{n}{n+1}=\frac{s}{n+1}$$

Scanned with CamScanner

sliding window :

فرض می‌کنیم یک پنجره داریم که طول آن مشخص است و کار آن را set می‌کند. مدیر جریان داده آن را set می‌کند طول آن را N در نظر می‌گیریم.

درمورد N تا داده آخر است که concern داریم برای ما مهم است. آیتم‌های جدیدتر برای ما مهم‌تر است و فقط روی آن‌ها پردازش انجام می‌دهیم. N معمولا خیلی بزرگ است. مثلا داده‌های یک سال اخیر آمازون. مثلا ممکنه برای هر آیتم یک جریان داده ۰ و ۱ ای داشته باشه.

وقتی از sliding window استفاده نکنیم خیلی فرق نداره الگوریتم ما $\log n$ باشه یا خطی برحسب طول داده‌ها چون برای ما نهایت داده این دو با هم فرق چندانی ندارد.

مثال: یک جریان داده از ۰ و ۱ها داریم. می‌خواهیم تعداد ۱ها را در K بیت آخر بدانیم.

راه‌حل: بیاییم N داده آخر را همیشه نگه داریم ($K \leq N$) و تعداد ۱ها را در K تای آخر آن بشماریم.

space complexity : $O(N)$ ← برای سریوش exact داشتن می‌توانیم کمتر از این عمل کنیم

پس مجبوریم از روش‌های approximate استفاده کنیم چون $O(N)$ حافظه زیاد است.

اولین روش approximate : فرض کنیم توزیع جریان داده در همه جای آن uniform است

روز جهانی سالمندان

۱۱ | مهر جمعه

۱۴ صفر ۱۴۴۲

2 October 2020

دو تا شمارنده نگه داریم: S ← تعداد ۱ها از ابتدای جریان داده

Z ← ۰ها

سپس طول جریان داده : S+Z

S+Z متغیر تصادفی نیست (قطعی است همیشه) اما S و Z متغیر تصادفی اند

برای هر range دلخواه مثل N می‌خواهیم تخمین بزنیم داخل این range چند تا ۱ داریم : $N \times \frac{S}{S+Z}$

$$E\left(\frac{S}{S+Z}\right) = \frac{1}{S+Z} E(S) = \frac{1}{S+Z} \text{mean}(S)$$

امید ریاضی به صورت کسر ← امید ریاضی تعداد اعداد = امید اینکه آیتم اول ۱ باشد + امید اینکه آیتم دوم ۱ باشد + ...

با احتمال p آیتم اول ۱ است با فرض اینکه l طول جریان داده باشد

$E(S) = lp$ میانگین توزیع

$\Rightarrow E\left(\frac{S}{S+Z}\right) = \frac{lp}{l} = p$

$E\left(N \times \frac{S}{S+Z}\right) = NP$ ← گفته می‌شود تخمین ما unbiased است چون امید ریاضی تخمین ما

space complexity : $O(\log N)$ تعداد بیت‌های لازم برای ذخیره‌سازی از مرتبه $O(N)$ ← ذخیره کردن S و Z

(با این فرض که جریان داده ورودی از N است) (تعداد ۱ها و ۰ها را در پنجره لغزان نگه میداریم)

Scanned with CamScanner

این الگوریتم فضای کمتری می‌گیرد با داشتن ۲ تا بیانی:
① جواب approximate است
② فرض محدودکننده‌ی اینکه توزیع تولید دیتا تغییر نمی‌کند را باید داشته باشیم

۱۳۹۹

خطای تخمین به صورت linear و به صورت وارون با اندازه پنجره ارتباط دارد.

جلسه سیزدهم - ۲۹/۱/۱۴۰۰

الگوریتم جلسه قبل این محدودیت را داشت که توزیع تولید دیتا در طول جریان داده نباید تغییر کند و مدیشن space complexity آن بود که $O(\log N)$ بود.

الگوریتم DGIM

میزان خطای این الگوریتم هیچ‌گاه از ۵۰ درصد بیشتر نمی‌شود، مثلاً اگه ۱۰ تا ۱ داشته باشیم: 10 ± 5

تعریف خطا: درصد تفاوت جواب approximate با واقعی تقسیم بر جواب واقعی

از بلوک بندی استفاده می‌کند. جریان داده را به تعدادی بلوک (بالکت) تقسیم می‌کند. اندازه بلوک‌ها به صورت نمایی زیاد می‌شود. اندازه هر بلوک به تعداد ۱های درون بلوک است. از هر بلوک با اندازه مشخص حداکثر ۲ تا می‌توانیم داشته باشیم. مثلاً بلوک با سایز 2^2 حداکثر ۲ تا می‌توان داشت.

هر آیتم دارای timestamp است ما از آن برای اشاره به آن آیتم استفاده می‌کنیم. فقط برای N تای آخر را نگه میداریم
بقیه‌ش را به mode N می‌بریم

برای ذخیره هر timestamp، $O(\log N)$ حافظه نیاز داریم.

برای هر بلوک ۲ تا داده نگه میداریم: ① timestamp سمت راست‌ترین آیتم ← $O(\log N)$ حافظه
② اندازه بالکت را نگه میداریم ← تعداد ۱های درون بالکت ← $O(\log \log N)$ بیت نیاز ← فقط توان رو نگه میداریم
چون توان از ۲ است
تعداد ۱ها

دیگر بالکت‌ها ← اندازه‌شان توانی از ۲ است. هر بالکت با ۱ شروع و به ۱ ختم می‌شود.

نمی‌توانیم بالکت با اندازه 2^k نداشته باشیم ولی بالکت با اندازه 2^{k+1} داشته باشیم

بالکت‌های ما با اندازه sort می‌شوند. ما فقط نیاز داریم داخل پنجره را بالکت بندی کنیم. آیتم‌های جدیدی که می‌آیند را بالکت بندی می‌کنیم و آیتم‌های قدیمی که بالکت‌شان از پنجره ما خارج شد، اطلاعات مربوط به آن‌ها را فراموش می‌کنیم. وقتی timestamp سمت راست‌ترین آیتم از N بیشتر شد باید ۱ آیتم از پنجره خارج کنیم.
دیگر بالکت‌ها overlap ندارند. هر ۱ به یک بالکت تعلق دارد (دقیقاً ۱ بالکت). ۰ها ممکنه توی بالکت باشند یا نباشند.

Scanned with CamScanner

فرض کنیم یک آیتم جدید از سمت راست وارد شده

می خواهیم بالت بندی اش کنیم. اگر آیتمی که باید دور انداخته شود تنها آیتم موجود در بالت اش باشد، آن بالت را فراموش می کنیم

بعد چک می کنیم که آیتم وارد شده ۱ است یا ۰. اگر ۰ باشد هیچ عمل دیگری لازم نیست انجام بدهیم ولی اگر ۱ بود ⇐

۱ یک بالت با اندازه $2^0 = 1$ ایجاد می کنیم و فیلدهای آن را set می کنیم. timestamp آیتم جدید را به mode N می بریم و

تا هم استپ بالت جدید را برایش قرار می دهیم. اندازه آن هم ۱ است.

۲ اگر تعداد بالت های با اندازه ۱، با اضافه شدن بالت جدید، حداکثر ۲ بود، هیچ مشکلی ایجاد نمی شود. اگر تعداد

بالت های با اندازه ۱، با اضافه کردن بالت جدید به ۳ رسید باید ۲ تا بالت قدیمی تر با اندازه ۱ را با هم merge کنیم

و یک بالت بزرگتر با اندازه $2^1 = 2$ بسازیم. اندازه اش که مشخص است، timestamp بالت جدید حاصل از merge

برابر سمت راست ترین timestamp است که همان timestamp بالت سمت راستی است.

۳ اگر تعداد بالت های با اندازه ۲ به ۳ رسید دوباره قبل قبل باید merge انجام شود، همین طور الی آخر برای

بالت های با سایز بیشتر. فقط تا آخر جریان داده پیش بریم. (مثال از این قسمت را اسلاید)

سوال: کاربرد timeStamp چیست؟

کمک می کند بفهمیم یک بالت قدیمی از پنجره خارج شده یا نه. اگر timestamp بالت ۷ ما بزرگتر از timestamp

آخرین عنصر داخل پنجره باشد می فهمیم که آن بالت را باید دور بیندازیم

۳ تا مسئله وجود دارد: ① بالت بندی را چگونه انجام بدهیم که شروط را داشته باشد ② از روی بالت بندی چگونه تعداد 1 ها را تخمین بزنیم

③ نشان بدهیم که max خطای این الگوریتم 1/2 است.

تخمین ما به این صورت است:

۱- بالت هایی که کامل در پنجره اند را اندازه شان را در نظر می گیریم

۲- بالت آخر که کسری از آن در پنجره است را نصف اندازه اش را در نظر می گیریم

تخمین ما می شود جمع ۱ و ۲

کاربرد timestamp ← می فهمیم یک بالت قدیمی از پنجره خارج شده است

روز نیروی انتظامی

چرا خطای تخمین حداکثر ۵۰ درصد است؟

$$\text{خطا} = \frac{|\text{میزان خطا}|}{\text{مقدار واقعی}}$$

Scanned with CamScanner

یک حداکثر برای صورت کسر و یک حداقل برای مخرج بدست می آوریم ← پس حد بالای کسر خطا را بدست می آوریم.

برای بایت های به جز بایت آخر که خطایی نداریم

۱۳۹۹

فرض کنیم اندازه بایت آخر 2^3 باشد. در اینصورت حداکثر خطای ما 2^{3-1} خواهد شد.

۲ حالت خیلی بد داریم: از بایت آخر آن چپ که درون پنجره است همه اش ۱ باشد و آن چپ بیرون پنجره است همه اش ۰ باشد. ② از بایت آخر آن چپ درون پنجره است همه اش ۰ باشد و آن چپ بیرون پنجره است همه اش ۱ باشد.

محاسبه حداقل برای مخرج:

حداقل تعداد ۱ها با فرض اینکه اندازه بایت آخر 2^3 باشد:

$$1+2+4+\dots+2^{3-1} = 2^3-1$$

از بایت آخر حداقل به اندازه ۱ در پنجره هست:

$$2^3-1+1 = 2^3$$

حداکثر میزان خطا ← $$\text{کسر خطا} = \frac{2^{3-1}}{2^3} = \frac{1}{2} = 50\%$$

تعمیم این الگوریتم:

① حالتی که بخواهیم K تا آیتم آخر را بررسی کنیم ← K تا آیتم آخر و بایت هایی که داخل آن هستند را در نظر میگیریم

سوال: چگونه آخرین بایت که نمونش را میخواهیم را تشخیص بدهیم؟ از طریق time stamp

بایت ها را ایندکس میکنیم تا جایی که سخت ترین timestamp آن از K بزرگتر شود. این بایت از پنجره خارج است، بایت قبلی آن بایتی است که بیشتر از آن داخل پنجره است. با دور هر آیتم ۱، باید کل بایت ها ایندکس شود که تعدادشان $\log n$ است.

چرا space complexity این الگوریتم $O((\log n)^2)$ است؟ time complexity

چون تعداد بایت های ما حداکثر $O(\log n)$ است و برای هر بایت در بدترین حالت $O(\log n)$ space نگه میداریم.

② extension دیگر برای الگوریتم: آیا می توان این الگوریتم را طوری تغییر داد که از حالت ۰ و ۱ خارج بشه و آیتم های ما عددهای مختلف باشند؟

دوباره بایت بندی داریم و بایت ها مانند همون قانون ها را رعایت می کنند. اندازه شان به صورت نمایی زیادی می شود

روز دامپزشکی

اندازه بایت ها: 16 8 4 2 1

Scanned with CamScanner

در این حالت هر بالکت یا باید به قدرتی که رنگش دارد وفادار باشد

یا اینکه یک عضو داشته باشد. یک بالکت اگر یک عضو داشته باشد می تواند قدرتی که پیش ذکر شده را رعایت نکند

ولی اگر بیشتر از یک عضو داشته باشد باید حتما این قدرت را رعایت کند. (قدرت اینکه 16, 8, 4, 2, 1 ... باشد طوری

در این حالت اندازه بالکت برابر حاصل جمع عناصر داخل بالکت است.

← مثال از این حالت در اسلایدها

در این حالت برای هر بالکت ۳ تا نمد نگه می داریم:

① اندازه واقعی بالکت

② نوع بالکت (از روی جدول طول بالکت تعیین می شود)

③ timestamp سمت راست ترین آیتم

در این حالت هم تخمین که برای بالکت آخری زدیم اندازه واقعی اش تقسیم بر ۲ است.

بررسی مسائل دیگری که در مورد جریان داده:

① Filtering ← هدف فیلترینگ: یک آیتم خاص از جریان داده داریم می خواهیم بدانیم کلید آن در یک مجموعه خیلی بزرگ از آیتم ها موجود هست یا نه. (Bloom filters)

② Counting distinct Elements ← در جریان داده عناصر تکراری می توانیم داشته باشیم، در این مسئله می خواهیم تعداد عناصر متمایز را بشماریم.

③ Estimating Moments ← تخمین moment های مختلف عناصر جریان داده. moment موضوعی است که با تکرار آیتم های خاص ارتباط دارد. اینکه آیتم های مختلف تعداد دفعات تکرارشون هم نزدیک باشه.

④ شمارش آیتم های پرتکرار در جریان داده (counting frequent items) (دفعات)

Scanned with CamScanner

فیلترینگ جریان داده ۹

یک جریان داده داریم که هر عضو آن یک چندتایی (tuple) است و یک کلید هم دارد (یک عضو از این چندتایی، کلید است)

۱۳۹۹

یک مجموعه بزرگ به نام S داریم که شامل یک سری کلید معتبر است. می خواهیم بدانیم برای هر آیتم (tuple) از جریان داده آیا کلیدش در این مجموعه خیلی بزرگ (S) هست یا نه.

راه حل اولیه: سرچ ساده ـ اگر بخواهیم زیاد سرچ کنیم خوب است که اعضای S را sort کنیم که در $O(|S|)$ اینصورت برای هر search، $O(\log|S|)$ زمان می برد. ← کاربردهایشان دارند ad به کاربر

کاربرد: Email Spam Filtering ـ مثلا برای هر ایمیل باید چک کنند آدرس این ایمیل در مجموعه آدرس هایی از آدرس ها هست یا نه. یا مثلا می توانید لیستی از آدرس های خوب نگه دارید و با آن هر ایمیل را برچسب بزنید

راه حل: از hash function استفاده کنیم. فرض کنیم مجموعه S دارای m عضو است.

یک آرایه ۰ و ۱ ای n تایی در نظر می گیریم که همه خانه هایش در ابتدا ۰ هستند.

یک hash function انتخاب می کنیم که ورودی اش یک عضو از مجموعه S و خروجی اش یکی از خانه ها تا n-1 آرایه است.

روی تک تک اعضای S اعمال می کنیم hash function را و آن خانه هایی از S (بزرگ) که عضوی به آن ها map می شود را ۱ می کنیم.

وقتی جریان داده می آید برای هر عضو آن، آن عضو یا کلیدش را از hash function عبور می دهیم، اگر به خانه ۱ مپ شد نتیجه می گیریم این عضو از جریان داده داخل مجموعه S هست. اگر به خانه ۰ مپ شد نتیجه می گیریم آن عضو در مجموعه S نیست.

تعریف positive بودن برای یک آیتم ← اگر آیتم در مجموعه S باشد positive است

تعریف Negative بودن برای آیتم ← " " " نباشد Negative است

false positive ← آیتم ها واقعا در مجموعه S نیست اما الگوریتم ما به غلط بگوید که این آیتم در S هست.

سوال: آیا در این الگوریتم امکان False positive داریم؟

Scanned with CamScanner

بله امکان FP هست. به دلیل اینکه تابع hash دارای collision یا برخورد است و ممکنه دو تا عضو متفاوت به یک خانه map شوند. مثلا یک عضو از مجموعه S دارید که به یک خانه map شده و آن خانه را 1 کرده. بعد در جریان داده یک عضو متفاوت دارید ولی چون collision داریم ممکنه تابع hash، این عضو متفاوت رو هم به آن 1، map کند در نتیجه FP داریم.

سوال: آیا ممکن است False Negative داشته باشیم؟ یعنی یک آیتم واقعا در مجموعه S باشد اما الگوریتم بگوید که نیست.

خیر احتمال FN نیست. زیرا اگر یک آیتم دل‌خواه عضو مجموعه S باشد قبلا در مرحله پیش پردازش map شده به همان خانه و آن را 1 کرده و الان هم با مشاهده‌ی آن در جریان داده، به همان خانه map می‌شود.

Scanned with CamScanner

جلسه چهاردهم - ۱۴۰۰/۱/۳۱

گفتیم که مجموعه آیتم‌ها را با S نشان می‌دهیم

آرایه بیتی (که دارای n خانه است) را با B نشان می‌دهیم. به صورت پیش‌فرض خانه‌های B صفر هستند و هر خانه‌ای که برش حداقل یک عضو از مجموعه S توسط HF مپ شود، آن خانه ۱ می‌شود.

ایراد بلوم این است: ① احتمال FP چقدر است؟ ② چگونه می‌توان FP را کاهش داد؟

مثال: $|S| = 1 \text{ billion}$ $|B| = 8 \text{ billion}$

در این حالت احتمال FP حدود $\frac{1}{8}$ است. علت اینکه دقیقاً $\frac{1}{8}$ نیست و بزرگ collision تابع هش است.

Scanned with CamScanner

$|B| = n$ $|S| = m$

فرض کنید m تا تیر و n تا هدف داریم. هر تیر دقیقاً به یک هدف می‌خورد و هر تیر با احتمال یکسان به یکی از هدف‌ها می‌خورد. احتمال اینکه یک خانه خاص حداقل یک تیر بهش بخورد چقدر است؟ (احتمال 1 شدن خانه)

احتمال اینکه یک تیر به یک خانه خاص برسد ← $\frac{1}{n}$

" " " " نرسد ← $1-\frac{1}{n}$

احتمال اینکه هیچ تیری به یک خانه خاص نرسد ← $(1-\frac{1}{n})^m$

احتمال اینکه یک خانه خاص 1 شود ← $1-(1-\frac{1}{n})^m$

(حداقل یک تیر به آن بخورد)

$$1-(1-\frac{1}{n})^m = 1-\underbrace{(1-\frac{1}{n})^{n(m/n)}}_{n\to\infty} = 1-(\frac{1}{e})^{m/n} = 1-e^{-m/n}$$

امید ریاضی تعداد خانه‌های 1 ← $n\times(1-e^{-m/n})$

نسبت خانه‌های 1 ← $\frac{n\times(1-e^{-m/n})}{n} = \boxed{1-e^{-m/n}}$

احتمال FP می‌شود همان نسبت خانه‌های 1 : تعداد خانه‌های 1 تقسیم بر کل خانه‌ها $= 1-e^{-m/n}$

به برای مثال متغیر گذاشتیم و $\frac{1}{8}$ بدست آمده بود با این فرمول $1-e^{-1/8} = 0.11$ بدست می‌آید که از $\frac{1}{8}$ کوچکتر است.

چگونه می‌توانیم احتمال FP را کمتر کنیم؟

Bloom Filter

به جای اینکه از یک hash function استفاده کنیم از k تا hash function مستقل از هم استفاده می‌کنیم

در فاز preprocessing ابتدا همه اعضای B را صفر می‌کنیم و به ازای خواندن هر عضو از مجموعه S، آن عضو را

تک تک این k تا HF ها می‌کنیم به بردار B و خانه متناظرش را 1 می‌کنیم. در حالت بیشینه k خانه و در حالت کمینه 1 خانه را 1 می‌کنیم به ازای هر عضو از S

در زمان اجرا: هر آیتم از جریان داده که آمد تک تک k تا HF آن را map می‌کنیم. اگر تک تک HF ها به 1 خانه

یک map شدند می‌گوییم که آن آیتم x در مجموعه S وجود دارد ولی اگر حداقل به ازای یکی از HF ها به خانه صفر

map شدند می‌گوییم که آیتم x در مجموعه S نیست. پس هم FN نداریم و هم FP داریم.

بررسی احتمال FP با استفاده از Bloom Filter

روز بزرگداشت حافظ

(زمان preprocessing) می‌توانیم این فرض را بکنیم که به جای m تا تیر، km تا تیر داریم چون hash function ها از هم مستقل اند و آیتم ها هم مستقل از هم پرتاب می‌شوند.

Scanned with CamScanner

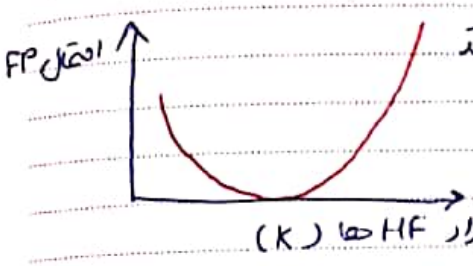
$1-e^{-km/n}$: پس نسبت خانه‌های 1 می‌شود

در run-time به ازای k تا HF چک کردن را انجام می‌دهیم.

۱۳۹۹

احتمال FP می‌شود : $(1-e^{-km/n})^k$

همیشه با افزایش k مقدار FP کمتر نمی‌شود، بلکه ابتدا کمتری شود اما از جایی به بعد بیشتر می‌شود. min تابع برحسب k در نقطه $\frac{n}{m}\ln(2)$ اتفاق می‌افتد



time complexity در زمان اجرا ← O(1) به ازای هر آیتم

① می‌توانیم یک بردار B بزرگ داشته باشیم و کل HF ها را مستقل از همدیگه روی این بردار B اعمال کنیم

② می‌توانیم بردار B را به k بخش تقسیم کنیم و هر کدام از HF ها را روی یکی از این زیربردارها اعمال کنیم. نتیجه تفاوتی ندارد. ($\frac{n}{k}$ هرکدام اندازه)

B را یکپارچه نگه می‌داریم ↑

$(1-e^{-km/n})^k$

حالتی که B را k بخش کنیم ← $(1-e^{-m/(n/k)})^k$

فرق این دو حالت در تعداد خانه‌ها و مقدار ریزی است که زیان می‌کنیم ولی ریکه داشتن یک B بزرگ راحت‌تر است (در پیاده‌سازی).

شمارش تعداد عناصر متمایز

مسئله: عناصر مجزا داده ما از یک مجموعه universe می‌آیند. هر عضو ممکنه 1 بار یا چند بار یا n بار بیاید. ما می‌خواهیم تعداد عناصر متمایزی که در مجموعه داده ما می‌آید را حساب کنیم.

یک راه ساده: برای هر عضو یک counter در نظر بگیریم (یا indicator در نظر بگیریم)

مقدار اولیه متناظر صفر است. وقتی عضو جدیدی می‌آید متناظرش را چک می‌کنیم، اگر 0 بود آن را 1 می‌کنیم و به دفعه counter اضافه می‌کنیم اگر متناظر 1 بود هیچ کاری نمی‌کنیم. یعنی به ازای هر عضو در مجموعه universe باید یک indicator تعریف کنیم، حداکثر چنین حافظه‌ای نداشته باشیم اما جواب exact فقط از همین راه بدست می‌آید. این راه حل O(n) حافظه نیاز دارد. (اگر مجموعه مقادیر ما متناهی نباشد)

کاربردهای این مسئله: مثلا یک کاربرد در crawler های web است که صفحه صفحه web را crawl و دانلود می‌کنیم

در این حالت می خواهیم صفحه های fake را وارد داده های خود کنیم.

به راه اینه که موقع crawl به سری تست های اولیه و سریع این ام

دهیم و بفهمیم آیا این صفحه fake است یا واقعی ؟

به راه اینه که تعداد کلمات متمایز آن صفحه را بشماریم. بررسی ها نشان داده صفحاتی که تعداد کلمه های متمایزشون

به تعداد کل کلمه ها خیلی زیاد یا خیلی کم باشه ، به احتمال زیاد fake هستند.

کاربرد دیگه ← یک شرکت تعداد url های distinct بشماری می خواهد بشمارد

کاربرد دیگه ← مثلا آمازون می خواهد تعداد فصول های متمایزی که فروخته را حساب کند ؟

مسئله ما : همانی که در اختیار داریم خیلی کمتر از O(n) است ، به دنبال راه حل approximate هستیم

روش Flajolet_Martin

در گام اول آیتم ها را با یک Hash function مپ می کنیم به اعداد ۱ تا N (یا ۰ تا N-1) ، برخورد هم نداریم

برای هر آیتم یک عایش $\log_2 N$ بیتی داریم ، یک عایش رشته ای داریم. تعداد صفرهایی که در

سمت راست آن عایش وجود دارد تا به اولین ۱ برسیم را r(a) می نامیم (a آیتم ماست)

مثال : $h(a)=12$ $\leftarrow$ $12=(1100)_2$ $\leftarrow$ $r(a)=2$ $\leftarrow$ اندازه‌ی tail

به ازای آیتم های جریان داده r را حساب کنیم و max این r ها را در R نگه میداریم

$R = \text{the maximum } r(a) \text{ seen}$ تنها چیزی که لازم نگه داریم R است

در واقع 2^R تخمین ما از تعداد عناصر متمایز است

فرض کنید تعداد واقعی عناصر متمایز ما m است. می خواهیم بدانیم این الگوریتم این ۲ تا ویژگی

را دارد : ① فرض کنید آیتمی باشد که اندازه tail اش r باشد اگر m خیلی بزرگ تر از 2^r باشد احتمال

دیدن این آیتم در جریان داده به ۱ میل می کند و احتمال ندیدنش به سمت صفر میل می کند.

② فرض کنید $m \ll 2^r$ ، احتمال اینکه این آیتم را در جریان داده ببینیم به سمت ۰ میل می کند

چرا اگر این دو ویژگی برقرار باشد R ما تخمین خوبی است ؟ فرض کنید ۲ تا شرط بالا برقرار است در این صورت ما مطمئنیم

tail ای که در جریان داده دیده ایم R است بین tail های با اندازه ۰ تا R را در جریان داده دیده ایم پس احتمال دیدنشان در جریان داده ۱ بوده

و احتمال ندیدنشان ۰ بوده پس $m \gg 2^r$. tail های با اندازه R+1 و بزرگتر را ندیده ایم پس احتمال دیدنشان ۱ است پس می توانیم

استدلال کنیم $m \ll 2^{R+1}$ و $m \ll 2^{R+2}$ و ...

Scanned with CamScanner

جلسه پانزدهم - ۱۴۰۰/۲/۵

$O(\log N)$ ← زمان‌بندی‌های لازم برای نگهداری هر آیتم

۱۳۹۹

احتمال اینکه برای یک آیتم خاص tail ای به اندازه‌ی حداقل r بسازیم چقدر است؟

$2^{\log(N)}$: تا بیت داریم و برای هرکدام ۲ حالت داریم ← فضای حالت

$2^{\log(N-r)}$ ← r بیت از سمت راست باید ۰ باشند و بقیه هم نسبت چی باشن

$$\text{احتمال} = \frac{2^{\log(N-r)}}{2^{\log(N)}} = \boxed{2^{-r}}$$

احتمال اینکه یک آیتم خاص tail ای به طول حداقل r نداشته باشد ← $\boxed{1-2^{-r}}$

احتمال اینکه هیچ‌کدام از m تا آیتم tail ای به طول حداقل r نداشته باشند ← $(1-2^{-r})^m$

↳ = احتمال اینکه ما در جریان داده‌مان tail ای به طول حداقل r نبینیم

$$(1-2^{-r})^m = \left((1-2^{-r})^{2^r}\right)^{(m2^{-r})} \approx e^{-m2^{-r}}$$

$$(1-2^{-r})^{2^r} = e^{-1}$$ با فرض اینکه r بزرگ باشد :

اگر $m \ll 2^r$ باشد : $m2^{-r} \to 0$

$$(1-2^{-r})^m \approx e^{-m2^{-r}} = e^0 = 1$$

اگر $m \gg 2^r$ باشد : $m2^{-r} \to \infty$

$$(1-2^{-r})^m \approx e^{-m2^{-r}} = 0$$

وقتی $m \ll 2^r$ باشد احتمال دیدن این tail به سمت ۱ میل می‌کند پس احتمال دیدن به سمت ۰ میل می‌کند

وقتی $m \gg 2^r$ " " "

اگر $k=0$ باشد، اعضای از مجموعه‌ای که 0 بار آمده اند $\leftarrow 0^0 = 0$

برای آن‌هایی که بار آمده اند (حداقل 1 بار) $\leftarrow 1 = (\text{عدد})^0$

پس این $\sum$ تعداد عناصر متمایز به ما را می‌دهد

اگر $k=1 \leftarrow$ 1st moment $\leftarrow$ تعداد کل عناصر جریان داده را نشان می‌دهد. به ازای هر آیتم تا

در مجموعه مادر، تعداد دفعاتی که آن آیتم در جریان داده آمده است را در نظر می‌گیریم و با هم جمع می‌کنیم و این عدد طول جریان داده را نشان می‌دهد

اگر $k=2 \leftarrow$ 2nd moment $\leftarrow$ surprise number

که به ما نشان می‌دهد که چه قدر این مقادیر مختلف m_i ها به هم نزدیک هستند یا از هم دیگه دور هستند.

هرچقدر حاصل این جمع کوچکتر باشد مقادیر m_i ها کم نزدیکترند.

مثال: فرض کنید 100 تا آیتم در جریان داده داریم که از 11 عضو مختلف هستند

item counts: 10, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9 surprise $S = 910$

item counts: 90, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 $(90)^2 + (1)^2 + \dots + (1)^2 = 8110 \leftarrow$ surprise

moment مرتبه دوم به ما می‌گوید عناصر مختلف چقدر کم تکرار یا frequency support

است یا چقدر از همدیگه دور است. مقدار بزرگ یا کوچک بودن moment مرتبه دوم نسبت به مقدار min و max خودش

باتوجه اینکه $\sum m_i$ ثابت است سنجیده می‌شود.

الگوریتم AMS

هدف این است که یک تخمین unbiased از moment مرتبه دوم

داشته باشیم.

X.el
X.val } تا جایی که حافظه به ما اجازه می‌دهد متغیرهای x تعریف می‌کنیم. هر x دو چیز را نگه میدارد

یک timestamp انتخاب می‌کنیم آیتم مربوطه آن را در X.el می‌گذاریم

و frequency آن آیتم از آن timestamp به بعد در جریان داده آمده را در X.val قرار می‌دهیم

فرض می‌کنیم جریان داده ما n تا آیتم دارد و بعد از بین آن‌ها می‌خواهیم moment مرتبه دوم را تخمین

بزنیم. برای هر x یک timestamp به طور uniformly at random انتخاب می‌کنیم. یعنی که در این

timestamp آمده را در X.el و از خود آن timestamp به بعد، تعداد تکرار را می‌شمریم و آن را در X.val می‌گذاریم.

Scanned with CamScanner

هر متغیر x مایک تخمین unbiased از moment
مرتبه دوم می دهد.

۱۳۹۹

برای هر x این مقدار را تعریف می کنیم:

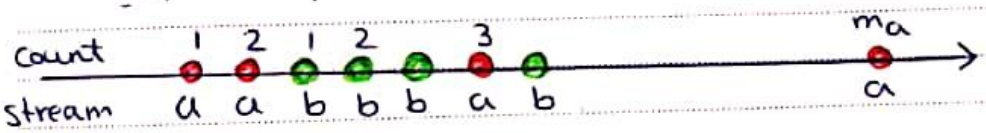
$$S = f(x) = n(2^c - 1)$$

c یک متغیر تصادفی است پس کل $f(x)$ یک متغیر تصادفی است و نشان خواهیم داد امید ریاضی آن برابر moment مرتبه دوم است.

نکته: به جایی که حافظه اجازه دهد چندین تا از این $f(x)$ها را نگه میداریم چون امید ریاضی هر کدام از این ها moment مرتبه دوم است، امید ریاضی میانگین آن ها هم moment مرتبه دوم خواهد بود.

تحلیل امید ریاضی

$c_t \leftarrow$ تعداد دفعاتی که آیتمی که در timestamp t آمده از آن جا (t) به بعد (تا آخر جریان داده) ظاهر می شود



$c_1 = m_a$ $\quad c_2 = m_a - 1$ $\quad c_3 = m_b$ $\quad c_4 = m_b - 1$

$c_5 = m_b - 2 = 2$

$$E[f(X)] = \frac{1}{n}\sum_{t=1}^{n} n(2^{c_t} - 1) = \frac{1}{n}\sum_i n(1 + 3 + 5 + \dots + 2m_i - 1)$$

یک تغیر متغیر روی $\sum$ می دهیم و به جای اینکه روی تک تک timestamp ها باشد روی آیتم های متمایز (distinct) در نظر می گیریم

آخرین occurance هر آیتم را که در نظر بگیریم c_t برای آن می شود $1 \leftarrow 2^{c_t - 1} = 1$

برای occurance یکی مانده به آخر $\leftarrow c_t = 2$ و $2^{c_t - 1} = 3$

برای دو تا مانده به آخر $\leftarrow c_t = 3$ و $2^{c_t - 1} = 5$

برای اولین occurance هر آیتم: $c_t = m_i$

$2^{c_t - 1} = 2m_i - 1$

شهادت امام رضا (ع) (۲۰۳ هـ ق) (تعطیل) - روز تربیت بدنی و ورزش

$$\sum_{j=1}^{m_i} (2j - 1) = 2\frac{m_i(m_i+1)}{2} - m_i = (m_i)^2 \Rightarrow E[f(x)] = \frac{1}{n}\sum_i n(m_i)^2 = \sum_i (m_i)^2 = S$$

moment مرتبه دوم

Scanned with CamScanner

این الگوریتم برای moment مرتبه سوم هم به همین شکل استفاده می‌شود و تنها تغییری که باید بدهیم تغییر مقدار $f(x)$ است:

$$f(x) = n(3c^2 - 3c + 1)$$

برای مرتبه kام $f(x)$ به این شکل است: $n(c^k - (c-1)^k)$

نکته: در عمل ما نمی‌دانیم که حافظه احاطه می‌دهد متغیر x تعریف می‌کنیم و $f(x)$ را محاسبه می‌کنیم. بعد xها را گروه‌بندی می‌کنیم و میانگین می‌گیریم بعد از این میانگین‌ها median (میانه) می‌گیریم. علت اینکه از میانه استفاده می‌کنیم: بعضی از xها ممکن است مقادیر پرت از moment مرتبه دوم به ما بدهد. و بنابراین از میانه استفاده می‌کنیم تا این مقادیر پرت را از بین ببریم.

نکته ۲: در دنیای واقعی اندازه جریان داده $n^{(n)}$ نامعلوم است یا اینکه می‌خواهیم برای nهای مختلف moment را حساب کنیم. چگونه می‌توانیم الگوریتم را تغییر دهیم که برای این موارد کار کند؟

۲ تا کار انجام می‌دهیم: ① اینکه همیشه طول جریان داده را در یک متغیر نگه می‌داریم

② وقتی timeStamp دل‌خواه n که می‌خواهیم moment مرتبه دوم را حساب کنیم، نیاز داریم مقادیر timestamp از ۱ تا n سمپل شده باشند (هر کدام با احتمال یکسان)

فرض کنیم مقدار سمپل‌ها رو از قبل می‌دونیم، حافظه‌مان هم مشخص است. مقداری از آن را به n (طول جریان داده) اختصاص می‌دهیم و مقداری را به متغیرهامون اختصاص می‌دهیم.

هر متغیر معادل sample گرفتن یک timestamp است. فرض کنید k تا متغیر در حافظه‌مون جای گیره پس وقتی timestamp دلخواه n که می‌خواهیم moment مرتبه دوم را برای آن حساب کنیم باید k تا متغیر را به صورت uniformly at random سمپل کرده باشیم. چگونه می‌توانیم این k تا سمپل را داشته باشیم با وجود اینکه طول جریان داده متغیره و آیتم جدیدی در هر لحظه وارد می‌شود؟ می‌توانیم از ملونه گیری با اندازه ثابت استفاده کنیم. (الگوریتم Reservoir Sampling)

هجرت پیامبر اکرم (ص) از مکه به مدینه

Scanned with CamScanner

۱۱۰۰

شمارش itemset ها در جریان داده

۱ یک حالت که جریان داده می‌تواند داشته باشد این است که هر عنصر آن یک آیتم است مثلا:

A B B B A A ....

می‌خواهیم frequency یا بسامد هر آیتم را حساب کنیم

۲ حالت دیگر این است که هر عنصر جریان داده مانیک تراکنش است (مجموعه‌ای از آیتم‌هاست)

basket _ transaction

در این حالت نه تنها آیتم‌های پرتکرار بلکه

itemset های پرتکرار را می‌خواهیم پیدا کنیم.

فعلا حالت اول را بررسی می‌کنیم که هر عنصر جریان داده یک آیتم است.

یک راه حل بدیهی برای شمارش آیتم‌های پرتکرار در صورتی که تعداد آیتم‌های متمایز موردنظرمان زیاد نباشد،

این است که برای هر آیتم یک جریان داده مستقل تولید کنیم. این جریان داده از ۰ و ۱ تشکیل شده.

اگر برای یک time Stamp خاص این آیتم در جریان داده آمد، ۱ داریم در این جریان داده جدید،

وگرنه اینصورت ۰ داریم. برای شمارش frequency یک آیتم با این جریان داده، کافیه تعداد ۱ها را

در این جریان داده جدید بشماریم.

Scanned with CamScanner

جلسه شانزدهم - ۱۴۰۰/۲/۷

در جلسه قبل روشی را برای یافتن آیتم‌های پرتکرار در جریان داده گفتیم. می‌توانیم این تکنیک را به itemset ها توسعه بدهیم اما همانطور که قبلا گفتیم تعداد itemset ها خیلی زیاد است و این روش خوب جواب نمی‌دهد. در ادامه یک روش اکتشافی (huristic) برای شمارش itemset های پرتکرار معرفی می‌کنیم. این روش به آیتم‌های جدیدتر وزن بیشتری می‌دهد به عبارتی اهمیت یک occurance متناسب با عمرش به صورت نمایی کاهش پیدا می‌کند.

فرض کنید یک جریان داده متشکل از اعداد داریم: $a_1, a_2, a_3, \dots$

می‌خواهیم جمع این اعداد را به دست آوریم با age یا عمر هر داده را در نظر بگیریم:

$$\sum_{i=1}^{t} a_i (1-c)^{t-i}$$

فرض کنید که الان در timeStamp = t هستیم. آیتمی که در timeStamp = i آمده عمرش برابر $t-i$ است، به این آیتم وزن $(1-c)^{t-i}$ می‌دهیم. هرچقدر عمرش بیشتر باشد، $1-c$ به توان بزرگتری می‌رسد و کوچکتر می‌شود وزن آن آیتم. c یک عدد ثابت است مثلا 10^{-6} و $1-c$ عددی بین ۰ و ۱ است. چون عمر هر آیتم $(t-i)$ در توان قرار دارد به این فرمول exponentially decaying می‌گویند.

فرض کنید یک آیتم جدید در زمان $t+1$ وارد می‌شود. برای آپدیت کردن جمع وزن‌دار، نسبت به آیتم جدید باید این کارها را انجام بدهیم:

۱. تک‌تک آیتم‌های قبلی یکی به عمرشان اضافه شده پس کل عبارت حاصل جمع را ضرب در $1-c$ می‌کنیم که معادل این است که تک‌تک a_i ها را در $1-c$ ضرب کنیم.

روز صادرات

Scanned with CamScanner

۲- کل این عبارت را با a_{t+1} جمع می‌کنیم که ضریب آن باید ۱ باشد چون عرآن صفر است

و $(1-c)^0 = 1$

برای بشمارش آیتم‌ها اینگونه عمل می‌کنیم:

برای هر آیتم دلخواه یهم چین فرمولی نگه میداریم:

$$\sum_{i=1}^{t} \delta_i \, (1-c)^{t-i}$$

روی زمان‌های مختلف از ۱ تا t، iterate می‌کنیم، برای هر زمان i یا آیتم در آن آمده که در اینصورت $\delta_i = 1$ وگرنه $\delta_i = 0$

حتی یک آیتم جدیدی می‌آید، تک تک فرمول‌هایی که برای آیتم‌های مختلف داریم را در $1-c$ ضرب می‌کنیم بعد به کانتری که متناظر با آیتم جدید است ۱ را اضافه می‌کنیم.

فرض کنید threshold برابر $\frac{1}{2}$ باشد. حداکثر چند تا فیلم خوب (که تعداد تکرارشان از threshold بیشتر است) می‌توانیم داشته باشیم؟ بیشترین حالت فقط ۱ فیلم می‌تواند باشد. حالا بیشترین حالت را می‌خواهیم حساب کنیم.

وزن‌هایی که آیتم‌های مختلف می‌توانند داشته باشند را با هم جمع می‌کنیم:

$$\sum_{t} (1-c)^t = \frac{1}{1-(1-c)} = 1/c$$

چون در هر timeStamp فقط به اندازه $(1-c)^{t-i}$ داریم چون δ_i فقط به ازای یک آیتم برابر ۱ می‌شود پس کافیه $\sum_{i=1}^{t} (1-c)^{t-i}$ داشته باشیم که معادل همین فرمول بالاست

بدترین حالت کل $\frac{1}{c}$ را فقط ۱ آیتم می‌گیرد در این صورت فقط یک فیلم پرتکرار داریم ولی حالتی که می‌خواهیم زیاد فیلم popular داشته باشیم به این صورت است که هرکدام از فیلم‌های ما دقیقاً $\frac{1}{2}$ وزن داشته باشند. در این صورت تعداد فیلم‌های خوب می‌شود $\frac{2}{c}$.

$$n \times \frac{1}{2} = \frac{1}{c} \Rightarrow n = \frac{2}{c}$$

Scanned with CamScanner

چگونه می‌توانیم این مورد را به itemset ها گسترش بدهیم؟

مشکل ما این است که تعداد itemset ها خیلی خیلی زیاد است و نمی‌توانیم برای هرکدام یک کانتر داشته باشیم

الگوریتم به این صورت عمل می‌کند: ① فرض کنید یک Basket جدید (B) بیاید. به تعداد کانتر در حافظه داریم که هرکدام را در C-1 ضرب می‌کنیم. ② بررسی می‌کنیم که این Basket جدید آیا آیتم‌هایی دارد که هنوز کانتر برای آن‌ها ایجاد نشده است؟ اگر چنین آیتم‌هایی وجود دارند برای آن‌ها یک کانتر ایجاد می‌کنیم و مقدار اولیه آن‌ها را برابر صفر قرار می‌دهیم. ③ به هر کانتری که متعلق به آیتم‌های تراکنش وارد شده است یکی اضافه می‌کنیم. ④ کانترهایی که مقدارشان از threshold (t) کوچکتر است حذف می‌کنیم. خیلی محافظه‌کارانه و سختگیرانه عمل می‌کنیم. نه تنها باید در انتها باید بیشتر باشند بلکه در هر لحظه باید از threshold بیشتر باشند.

⑤ بررسی کانترهای جدید برای itemset های دون ایجاد می‌کنیم. تک تک زیرمجموعه‌های B که اندازه‌شان بزرگتر از L است را در نظر می‌گیریم (S). برای هر زیرمجموعه S یک شرط را چک می‌کنیم اگر این شرط برقرار بود برایش یک کانتر ایجاد می‌کنیم. این شرط این است که همه‌ی زیرمجموعه‌های S که جزء خودش در timeStamp های قبلی بدون کانتر برایشون ایجاد شده باشه. مثلا برای {ز و ا} کانتر ایجاد می‌کنیم اگر قبلا برای {ز}، {ا} و {ز و ا} کانتر ایجاد شده باشه و در فاز drop کردن (مرحله ۴) حذف نشده باشند.

Scanned with CamScanner

ماشین لرنینگ روی جریان داده ها

مدل هایی که روی جریان داده ها خوب جواب داده اند

و خوب هستند، مدل های مبتنی بر درخت ۱۳۹۹

تقسیم هستند.

supervised Learning

هدف این است function ای را یاد بگیریم که input را به output مپ کند.

تعدادی داده آموزشی داریم : $(x_i, y_i = f(x_i))$ ← برداری از فیچرها

y_i می تواند label مربوط به کلاس باشد یا یک مقدار عددی باشد. f ارتباط بین x_i و y_i را بیان می کند

unsupervised ← دیتا برچسب ندارد

semi-supervised ← تعدادی از داده های train برچسب دارند

Active Learning ← بعد از اینکه یک پیش بینی در مورد برچسب دیتا ای که داریم داریم برچسب واقعی در اختیار ما قرار می گیرد و می توانیم از آن در جهت بهبود مدل استفاده کنیم

Regression ← Real number

Classification ← Categorical

Parse Tree , Ranking of Items ← Complex object

y

می خواهیم مدل مان generalization داشته باشد نه overfitting (برای داده های جدید خوب کار کند)

سال ۲۰۰۱ ← افزایش داده ها به بهبود مدل کمک می کند، بسیاری از مهم ترین عوامل خوب مدل است

سال ۲۰۱۷ ← برای بعضی از task ها مدل عمیق می باید با داده زیاد

Decision Tree Learning

می خواهیم روی یک ویژگی prediction انجام بدهیم براساس یک سری ویژگی های دیگر

n example (x_i, y_i)

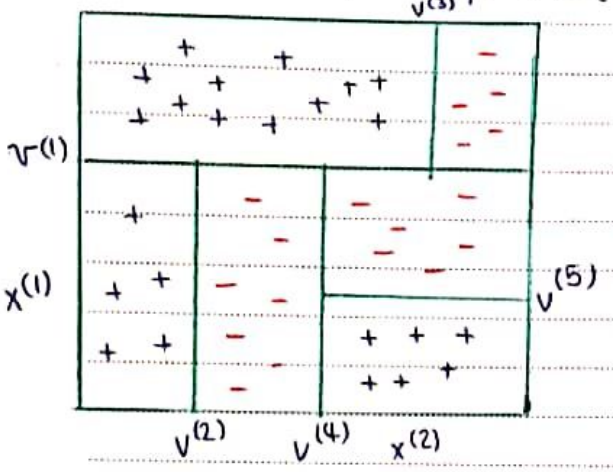
d feature : $x^{(1)}, x^{(2)}, \dots, x^{(d)}$

بردار d بعدی

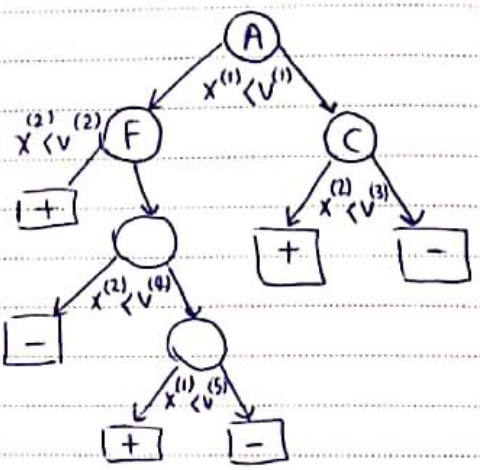
در ابتدا فرض می‌کنیم تمام دیتا در یک رئوس است. یک یک شرط را چک می‌کنیم و دیتا split می‌شود به ۲ یا چند تا.

(در این جا تمرکزما روی درخت تصمیم باینری است)

سوال: چگونه یک دیتایی که به ما داده شده را با درخت تصمیم پیش‌بینی کنیم $\hat{y}_i$ رو؟



feature space در درخت تصمیم:



درخت تصمیم چگونه ساخته می‌شود؟

اساس ساخت درخت تصمیم ← برای هر بود یک فیچر و یک threshold انتخاب کنیم

فرض کنیم در فرآیند ساخت به یک بود رسیده‌ایم. به ۲ سوال باید پاسخ دهید:

① آیا split کردن را ادامه بدهیم؟

② اگر قرار است split شود، کدام فیچر و با چه threshold ای استفاده شود؟

② اگر قرار نیست split شود، حالا که این بود یک بود برگ است، لیبل آن چیزی باشد؟

① چگونه بهترین split را پیدا کنیم؟

معمولا باید یک معیار را optimize کنیم اگر Regression ای task است معمول معیار purity (خلوص) خوب عمل می‌کند.

D کل دیتا D_L زیردرخت چپ D_R زیردرخت راست

برای split های مختلف یکسان است

به ازای هر فیچر خاص و یک مقدار خاص برای آن فیچر این مقدار را حساب می‌کنیم:

$$|D| \cdot var(D) - (|D_L| \cdot var(D_L) + |D_R| \cdot var(D_R))$$

هرکدام این عبارت را max کند که معادل min کردن عبارت سمت راست است

$$var(D) = \frac{1}{n} \sum_{i \in D} (y_i - \bar{y})^2$$

Scanned with CamScanner

اگر classification task باشد معیار Information Gain خوب عمل می‌کند.

در هر نود خاص که یک دیتای training به آن رسیده، برای فیچرهای مختلف $IG(Y|X)$ را حساب می‌کنیم، آن فیچری که بیشترین IG را برای نود انتخاب می‌کنیم. یعنی آن فیچر اطلاعات بیشتری درمورد Y به ما می‌دهد و Y بیشتر به آن وابسته است.

Entropy برای متغیر تصادفی X :

$$H(X) = -\sum_{j=1}^{m} P(x_j) \log P(x_j)$$

هرکدام از x_jها یکی از مقادیری است که متغیر X می‌تواند بگیرد

$P(x_j)$ ← احتمال اینکه متغیر X شما می‌تواند مقدار x_j را بگیرد

نکته: Entropy(X) وقتی زیاد می‌شود که مقادیر مختلفی که متغیر X می‌تواند بگیرد احتمال‌های یکسانی داشته باشد.

Entropy(X) وقتی کم می‌شود که X بعضی از مقادیر را با احتمال زیاد و بعضی مقادیر را با احتمال کم بگیرد

Entropy یک مقدار از میزان تصادفی بودن یک متغیر تصادفی است، ناشناختگی بالا را نشان می‌دهد

جلسه هفدهم - ۱۴۰۰/۲/۱۲

← مثالی از آنتروپی در اسلایدها

آنتروپی شرطی خاص : $H(Y|X=v)$

آنتروپی شرطی: $H(Y|X)=\sum_j P(X=v_j) H(Y|X=v_j)$ مقدار خاص متغیر X را می‌دانیم. با فرض اینکه مقدار متغیر X در اختیار ما قرار داده نشده می‌خواهیم بدانیم آنتروپی Y چقدر است

← میزان ناشناختگی Y را به ما می‌دهد با شرط اینکه X را داشته باشیم. حالا میزان ناشناختگی Y کمتر می‌شود یا تغییری نمی‌کند.

ما می‌خواهیم Y را به کمک فیچرها پیش‌بینی کنیم. فیچری که ناشناختگی Y را بیشتر کاهش بدهد مفید است

Scanned with CamScanner

یعنی چیزی که آنتروپی شرطی را حین کاهش بدهد مفید است.

$$IG(Y|X) = H(Y) - H(Y|X)$$

میزان اطلاعاتی که X درمورد Y به ما می دهد

می تواند با منفی آنتروپی شرطی $Y|X$ بیشتر داده بشه. چون هرچقدر X اطلاعات بیشتری درمورد Y بدهد، آنتروپی یا ناشناختگی Y درحضور X کمتر می شه و $H(Y|X)$ کمتر خواهد بود.

(2) گام دوم: شرط توقف (Stopping Criteria) ← در یک گره برگ هستیم و مقداری دیتا داریم. آیا تقسیم گره به دو تا زیرگره را ادامه دهیم یا همین جا متوقف شویم؟ یک معیار این است که میزان purity (خلوص) دیتای موجود در leaf را با یک threshold مقایسه کنیم. purity هم قبلا تعریف کردیم که با var تعریف می شد. $var(y) < \varepsilon$. یک معیار دیگه انتخاب example هایی که در گره است از یک تم شوند اگر بیشتر بود گره را تقسیم کنیم در غیر اینصورت متوقف شویم.

(3) گام سوم: How to predict? -

به سری دیتای آموزشی در گره برگ جمع شده، می خواهیم ببینیم چگونه از این دیتا برای تست استفاده کنیم؟ مثلا اگر مسئله ما Reggression باشد یه راه اینه که از مقدار y های داده های جمع شده در این برگ میانگین بگیریم. ② یه راه دیگه: یک مدل رگرسیون خطی با استفاده از مثال های موجود در برگ بسازیم.

توضیح: فرض می کنیم یک سری وزن های $w_1, w_2, \dots, w_d$ درطی مراحل learning یادگرفته شده اند و برچسب بردار x ها به صورت $w_1x_1 + w_2x_2 + \dots + w_dx_d = y$ تقریب می شود. با ضرب وزن ها در ترتیب ورودی خطی با استفاده از اصل حداقل مربعات صورت می گیرد. به ترکیب خطی از ستون های ماتریس ویژگی ها $(n \times d)$ پیدا می کنیم و آن را طوری تعیین می کنیم که کمترین فاصله را با بردار y (n خانه) داشته باشد.

درحالتی که مسئله classification باشد می توانیم از کلاس majority استفاده کنیم. مثلا n تا داده داریم که 3 تا از آن ها کلاس 1 و 2 تا کلاس 0 است. برای برگ داریم از بین آن ها کلاسی که بیشترین فراوانی را دارد به عنوان کلاس آن برگ در نظر می گیریم.

دو کاربردی که درخت تصمیم در مورد آن خوب جواب داده:

① user profile classfication ② Landing page bounce prediction ← صفحه های تبلیغی که در یک page می آید و می چک

Scanned with CamScanner

کاربر کلیک می‌کند دیگ صفحه بزرگ بازی می‌شود، ممکن است صفحه را تا آخر رود یا نرود. می‌خواهیم پیش‌بینی کنیم که آیا کاربر پایین می‌رود یا خیر.

۱۳۹۹

مزایای درخت تصمیم: ① easy to understand ← علت انتخاب‌شون تا حد زیادی مشخصه،

interpretable و explainable هستند

② easy to implement

③ هم برای classification هم برای Regression راحت استفاده می‌شوند

overfitting مهم‌ترین مشکل: حساس به خطا و بیگین مدل‌شان

Bagging ←

یکی از تکنیک‌ها برای جلوگیری از overfitting استفاده از ensembling است. مثلا ۵ تا مدل متفاوت داشته باشیم و پیش‌بینی آن‌ها را aggregate کنیم. (متفاوت بودن مدل‌ها همین)

یک تکنیک ensemble learning که روی درخت‌های تصمیم اعمال می‌شود bagging است.

K تا سمپل از دیتاست می‌گیرید. روی هرکدام از سمپل‌ها یک درخت تصمیم مستقل آموزش می‌دهید و خروجی‌ها را ترکیب می‌کنید

هر سمپل اینگونه ساخته می‌شود: n تا سمپل تصادفی با جایگذاری انتخاب می‌کنید.

(حدودا ۶۴ درصدشان غیرتکراری‌اند - ۳۳ درصد تکراری)

متن داده تست می‌آید هرکدام از درخت‌ها کلاس داده تست را پیش‌بینی می‌کنند بعد Majority_voting انجام می‌دهیم. (Random Forest)

بهبود Random Forest

روی ستون‌ها sampling انجام بدهیم برای اینکه diversity درخت‌های تصمیم بالا برود.

↓

به آن Feature bagging می‌گویند

از d تا فیچر که داریم k تا از آن‌ها را به‌طور تصادفی انتخاب کنیم. و هر درخت را با k تا فیچر بسازیم

پیش‌بینی اینکه diversity بالا برود هم روی sampling دیتا داریم که به آن data bagging می‌گویند و هم روی فیچرها sampling داریم که به آن Feature bagging می‌گویند. روشی که هر دوی این‌ها را دارد Random Forest می‌گویند.

Mining Decision Trees From Data Streams ← جدیدترین و معروف‌ترین روش‌هایی که برای classification جریان داده و معرفی شده‌اند روش‌های مبتنی بر درخت تصمیم هستند.

Scanned with CamScanner

① درحالت traditional مدول کل دیتا را داریم

② Random access داریم، اگر بخواهیم به یک دیتای خاص دسترسی داشته باشیم Hashing، indexing داریم

③ زمان زیادی داریم نسبت به حالت جریان داده‌ای

اما در مورد جریان داده حجم دیتا را یک جا نداریم، Random access نداریم و زمان هم کم است و حتی محاسبات ساده روی دیتا ممکن است زمان‌بر باشد

Hoeffding Trees

فرض کنید دیتای ما از نوع ریشه است.

اگر بخواهیم به طور قطعی بهترین split را پیدا کنیم باید تا بی نهایت صبر کنیم تا همه داده‌ها بیایند!

بنابراین باید به صورت غیرقطعی split بهتر را پیدا کنیم. تعدادی از example های ما که دارد وارد درخت شوند best split را پیدا کنیم. بعد از اینکه برای نود اول بهترین split پیدا شد، دیتای ما وارد دو تا بچه‌ی این نود جدید شده، بعد دوباره صبر می‌کنیم تا در بچه‌ها هم دیتای کافی که جمع شد بهترین split را برای آن‌ها هم پیدا کنیم.

Hoeffding Bound

↓

یک حد بالایی برای خطای تقریب ما می‌زند.

فرض کنید متغیر تصادفی X در range دونه‌ی روبه‌رو باشد: $[0, R]$

فرض کنید n تا مشاهده از X داریم و می‌خواهیم True Mean را a

حساب کنیم (میانگین واقعی X)

فضای حالت چون بزرگ است به جای اینکه کل آن را در نظر بگیریم n تا حالت را در نظر می‌گیریم و میانگین این مقادیر را حساب می‌کنیم. این میانگین محاسبه شده را a' (Empirical Mean) می‌نامیم.

a' می‌تواند تقریب خوبی از a به ما بدهد. می‌خواهیم خطای ما با احتمال حداقل $\delta - 1$، از ε کوچکتر باشد. Hoeffding Bound به ما می‌گوید n باید از این رابطه پیروی کند:

$$\varepsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

n باید حداقل این مقدار باشد ←

ε و δ را خود طراح تعیین می‌کند

Scanned with CamScanner

error guaranty ← $a' - a \leq \varepsilon$

تا وقتی برقرار است که توزیع شما که متغیر تصادفی از آن انتخاب می‌شود تغییر نکند

۱۳۹۹

فرض کنید X_a بهترین split را بعد از دیدن example n به ما بدهد و X_b دومین بهترین split را بدهد. G معیاری است که برای انتخاب فیچرها از روش استفاده می‌کنیم (مثلا IG) فیچر چیزی است که

$\bar{G}(X_a)$ ← $G(X_a)$ که از example n محاسبه شده

$\bar{G}(X_b)$ ← example n که از روی آن X_b پیدا می‌کنید

عده‌ای که با خودشان داریم خود وقتی است که کل جریان داده را استفاده کنیم

$\Delta\bar{G} = \bar{G}(X_a) - \bar{G}(X_b)$

$\Delta G = G(X_a) - G(X_b)$ ← میزان واقعی تفاوت score برای X_a و X_b

ε و δ را انتخاب کرده‌ایم و فرض کنیم n از رابطه زیر پیروی کند:

$$\varepsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

Hoeffding bound به ما می‌گوید با احتمال $1-\delta$:

$\Delta G > \Delta\bar{G} - \varepsilon$

فرض کنید شرط دیگری را هم چک کنیم، علاوه بر کافی بودن تعداد example ها شرط زیر را هم چک کنیم: $\Delta\bar{G} > \varepsilon$

نکته: هرچقدر δ کوچکتر باشد تعداد سمپل بیشتری نیاز داریم

در این صورت با احتمال $1-\delta$: $\Delta G > 0$

یعنی با احتمال خیلی زیاد روی کل جریان داده، X_a ما به درستی انتخاب شده و امتیاز بیشتری از X_b گرفته.

جلسه چهارم - ۱۴۰۰/۲/۱۹

← مثالی از مطالب جلسه قبل در اسلاید ۶ بیان شده

یک سیستم efficient براساس Hoeffding Tree ← VFDT

به طور مستقل چک می‌کند گره‌ها و کدام یک از split ها رسیده به پایان

اطلاعات را در گره‌ها ذخیره می‌کنیم در برگ‌ها ذخیره می‌کنیم چون برای گره‌های میانی تصمیم گرفته شده

سمپل جدید ← میانی

دیگر هیچ اتفاقی لازم نیست برایشان انجام بگیرد. نه split نه prediction و ...

Scanned with CamScanner

Mining Time-changing Data Stream

مدل درخت کار را باید ای اجام دهد تا نسبت به تغییرات توزیع داده سازگار باشد

① تغییر را متوجه شود

② ساختار درخت را adapt کند

هر الگوریتمی که کوشد Concept Drift کند باید این دو توانایی را داشته باشد

CVFDT

مشاهداتی که پشت CVFDT وجود دارد:

① درخت حاصل از توزیع داده در طول زمان تغییر پیدا می کند Hoeffding bound مربوط به error guaranty

معتبر است و لازم نیست که تغییر بکند (یعنی best split بهترین split تغییر کند)

② مشاهده کرده ایم که مثلا کوشش می زیشه best split بیست. آیا می توانیم حالا زیر درخت که در این گره خاص ریشه دارد را به یکباره کنار بگذاریم و شروع کنیم از الان به بعد این زیر درخت خاص را توسعه بدهیم؟ چرا می توانیم در عمل خوب جواب می دهد

چرا می توانیم؟ چون این زیر درخت outdated شده در مقابلش هنوز برای آینده تولید (بیان) همین،

زیر درخت جدید که داریم داده خوبی بیست نمیک زیر درخت هرچند outdate را کنار بگذاریم و جای آن اوراقی داشته باشیم

پس ما می توانیم بعد از اینکه کشف کردیم دیگ گره خاص best split انجام نشده، آن را کنار بگذاریم

بلکه آن را هم جیش نگه می داریم و از آن استفاده می کنیم تا زمانی که زیر درخت جدید مربوط به آن گره به اندازه کافی رشد کرده باشد. نتیجه این است که ممکنه روی یک گره چند تا زیر درخت داشته باشیم ← alternate.

یعنی چند تا alternate Tree داشته باشیم (درخت جایگزین - درختی که در جوان به تغییرات احتمالی تولید شده اند)

(۳) در زمان مناسب performance درخت outdate و alternate را مقایسه می کنیم و دقیق تر را نگه میداریم

الگوریتم CVFDT

برای هر نود در HT یک سری alternative tree خالی تعریف می شود

برای هر مثال (x, y) این مراحل انجام می گیرد:

① بیایم example را از ریشه رها می کنیم تا به یک نود برگ برسد. به هر نود که می رسیم ۲ تا کار انجام می دهد:

① اطلاعات آن نود را update می کنیم ← معنی این است که اطلاعات example ها را در نودهای میان هم ذخیره می کنیم اما در VFDT این کار را نمی کردیم

② اگر آن نود alternate Tree داشت اطلاعات را برای آن هم می فرستادیم (example را از آن هم عبور میدهیم)

Scanned with CamScanner

② example جدید را به sliding window اضافه می‌کنیم

ما یک sliding window داریم در CVFDT . مدل را براساس example هایی که در پنجره فعلی وجود دارند می‌سازیم

۱۳۹۹

③ قدیمی‌ترین عضو sliding window را حذف کرده و اثر آن را هم از درخت اصلی و هم از درخت‌های ALT حذف می‌کنیم. (از نظر منطقی ساختار درخت هم باید تغییر کند اما تغییر نمی‌دهیم، فقط اطلاعات را از نود پاک می‌کنیم)

④ تابع CVFDTGrow را call می‌کنیم: بررسی می‌کند برگ یا برگ‌ها به حالتی رسیده‌اند که split بشوند یا نه؟

⑤ تابع CheckSplitValidity: چک می‌کند آیا تغییری در آینده تولید شده یا انجام گرفته یا نه؟ (به طور مثال بعد از این 1000 example چک میکند)

برای این کار چک می‌کند آیا نودهای ما best split‌شان تغییر کرده یا نه.

CVFDTGrow

برای هر نودی که example از آن عبور می‌کند:

① اطلاعات ذخیره شده در آن نود را آپدیت می‌کنیم و اطلاعات example را به آن اضافه می‌کنیم.

② اگر آن نود در درخت ALT داشت تابع CDFDTGrow را برای آن هم صدا می‌زنیم.

بعد به سراغ گره‌های برگ می‌رویم. گره‌های برگی که داده ما وارد آن شده بررسی می‌کنیم. شرط‌های HT را برای آن بررسی می‌کنیم: اینکه به اندازه کافی example در آن برگ آمده یا نه و اینکه $\varepsilon > \Delta G$ هست یا نه

اگر به این نتیجه رسیدیم که وقتش رسیده این گره را split کنیم چیزی که بهترین split را می‌دهد انتخاب می‌کنیم

بر اساس آن Split را شکل می‌دهیم. اگر فیچری که بهترین split را می‌دهد null باشد معنیش این نیست که باید متوقف بشویم

forget old example

old example ـ این که از sliding window بیرون انداخته می‌شود

برای هر نود (حتی نودهای ALT) یک آبجکت unique درنظر می‌گیریم. ۲ دیتا دارد: ① در طول زمان افزایشی است

② یکتا است. نه تنها در یک درخت بلکه در همه درخت‌ها

Scanned with CamScanner

هر example مایک فیلد به نام MaxID داریم

وقتی example وارد درخت می شود از نودهای مختلف در درخت اصلی و درخت های ALT عبور می کند. آن نودی که بیشترین آیدی را داشته و این example از آن عبور کرده در MaxID ذخیره می شود.

Forget Example

میایم example را از نود ریشه رهایی کنیم، به هر نودی که می رسیم اگر آیدی آن کوچکتر از MaxID این example بود اطلاعات آن example را از این نود پاک می کنیم. اگر آن نود درخت ALT هم است، برای آن هم این کار را انجام می دهیم. یعنی example را از ریشه رها می کنیم و تابع Forget Example را برای درخت ALT هم صدا می زنیم.

چرا اگر آیدی یک نود از MaxID نود example ما کمتر باشد اطلاعات example را پاک می کنیم؟

چون به این معنی است که وقتی example ما آمده این نود در درخت وجود داشته است و example ما با او هم از آن عبور کرده چون مسیرش همان قبلی است، پس اطلاعات example در آن نود ذخیره شده باید پاک شود

CheckSplitValidity

تفاوت عده CVFDT با VFDT: در VFDT اطلاعات مربوط به example را فقط در برگ ها ذخیره می کنیم چون کلاس را نودهای میانی دیگه تعیین شده. و چک کردن شرط توقف برای نودهای میانی prediction ، split لازم نیست انجام شه

اما در CVFDT اطلاعات example را در گره های میانی هم ذخیره می کنیم.

بعد از اینکه اطلاعات در گره های میانی ذخیره شدند میایم به صورت periodically چک می کنیم:

آیا best split ما هنوز best split هست؟ اگر نود ریشه، هیچی وگرنه یک ALT Tree برای آن نود درست می کنیم

کمینه مقدار ALT Tree های فعلی شه

smoothly adjust to concept drift ← درخت های out-date را به یکباره کنار نمی گذاریم بلکه این تغییر از زیر درخت قدیم به جدید را به آرامی و به صورت smooth انجام می دهیم. یعنی وقتی درخت جدید شکل گرفت جوری تبدیلش می کنیم بلکه روشن می دهیم.

به‌طور عدادم کارایی درخت‌های ALT ای که empty نیستند (حداقل یک example گرفته‌اند) را چک می‌کنیم. (در مقایسه با درخت اصلی) • چند تا پارامتر را چک می‌کنیم: ① f ← بعد از هر f تا example چک می‌کنیم split هایی که داریم best هستیابه

② پارامترهایی که می‌خواهیم از timestamp کارایی ALT ها را بسنجیم

③ m ← پارامتری که کاربر تعیین می‌کند

وقتی که زمان چک کردن کارایی رسید، m تا example بعدی را به عنوان test در نظر می‌گیریم و کارایی درخت‌های ALT را با آن‌ها می‌سنجیم و درخت‌ها را با آن‌ها آموزش می‌دهیم.

درخت‌های ALT را به شرطی نگه می‌داریم که دقت و کارایی آن‌ها در طول زمان افزایشی باشد خود Tree outdated را به شرطی نگه میداریم که performance آن از همه درخت‌های ALT کمتر باشد. اگر کارایی اش در یک فاز تست دور تر از یک ALT بود جایگزین می‌شود.

* راهکارهای CVFDT برای adapt کردن Concept drift :

① استفاده از sliding window ← مدل را بر حسب n تا دیتای جدید می‌سازیم دیتای قدیمی از پنجره خارج است

② استفاده از تابع checksplitvalidity

③ به صورت Dynamic اندازه sliding window را تغییر می‌دهد. وضعیت درخت‌ها تقیم را چک می‌کند وقتی تعداد درخت‌های ALT خیلی زیاد است بدین معناست که نرخ تغییر توزیع بالاست پس در چنین شرایطی لازم است که اندازه sliding window کوچک باشد تا بتوانیم آخرین تغییرات را در مدل داشته باشیم اما اگر تعداد درخت‌های ALT کم باشد نشانه این است که تغییر در توزیع کم بوده پس باید sliding window را بزرگ کنیم در نتیجه می‌توانیم مدل را بر حسب آخرین تعداد دیتای بیشتری بسازیم

Scanned with CamScanner

جلسه نوزدهم - ۱۴۰۰/۲/۲۱

صرف نظر از classification تغییر توزیع داده‌ها، خود Concept drift مسئله‌ای مهیب و کاربردهای خودش رو داره.

اینکه ما متوجه بشویم دیتاهای ما توزیعشان تغییری پیدا کرده یا نه. (صرف نظر از اینکه مسئله‌ی اصلی classification داشته باشیم یا نه). چندتا تکنیک اکتشافی ساده وجود دارد

فرض کنید یک جریان داده داریم: $\epsilon_1, \epsilon_2, \epsilon_3, \dots$ و هرکدام از ϵ_iها یک عدد هستند. در Concept drift به صورت کلی ما می‌خواهیم در هر زمان t اگر تغییری در فرآیند تولید دیتا احساس کردیم یک سیگنال آلارم (alarm) صادر کنیم. در برخی مواردی توانیم مشخصات فرآیند جدید را هم در alarm به کاربر اعلام کنیم.

① تکنیک اول: CUSUM Test

یک جمع از عناصر جریان داده می‌گیرد. این جمع را به عنوان indicator برای هر timeStamp در نظر می‌گیرد و آن را با g_i نشان می‌دهیم:

$$g_0 = 0 \qquad g_t = \max(0, g_{t-1} + \epsilon_t - \nu)$$

$g_1, g_2, g_3, \dots$ ← این‌ها indicator‌های ما هستند و مشخص می‌کنند که باید alarm بفرستیم یا نه. با ورود هر عنصر جدید، آن را با حاصل جمع قبلی جمع می‌کند و یک مقدار حاصل و مشخص (ν) را از آن کم می‌کند. ν معمولاً آن چیزی است که انتظار داریم مقادیر جریان داده‌مان باشند. اگر مقدار جدید اضافه شده از مقدار مورد انتظار ما خیلی متفاوت باشد مقدار حاصل جمع خیلی تغییر می‌کند و اگر از یک threshold بیشتر شد alarm صادر می‌کنیم.

if $g_t > h$ then alarm & $g_t = 0$

ν و h پارامترهایی‌اند که توسط کاربر تعیین می‌شوند. معمولاً ν مثبت است و $\epsilon_t > \nu$

② تکنیک دوم: page Hinckley Test

$$g_0 = 0 \qquad g_t = g_{t-1} + (\epsilon_t - \nu)$$

مشابه روش قبل. با این تفاوت که مستقیماً با آستانه مقایسه نکرده.

$$G_t = \min(g_t)$$

G_t قرار داده: indicator ای که تا الان داشتیم را در

if $g_t - G_t > h$ then alarm and $g_t = 0$

بعد اومده $g_t - G_t$ را با آستانه مقایسه کرده:

Scanned with CamScanner

③ تکنیک سوم: Geometric Moving Average Test

$g_0 = 0 \qquad g_t = \lambda g_{t-1} + (1-\lambda) E_t$

if $g_t > h$ then alarm and $g_t = 0$

در واقع یک جور میانگین گیری وزن دار است.

کاربر ۲ تا پارامتر را باید set کند: پارامتر h و λ.

پارامتر λ اهمیت عضو جدید را نسبت به عناصر قبلی به ما نشان می دهد

Scanned with CamScanner

۱۸ | آبان | ۲۲ ربیع الاول ۱۴۴۲
یکشنبه | 8 Nov 2020

خوشه‌بندی "clustring"

تعاریف مسئله:

ورودی: مجموعه‌ای از objectها و یک معیار (notion) از فاصله و گاهی اوقات تعداد خوشه‌ها

یک notion از فاصله داریم که ۲ ورودی (object) می‌گیرد و فاصله آن‌ها را خروجی می‌دهد (خروجی یک عدد غیرمنفی است)

فرض می‌کنیم distance و similarity بین ۰ و ۱ هستند: similarity = 1 - distance

Scanned with CamScanner

هدف در خوشه بندی: یک گروه بندی یا پارتیشن بندی یا افراز روی دیتاپوینت هامون اعمال کنیم

به طوری که این گروه بندی ۲ تا شرط را رعایت کند:

① دیتاهایی که داخل یک گروه (cluster) هستند فاصله هایشان از هم هرچقدر minimize شود. (خیلی کم شبیه باشند)

② نقاطی که به کلاسترهای مختلف تعلق دارند فاصله شان از هم max شود (غیر شبیه باشند)

مشکلی که در یک دیتا داریم برای خوشه بندی:

① تعداد نقاط خیلی زیاد است ② هر object (نقطه) در یک فضای high dimensional است.

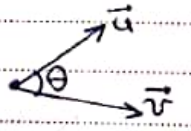
Scanned with CamScanner

فاصله اقلیدسی ← برای بردارها بیان می‌شود. بردار اختلاف دو بردار را حساب می‌کنیم و نرم ۲ آن بردار را حساب می‌کنیم. (معادل هر نقطه یک بردار وجود دارد در فضای اقلیدسی ← تناظر یک به یک، به ازای هر نقطه می‌توانیم یک بردار از مبدأ به سمت آن نقطه رسم کنیم) [معیار فاصله jacard را قبلا معرفی کردیم]

cosine Similarity

هر چقدر زاویه بین دو بردار کوچکتر باشد، به هم نزدیکترند. پس این زاویه معیاری از distance است. پس cos این زاویه را به عنوان معیاری از شباهت در نظر می‌گیریم که هر تی بردارها روی هم دیگر باشند زاویه بین شان ۰ است بیشترین شباهت را با هم دارند و cos بیشترین مقدار خود را دارد.

$$\cos\theta = \frac{u \cdot v}{|u||v|}$$



edit distance

یک سری عملیات پایه‌ای در نظر می‌گیریم. مثلا افزودن کاراکتر، حذف کاراکتر و جایگزین کردن کاراکتر. (update)

در edit distance اندازه می‌گیریم چند تا از این عملیات پایه‌ای لازم است تا یک رشته به یک رشته دیگر تبدیل شود

در مورد گراف‌ها هم کاربرد دارد (در مورد گراف از فاصله cos و اقلیدسی نمی‌توان استفاده کرد) حذف یال، اضافه کردن یال و تغییر برچسب یال را عملیات پایه‌ای در نظر می‌گیرند

outlier ← نقطه‌ای که در همسایگی اش به اندازه کافی همسایه وجود ندارد

نقطه‌ای که به هیچ کدام از cluster ها تعلق ندارد

مثال clustering: خوشه‌بندی اجرام آسمانی، خوشه‌بندی CD های موسیقی، خوشه‌بندی اسناد (Documents)

مسئله clustering یک مسئله NP_hard است. پیدا کردن جواب optimal در NP_hard است

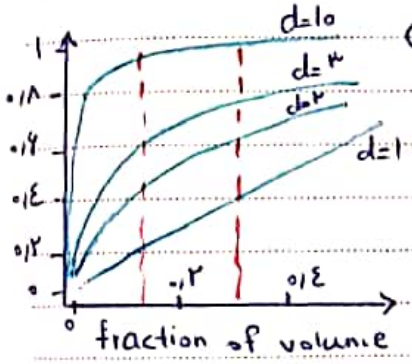
3. optimization

در عمل این مسئله NP-hard را حل می‌کنیم. از روش‌های اکتشافی استفاده می‌کنیم و مرتبه‌های $O(n \log n)$ و $O(n^2)$ را حل دارند. تا زمانی که تعداد dimension ها کم است مشکلی نداریم اما با افزایش تعداد ابعاد مشکلاتی رخ می‌دهد. از جمله: Curse of dimensionality

فرض کنید یک مکعب $1 \times 1 \times 1$ داریم و ۱۰۰۰ نقطه را به طور کاملا یکنواخت در آن پخش کرده‌ایم. هر نقطه $\frac{1}{1000}$ از حجم مکعب را به خود اختصاص داده. اگر بخواهیم ۱۰ نقطه‌ی همسایه‌ی یک نقطه را پیدا کنیم حجم $\frac{1}{1000} = \frac{10}{10000}$ نیاز است

اگر تعداد ابعاد ۲ باشد ضلع مکعب می‌شود ۰/۰۳

" " ۳ " " " ۰/۱ ← ۳ برای حالت قبلی



معمولا با fraction کوچکی از حجم را می‌خواهیم و در این قسمت رشته بیشتر است در نمودار روبه رو:

روش‌های clustring

① Hierarchical (سلسله مراتبی): در مرحله اول هر کدام از داده‌ها یک cluster اند

خود این روش‌ها به ۲ دسته تقسیم می‌شوند: الف) Agglomerative (تجمعی) ← از پایین به بالا این سلسله مراتب را می‌سازند. ب) روش‌های مبتنی بر تقسیم (Divisive) ← این سلسله مراتب را از بالا به پایین می‌سازند

② Point Assignment: به سری کلاستر در نظر می‌گیرند و برای هر کلاستر یک Representetive یا نماینده داریم. هر کدام از نقاط را بر حسب فاصله‌شان از Representetive ها، به کلاستر ها assign می‌کنند. به صورت تکراری این فرآیند را ادامه می‌دهند. نماینده‌ها را آپدیت می‌کنند و از روی نماینده‌های آپدیت شده دوباره point assignment را انجام می‌دهند.

"Hierachical clustering"

به چند سوال باید پاسخ بدهیم:

① اگر cluster بیش از یک نقطه داشته باشد چگونه آن را نمایندگی کنیم؟

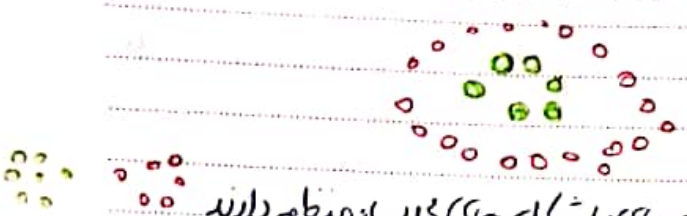
② چه معیاری فاصله کلاستر ها را اندازه بگیریم و دو کلاستری که کم‌ترین هستند را پیدا کنیم؟

③ کی فرآیند ترکیب cluster ها را ادامه بدهیم؟

Scanned with CamScanner

روش های سلسله مراتبی زمانی خوبی کار می کنند که

cluster های ما شکل های پیچیده داشته باشند



روش های point assignment وقتی خوب کار می کنند که cluster های ما شکل های گرد و منظم دارند

کلا وقتی خوشه ها شکل گرد دارند هم clustering هم dimension Reduction راحت می شود. مثلا شکل های

گرد بالا را در ۲ بعد در نظر بگیرید خیلی راحت می توانید تک بعدی دلخواه در نظر بگیرید و نقطه ها را به فضای ۱ بعدی مپ کنید

سوال: چگونه خوشه ها را نمایش می کنیم؟

اگه در فضای برداری باشیم راه حل راحت است. فضای برداری نسبت به جمع برداری و ضرب اسکالر بسته است

بردارهای عناصر یک خوشه را جمع کرده و میانگین می گیریم ← مرکز خوشه را با میانگین بردارهای آن می دهیم به این نقطه

centroid می گوییم

سوال: فاصله خوشه ها را چگونه تعریف کنیم؟ فاصله centroid ها (مثال خواسته شود)

جلسه بیستم - ۱۴۰۰/۲/۲۶

مثالی از داده ای که برداری نیست ← گراف ها ← مثلا میانگین دو تا گراف تعریف نشده

فاصله دو تا گراف رو می تونیم تعریف کنیم اما میانگین نه

① راه حل: تعریف clustroid ← یکی از اعضای خود cluster چون ترکیب خطی اعضا تعریف نشده نیست

① یه راه تعیین clusteroid ← فاصله هر عضو از سایر اعضا را حساب می کنیم. max این فاصله ها را

می گیریم و بین این max ها min می گیریم اون داده می شه clustroid

② میانگین فاصله هر عضو از عضا را حساب می کنیم، اونی که کمترین میانگین را دارد ← clustroid

③ کمترین جمع توان ۲ فاصله از سایر نقاط $\arg\min_{c} \sum_{x \in C} d(x,c)^2$

تفاوت ۲ و ۳ ← در ۳، outlier ها بیشتر جریمه می شوند

تفاوت centroid ، clusteroid ← centroid عضو یکی از اعضای کلاستر نبوده اما clusteroid باید یکی از اعضا باشد

Scanned with CamScanner

clusterهایی که هم نزدیکترین هستند باهم ادغام نیم

۱۳۹۹

فاصله بین clusterها هم می‌شود فاصله clustroidها.

2 یک راه‌حل دیگر برای داده‌هایی که بردار نیستند این است که اصلا centroid ، clustroid تعریف نکنیم. فاصله را اینگونه تعریف کنیم:

هر عضو از کلاستر اول را فاصله‌اش را از هر عضو از کلاستر دوم به دست آورید بعد مثلا می‌توانیم min این فاصله‌ها را به عنوان فاصله دو کلاستر در نظر بگیریم یا average فاصله‌ها یا max فاصله‌ها را به عنوان فاصله دو کلاستر در نظر بگیریم (معمولا min رایج‌تره)

3 راه‌حل سوم برای تشخیص اینکه کدام cluster ها باید باهم ترکیب بشوند ← cohesion (همبستگی) (انسجام)

آن کلاسترهایی که به نظرتان رسد مناسب اند برای ادغام، ادغام می‌کنید. برای این کلاستر به وجود آمده نمره cohesion (انسجام) را برای آن حساب می‌کنیم. برای همه کلاسترهای جدید این کار را می‌کنیم و کلاستری که بیشترین امتیاز چسبندگی را دارد انتخاب می‌کنیم

معیارهای cohesion : ① نقطه clusterها را در نظر بگیریم. هرچقدر نقطه بیشتر cohesion کمتر

نقطه در cluster ← صدکتر فاصله‌ای که ۲ تا data point دارند در کلاستر.

② average distance ← بین هر ۲ نقطه داخل cluster فاصله را حساب کرده و جمع کنیم و تقسیم بر تعدادشان کنیم. هرچقدر average distance بیشتر cohision کمتر

③ چگالی کلاستر را حساب می‌کنیم. تعریف چگالی ← تقسیم تعداد اعضای کلاستر بر قطر کلاستر

تعریف دوم چگالی ← تقسیم تعداد اعضای کلاستر بر average distance

هرچقدر چگالی بیشتر باشد colesion هم بیشتر

سوال: حالت اول: حتی داریم خوشه‌بندی را از پایین به بالا انجام می‌دهیم تا کجا ادامه بدهیم؟

پیدا کردن اینکه متوقف بشویم، تا بالا ادامه بدهیم - در عمل زیاد استفاده می‌شود

Scanned with CamScanner

حالت دوم: K

به راه دیگه ای اگر تعداد کلاسترها را بدانیم از پایین شروع می‌کنیم به ادغام کردن تا به تعداد k برسیم

حالت سوم ← به سری روش‌های اکتشافی استفاده میکنیم. مثلا کمترین کلاستر حدیدی که ساخته می‌شود به اندازه کافی خوب نباشد. مثلا قطرش از یه حدی بیشتر بود متوقف میشویم. یا مثلا چگالی اش از یه حدی کمتر باشه (ما دوست داریم کلاسترها dense باشند)

یا مثلا نسبت به کلاسترهای قبلی که پیدا کرده‌ایم کیفیت آن کاهش زیادی پیدا کند (معیار: قطر، چگالی و...

سوال: از بین این روش‌ها کدام بهتر است؟ کاملا به دیتای شما بستگی دارد ← مثال حد اسلاید بعدی

K_means clustering

۱ فرض می‌کنیم در فضای برداری هستیم، بنابراین گیر فاز است

۲ تعداد خوشه‌ها معلوم است

۳ دو مرحله اصلی دارند این روش‌ها: (روش‌های point assignment)

الف) مقداردهی اولیه به کلاسترها

ب) assign کردن نقاط به کلاسترها

۲۳ آبان جمعه

۲۷ ربیع الاول ۱۴۴۲

13 Nov 2020

point assignment

مهم‌ترین نقطه ضعف این دسته از الگوریتم‌ها حساسیت زیادشان به مقادیر اولیه representetive ها می‌باشد. نقطه ضعف دیگه: بهینگی فرض در مورد شکل خوشه‌ها

برای بهبود مقداردهی اولیه به representetive ها از روش‌های اکتشافی استفاده می‌شود.

الگوریتم k_means

k تا نقطه انتخاب می‌کنید ← به راه اینکه k تا نقطه تصادفی انتخاب کنیم (uniformly at Random)

راه دیگه: نقطه اول را کاملا تصادفی، نقطه دوم ← نقطه‌ای که از نقطه اول بیشترین فاصله را دارد

نقطه i ام ← به گونه‌ای که از ۱-i نقطه قبلی بیشترین فاصله را داشته باشد

Scanned with CamScanner

ولی در عمل خیلی خوب جواب می‌دهد.

راه سوم ← کلاستر اول uniformly at random ، برای کلاستر دوم احتمالی عمل کنیم

کلاستر دوم ← به هر نقطه p انتخاب نشده تا الان یک احتمال $D(p)^2$ متناسب با فاصله نقطه p تا representetive اول اختصاص می‌دهیم و هر نقطه را متناسب با احتمالش انتخاب می‌کنیم

هرچقدر فاصله بیشتر احتمال انتخاب بیشتر

K_Means++

نقطه مثبت الگوریتم‌های point assignment مثل k-means ، k-means++ این است که پیچیدگی محاسباتی کمی دارند در نتیجه وقتی داده‌ها زیاد هستند خوب عمل می‌کنیم (زمان کمی می‌برند)

k-means++ سعی دارد نقطه ضعف الگوریتم‌های point assignment را حذف کند و جنبه مثبت را در عین حال نگه دارد:

حساسیت به مراکز اولیه

clustering روی دیتا را با خود k-means انجام می‌دهیم بعد به sample کوچک از دیتا می‌آییم روی این sample یک الگوریتم با time Complexity بالا اجرا می‌کنیم تا کیفیت خوشه‌بندی خوب باشد. بعد centroid ها را از این خوشه‌ها استخراج کرده و به عنوان مقادیر اولیه k تا کلاستر مان در نظر می‌گیریم

مثلا سلسله مراتبی

سوال: تعداد sample ها باید چقدر باشد؟ $O(K \log n)$ ← n تعداد داده‌ها و K تعداد خوشه‌ها

سوال: چگونه sample ها را انتخاب کنیم؟

نود اول را به صورت رندوم انتخاب می‌کنیم برای انتخاب دوم، به هر point یک احتمال می‌دهیم متناسب با $D(p)^2$ ← فاصله بین نقطه و نزدیکترین نقطه انتخاب شده (که قبلا انتخاب شده اند)

بقیه هر نود باقی مانده را با این احتمال انتخاب می‌کنیم

مثلا نود سوم را کی می‌خواهیم انتخاب کنیم فاصله هر نود از 2 تا نود اول و دوم را که انتخاب شده اند حساب می‌کنیم هرکدام کمتر بود می‌شود $D(p)$ ، آن نود را با احتمال $D(p)^2$ انتخاب می‌کنیم

روز کتاب . کتاب خوانی و کتابدار

Scanned with CamScanner

point assignment ← گام صفر گفته نشد

1) در گام اول هر نقطه را به کلاستری که به centroid آن نزدیکتر است اختصاص می دهیم.

2) بعد از اینکه همه داده ها به کلاستر assign شدند، centroid ها را update می کنیم برحسب اعضای جدیدی که آن کلاستر گرفته. مثلا avg اعضای کلاستر می شه representetive جدیدی برای آن کلاستر

3) دوباره همه نقاط را به کلاسترهای جدیدشان اختصاص می دهیم.

گام 2 و 3 را آنقدر تکرار می کنیم که هیچ این رخ دهد.

هیچ این: centroid ها دیگه تغییر نکنند زیاد. نرم 2 می گیریم از تفاضل بردار centroid جدید و قبلی. این اعداد را برای کلاسترهای مختلف با هم جمع می کنیم دارنیک threshold اگر کمتر بود متوقف می شویم

② نقاط باین کلاسترهای ما جابجا نشوند. همان کلاستری که در iteration قبلی دارند بعد از آپدیت centroid ها هم دارند ③ تعداد مشخصی iteration برای گام 2 و 3 الگوریتم set کنیم.

مرتبه زمانی K_means : $O(n*k*i*d)$ d تعداد ابعاد، i تعداد iteration ها

یک متغیر n و d و k ثابت اند ← زمان خطی دارد : $O(n)$

← مثال از K_means در اسلایدها

سوال: K مناسب را چگونه انتخاب کنیم؟

اندازه دیگر جلد K شروع می کنیم و آن را افزایش می دهیم. به ازای هر K الگوریتم خوشه بندی را اعمال می کنیم از خروجی الگوریتم average distance را محاسبه می کنیم. تا جایی K را زیاد می کنیم که با زیاد کردن آن کاهش شدید avg dis داشته باشیم. هرگاه کاهش avg dis کم شد دیگه K را زیاد نمی کنیم.

چرا؟ اولش که تعداد خوشه ها از تعداد واقعی آن ها کمتره تعداد زیادی long distance داریم long distance ها خیلی زیادند چون فاصله داده های خوشه های مختلف اند.

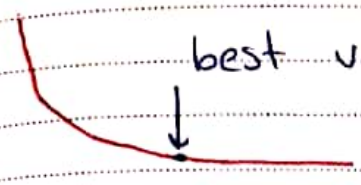
وقتی تعداد خوشه ها را زیاد می کنیم long distance ها حذف می شوند و از بین می روند. در نتیجه کاهش شدید

Scanned with CamScanner

long distance دیگه avg dis داریم. وقتی تعداد مناسب کلاسترها رو پیدا کردیم یعنی

نداریم. پس وقتی k رو زیادی کنیم short distance ها حذف می‌شوند و خیلی avg dis را

کم می‌کنند



الگوریتم BFR

این الگوریتم K_means را توسعه می‌دهد به حالتی که دیتای ما خیلی خیلی زیاده. K_means بر این

فرض بود که کل دیتای ما در یک حافظه جای می‌گیرد.

در این الگوریتم دیتای ما به صورت bag bag میاد و داخل حافظه می‌نشیند. ← حُسن الگوریتم

عیب الگوریتم ← یه سری فرض در مورد دیتای شما و ساختار آن دارد. اگر این فرض ها برقرار باشند کیفیت

خوشه بندی زیاد است اما اگر برقرار نباشد کیفیت کم است.

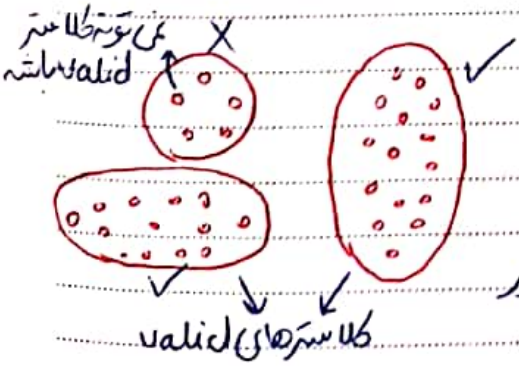
فرض که الگوریتم می‌کند:

کلاسترهای ما در امتداد یکی از dimension ها توزیع شده‌اند و از توزیع نرمال پیروی می‌کنند

فرض کنیم نقاط در فضای ۲ بعدی هستند. شکل کلاسترهای ما باید به گونه‌ای باشد که در امتداد محورها

یا محور افقی توزیع شده باشند. یعنی به شکل بیضی‌ای در امتداد یکی از

محورها باشد:



و خود کلاسترهم باید توزیع نرمال داشته باشند، یعنی در راستای اون محور

مصله هیچ نقطه‌ها از centroid از توزیع نرمال باید پیروی کند.

وقتی می‌گیم در امتداد یکی از محورها باشه یعنی در امتداد بقیه محورها واریانس دیتا خیلی پایین باشه به گونه‌ای

که بتوان از آن صرف نظر کرد.

Scanned with CamScanner

هدف این الگوریتم این است که به سری representetive خوب برای k تا کلاستر اولیه پیدا کند

و بعد از آن representetive های اولیه استفاده کند و point assignment را انجام بدهد

از این کلاستر های اولیه می توانیم برای مقداردهی اولیه کلاستر هامون استفاده کنیم یا اینکه خودشان

کلاستر اولیه کلاستر تک نقطه ای هم باشند

این فرض را هم برای این می کند که دیتای ما خیلی اینوه است دیگر نمی توانیم track تک تک دیتا ها

را نگه داریم در نتیجه با این فرض می توانیم به صورت ساده و بهینه track دیتا را نگه داریم.

Scanned with CamScanner

جلسه بیست و یکم - ۱۴۰۰/۲/۲۸

ایده کلی BFR ← به جای اینکه تک تک داده‌ها را ذخیره کنیم، یک اثر (trace) از دیتاپوینت‌هایی که به اندازه کافی به هم نزدیک هستند نگه دارد. (یک representetive برای همه‌ی آن‌ها داشته باشد) در نتیجه حافظه مصرفی از O(n) خیلی کمتری می‌شود.

مراحل الگوریتم

① (K را می‌دانیم) ← می‌آییم K تا centroid را مقداردهی اولیه می‌کنیم

② دیتا را bag به bag می‌خوانیم و وارد حافظه اصلی می‌کنیم.

گام ۳ تا ۵ ← پردازش دیتای خوانده شده

③ برای هر دیتاپوینت نزدیکترین centroid به آن را پیدا کرده و اگر این فاصله از یک threshold کمتر بود assignment را انجام می‌دهیم

④ دیتاهای assign نشده به همراه دیتاهای اینزوله‌ای که از bag های قبلی باقی مانده را گروه‌بندی می‌کنیم. این نقاط هم نزدیک اند اما به هیچ کدام از کلاسترهای اصلی ما نزدیک نیستند. به آن‌ها: compressed set می‌گوییم

⑤ compressed set هایی که قابل ادغام هستند باهم ادغام می‌کنیم.

این مراحل را آنقدر تکرار می‌کنیم تا کل دیتا خوشه‌بندی شود.

در گام اول برای تعیین centroid ها bag اول را می‌خوانیم و به حافظه اصلی می‌آوریم. برای انتخاب

centroid های توانیم از همان روش های قبلی استفاده کنیم. مثلا نقطه اول تصادفی، نقطه دوم

8 را طوری که فاصله اش از نقطه اول max باشد و ... . یا مثلا k تا نقطه تصادفی انتخاب می کنیم.

9 هم چنین می توانیم از k-means++ استفاده کنیم. از خود bag اول یک sample ایجاد می کنیم

10 و روی آن سعیل یک الگوریتم clustering با هزینه بالا و کیفیت خوب اعمال می کنیم (روش Hierachical یا density)

11 به ری centroid استخراج می کنیم و به عنوان مقادیر اولیه به الگوریتم clustering می دهیم

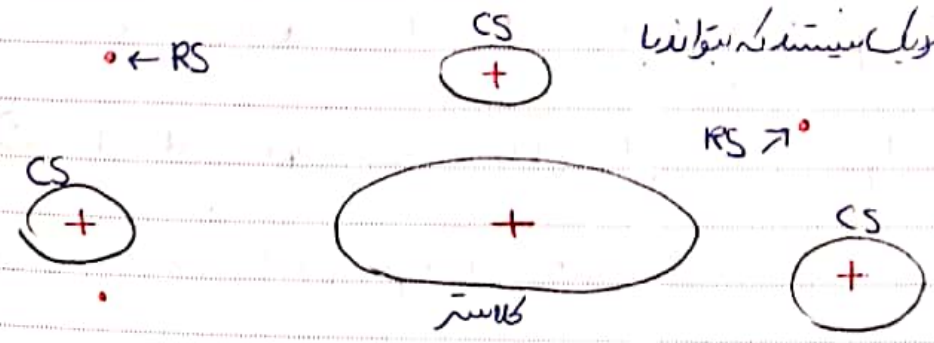
12 ۳ تا مجموعه داریم در این الگوریتم:

① Discard Set (DS) ← نقاطی که به اندازه کافی به representetive یک کلاستر نزدیک اند و عضو آن کلاستر هستند. این ها هم دیگر خیلی نزدیکند چون داخل یک کلاستر اند به centroid کلاستر هم نزدیک اند. ما می توانیم این ها را خلاصه کنیم و لازم نیست تک تک ذخیره کنیم.

② Compressed Set (CS) ← مجموعه نقاطی که خودشان هم نزدیکند اما به هیچ کلاستری تعلق ندارند

می توانیم آن ها را خلاصه کنیم و ... representetive اینجا با centroid های کلاستر جایشان فرق دارد چون به هیچ کلاستری تعلق نگرفته اند.

③ Retained Set (RS) ← می توانیم آن ها را خلاصه کنیم. نقاط این مجموعه اولا به هیچ کلاستری نزدیک نیستند. دوما به هیچ نقطه دیگری نزدیک نیستند که بتوانند با آن گروه شوند.



Scanned with CamScanner

چگونه یک مجموعه از نقاط را خلاصه کنیم؟

اطلاعات مربوط به خود نقاط را نگه نمی داریم. خود این نقاط را تک تک نگه نمی داریم. مواردی که نگه می داریم:

① مقدار نقاط ← N

② بردار جمع (SUM) ← خانه ی nام بردار جمع ← حاصل جمع خانه ی nام اعضا را نگه می دارد

③ بردار SUMSQ ← حاصل جمع توان دوم را نگه می دارد. خانه ی nام ← حاصل جمع توان دو خانه nام اعضا

حافظه ای که نیاز داریم ← $2d+1$ که d مقدار ابعاد بردارهاست

به دست آوردن بردار Average ← بردار SUM تقسیم بر مقدار

این الگوریتم به سری اطلاعات در مورد کلاستر ها (DS ها) نیاز دارد. یکبار آن ها میانگین (centroid) و یکی هم بردار انحراف از معیار است

$$\text{بردار واریانس} = (SUMSQ_i/N) - (SUM_i/N)^2$$

۱۲ جمعه ۲۰ ذی القعده ۱۴۳۹

3 Aug 2018

برای هر dimension مستقل از سایر بعدها انحراف معیار در آن dimension را حساب می کنیم

گام ۳ تا ۵:

۳ نقاطی که به اندازه کافی به یک کلاستر نزدیک اند assign می کنیم

۴ یک الگوریتم clustering روی داده های باقی مانده و هم چنین نقاط پرت مانده اعمال می کنیم.

نقاط پرت bag اگر که کلاستر نشده اند از iteration های قبلی (نه DS و نه CS بوده اند)

۵ اطلاعات Discard set را آپدیت می کنیم.

ابتدا بررسی می کنیم که CS های جدیدی که از خوشه بندی در مرحله ۴ ساخته شده اند با CS های قدیمی از iteration های قبلی قابل merge شدن هستند یا نه. اگر شدنی باشد انجام می گیرد. دوم اطلاعات مربوط به کلاستر ها مون رو آپدیت می کنیم. اطلاعات مربوط به CS ها را هم آپدیت می کنیم. اگر CS ها با هم ادغام نشده بودند اطلاعاتشان آپدیت می شود و اگر نقطه ای به یک کلاستر اضافه شد اطلاعات آن کلاستر را هم آپدیت می کنیم.

Scanned with CamScanner

در bag آخر به طور اختیاری می‌توانیم این دو کار را انجام بدهیم:

بیایم هر CS را به کلاستری که به آن نزدیک‌تر از بقیه کلاسترهاست، assign کنیم و هر نقطه ایزوله در RS را می‌توانیم به عنوان نقطه ایزوله در نظر بگیریم یا اینکه آن را هم به نزدیک‌ترین کلاستر assign کنیم

سوال: با توجه به توزیعی که دیتای ما در این حا دارد از چه معیار فاصله‌ای خوب است که استفاده کنیم؟

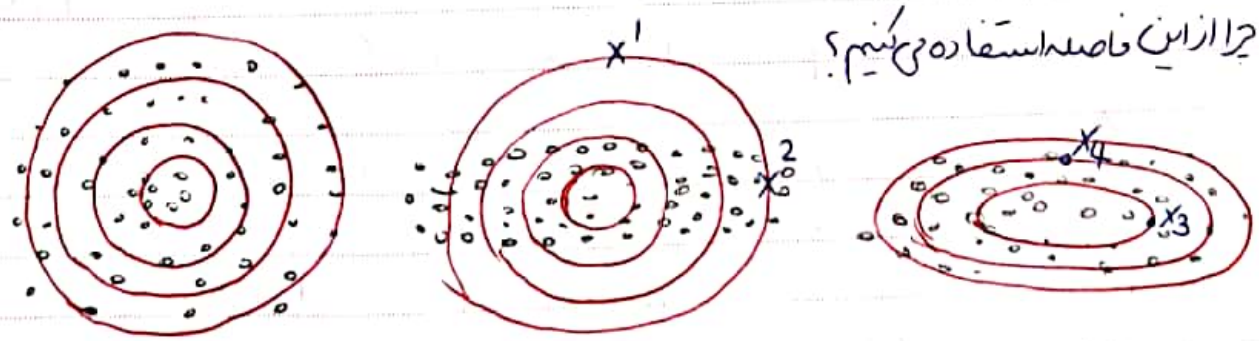
می‌دانیم که اعضای کلاستر در راستای یکی از کلاسترهای ما توزیع نرمال دارد

از Mahalanobis Distance استفاده می‌کنیم:

$$d(x,C) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - c_i}{\sigma_i}\right)^2}$$

شبیه فاصله اقلیدسی است با این تفاوت که در هر dimension بعد از حاسبه تفاضل، آن را با انحراف معیار دیتا در آن dimension، نرمال می‌کنیم و بعد به توان ۲ می‌رسانیم.

چرا از این فاصله استفاده می‌کنیم؟



در شکل سمت چپ توزیع دیتا در هر dimension ها نرمال است. نقاطی که روی یک دایره هستند فاصله یکسانی از مرکز خوشه دارند. شکل وسط توزیع دیتا در محور افقی normal است. در شکل وسط واریانس دیتا در راستای

Scanned with CamScanner

محور عمودی خیلی کم است و دیتا در راستای محور افقی align شده. نقطه 1 و 2 طبق معیار اقلیدسی یا منهتن فاصله یکسانی تا مبدأ پیدا می‌کنند. آیا معقول است که این دو نقطه فاصله یکسانی از مبدأ داشته باشند؟ خیر،

معقول است که بگوییم x_1 فاصله بیشتری دارد چون در راستای محور افقی نقاط از همدیگه دورند، کلاستر جا در راستای محور افقی کشیده شده. فاصله‌ها در راستای محور افقی از یکدیگر زیاد است و اگه فاصله‌ای که x_2 از مبدأ دارد فاصله زیادی به حساب می‌آید اما در راستای محور عمودی نقاط از همدیگر دور نیستند و اگه فاصله که x_1 دارد، زیاد حسوب می‌شود.

پس معیار فاصله ما باید خودش را با توزیع دیتا در dimension‌های مختلف تطبیق بدهد در نتیجه فاصله اقلیدسی برای حالت‌هایی که توزیع دیتا در dimension‌های مختلف متفاوت است، مناسب نیست. (فاصله منهتن و ... هم مناسب نیست)

در صورت فاصله Mahalanobis نقاطی که روی یک بیضی قرار دارن فاصله Mahalanobis یکسانی از مرکز کلاستر دارند

در اینجا فاصله‌ها در محور x‌ها بیشتر بزرگی تقسیم می‌شود چون واریانس زیادی در محور x داریم و فاصله‌ها در محور y‌ها را بیشتر بزرگی تقسیم کرده‌ایم چون واریانس کمی در راستای محور y داریم.

وقتی از Mahalanobis distance استفاده کرده‌ایم یک فاصله بزرگ در راستای محور x‌ها را برگشته با یک فاصله بزرگ در راستای محور y‌ها. مثلا فاصله اقلیدسی نقطه x_3 و x_4 از مبدأ تقریباً با هم برابر است ولی فاصله Mahalanobis نقطه x_3 از x_4 نسبت به مبدأ کمتر است و منطقی هم هست چون x_3 در راستای محور x‌ها قرار گرفته که در این راستا نقاط فاصله‌ی زیادی از هم دارند در نتیجه اینقدر فاصله، فاصله‌ی زیادی حسوب می‌شود در حالی که x_4 در راستای محور y قرار گرفته و این فاصله در آن راستا فاصله زیادی حسوب می‌شود.

Scanned with CamScanner

9 Aug 2018

8 Aug 2018

سوال: دوتا Compressed set را چه زمانی باهم ترکیب کنیم؟

به ازاء اینکه دوتا CS را ترکیب کنیم diameter کلاستر حاصل را حساب کنیم، اگر مناسب بود که داریم اما برای این باید تک تک نقاط رانگه داریم تا بتوانیم فاصله دو نقطه‌ای که از هم دورترند را پیدا کنیم. اما در این صورت باید $O(N)$ اختصاص بدهیم که نمی خواهیم.

برد اور یانس که قبلا محاسبه کردیم

به جای اینکه دقیق حساب کنیم تخمین می زنیم ← از واریانس استفاده می کنیم، σ^2 را جایگزین قطر می کنیم. اگر واریانس مجموعه ترکیب شده از یک threshold کمتر بود آن را می پذیریم.

بنابراین در این جا مشخص می شود که چرا این الگوریتم این فرض را در مورد داده‌ها می کرد که در راستای یکی از محورها align شده. چون اگر align شده نبودند برای تخمین قطر مشکلات زیادی داشتیم و راه حل اصلی این بود که تک تک نقاط رانگه داریم و

۱۹ جمعه ۲۷ ذی القعده ۱۴۳۹

10 Aug 2018

اگر می خواستیم تخمین بزنیم باید واریانس و کوواریانس را در بعدهای مختلف تخمین می زدیم و لازم بود ماتریس کوواریانس را نگه میداشتیم و $O(d^2)$ می شود حافظه‌مون.

وقتی دیتا در راستای یکی محورها align شده، واریانس می تواند تقریب خوبی از diameter به ما بدهد.

نقطه ضعف الگوریتم‌های point assignment این است که یک سری فرض در مورد ساختار و شکل خوشه‌های کشف مثلا Kmeans فرض می کند کلاسترها خوب و well formed حول یک مرکز است یا BFR فرض می کند که کلاسترها در راستای یکی از بعدها کشیده شده‌اند. حتی BFR کلاسترهایی که کشیده اند اما در راستای ! بعد کشیده نشده اند را هم بتوان نمی کند مثل این کلاستر:

چون برای محاسبه تقاطع نیاز نه که کوواریانس را نگه داریم.

Scanned with CamScanner

12 Aug 2018 — 11 Aug 2018

الگوریتم CURE

نقطه ضعف الگوریتم های point assignment این است که در مورد ساختار و شکل داده ها فرض هایی می کنند

در این الگوریتم سعی شده در عین حال که پیچیدگی زمانی هنوز linear است الگوریتم بتواند روی کلاسترهای پیچیده هم اعمال شود. (به جز point assignment است مثل hierarchical, density basal پیچیدگی زمان بالا ندارد)

چرا الگوریتم های point assignment کلاسترهای خوب و منظم دارند؟ چون یک نقطه به سلطان centroid داریم و فاصله نقاط را از آن چک می کنیم ← شکل کروی تولید می شود

حالا به جای اینکه یک centroid در نظر بگیریم چند تا centroid در نظر بگیریم. در این صورت دنیای خوشه ای شمارتر کیبی از مثلا ۵ تا کره می شود که شکل نامنظمی خواهد داشت.

الگوریتم CURE دو فاز (scan) اصلی روی دنیای ورودی دارد:

pass1

① عمل ۰ ← یک سمپل تصادفی از دنیا انتخاب می کنیم به طوری که در حافظه اصلی جا بگیرد و آن را در حافظه اصلی لودی کنید.

② عمل ۱ ← یک الگوریتم با پیچیدگی زمان بالا (مثلا Hierachical) روی این سمپل اعمال می کنیم و یک خوشه بندی اولیه به دست می آوریم.

③ عمل ۲ ← برای هر کلاستر به دست آمده centroid انتخاب می کنیم. مثلا برای هر کدام ۵ تا انتخاب می کنیم. در اثر خوشه بندی که در عمل ۱ انجام دادیم به ازای هر خوشه centroid هایشان داریم. در هر خوشه این کار را انجام می دهیم: representetive اول را به گونه ای انتخاب می کنیم که از centroid بیشترین فاصله

Scanned with CamScanner

را داشته باشد. دومی باید بیشترین فاصله را از centroid داری داشته باشد...

8 در آخر همه را به اندازه حدوداً ۲۰ درصد شیفت می دهیم به سمت centroid تر دیک می کنیم

9

10 Representetive

pass 2

در این مرحله کل دیتاست را scan می کنیم و هر نقطه را به نزدیکترین کلاستر assign می کنیم. در عمل

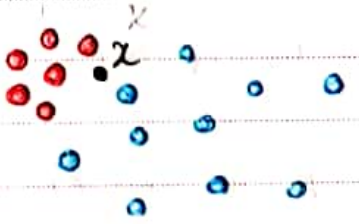
11 حول هر Representetive کره ای شکل می گیرد و خوشه ها شکل نامنظمی خواهند داشت

12 عامل دومی که باعث می شود این الگوریتم بتواند کلاسترهای پیچیده تولید کند این است که روی sample از

13 یک الگوریتم پیچیده مثلا Hierachical استفاده می کند. نقاطی که انتخاب می شوند وضعیت پیچیده ای پیدا می کنند

14 در نتیجه کلاسترینگ های هم شکل پیچیده ای خواهد داشت.

15 سوال: چرا Representetive ها را ۲۰ درصد به سمت centroid تر دیک می کنیم؟



16 در شکل روبه رو نقطه x بیشتر به خوشه قرمز احتمال دارد assign شود

17 یا آبی؟ خوشه قرمز چون همسایگی نقاط قرمز در همسایگی نقطه x بیشتر است یعنی اگر یک دایره رسم کنیم حول نقطه x، تعداد نقاط قرمز بیشتری در این دایره قرار می گیرد.

18 پس منطقی است که نقاط قرمز به عنوان کلاستر اصلی این الگوریتم انتخاب شود. وقتی Representetive کلاستر آبی را به سمت centroid آن شیفت می دهیم فاصله ای که به سمت داخل حرکت می کند بیشتر از فاصله ای است که Representetive کلاستر قرمز به سمت داخل حرکت می کند در نتیجه فاصله نقطه x از Representetive کلاستر آبی بیشتر از قرمز می شود و x به درستی در خوشه قرمز جای می گیرد. به همین دلیل است که شیفت دادن منطقی است.

Scanned with CamScanner

پیچیدگی زمانی الگوریتم: $O(NdKl)$ — l تعداد representative های هر کلاستر

N تعداد داده‌ها — K تعداد کلاسترها

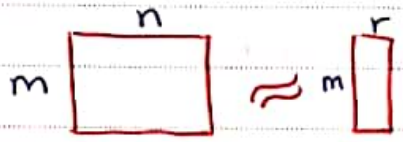
d تعداد ابعاد

جلسه بیست و دوم — ۱۴۰۰/۳/۲

"کاهش بُعد" Dimensionality Reduction

حالت استاندارد دیتا یک ماتریس $m \times n$ است. می‌خواهیم تعداد ستون‌ها را کم کنیم.

یعنی می‌خواهیم ماتریس $m \times n$ را با یک ماتریس با ستون‌های کمتر $(m \times r)$ تقریب بزنیم



در مثال زیر برای هر دیتا پوینت سه تا مولفه اول و ۲ تا مولفه آخر با هم برابرند. می‌توانیم دیتا را از فضای $\mathbb{R}^5$ به فضای $\mathbb{R}^2$ کاهش بُعد بدهیم.

دیتای ما در واقع در اینجا ۲ بُعدی است.

۲۶ جمعه ۵ ذی الحجه ۱۴۳۹ — 17 Aug 2018

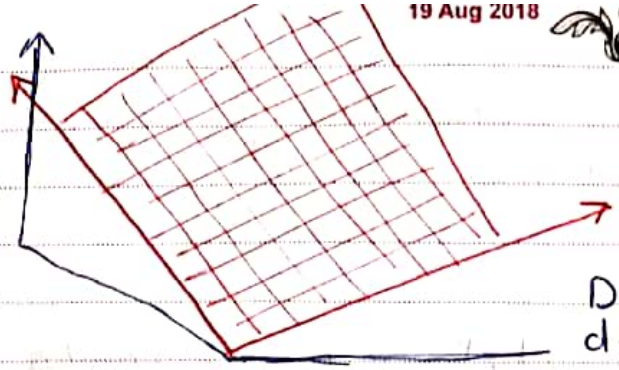
	Features					
D_1	1	1	1	0	0	[1 0]
D_2	2	2	2	0	0	[2 0]
D_3	1	1	1	0	0	[1 0]
D_4	5	5	5	0	0	[5 0]
D_5	0	0	0	2	2	[0 2]
D_6	0	0	0	3	3	[0 3]
D_7	0	0	0	1	1	[0 1]

۲ اتفاق می‌تواند در کاهش بعد رخ بدهد:

① تقریب بزنیم به فضای واقعی data point ها را پیدا می‌کنیم. یک پایه (دستگاه مختصات)

Scanned with CamScanner

19 Aug 2018 — 18 Aug 2018



برای آن پیدا می‌کنیم و مختصات نقاط را

برحسب آن پایه‌ها بیان می‌کنند.

مثال مبتنی از این نوع بود.

② تقریب داریم ← یک dimension یا direction پیدا می‌کنیم

و سعی می‌کنیم دیتا را تقریب بزنیم. در شکل دو بعد رو مختصات نقاط را در

یک کور ۱ بُعدی (دستگاه مختصات ۱ بُعدی) بیان کرده‌ایم.

الگوریتم برای هر دو حالت یکسان است فقط در حالت اول خطا نداریم در حالت دوم کمی خطا داریم.

ما می‌خواهیم latent factor های دیتا را استخراج کنیم. direction یا dimension ای را انتخاب می‌کنیم که

در آن دیتای ما بیشترین واریانس را دارد. این direction می‌شود direction اصلی ما. بعدی هم بین همه‌ی

dimension هایی که برادی عمودند این که بیشترین واریانس را دیتا در آن دارد انتخاب می‌کنیم. سومی باید

عمود برادی دومی و بین همه‌ی آن‌هایی که برادی و دومی عمودند این که واریانس بیشتری دارد انتخاب می‌شود و...

نکته: در فضای n بُعدی متناظر با هر نقطه یک بردار داریم (از مبدا به آن نقطه)

نکته: عمود بودن دو بردار به این معنی است که ضرب داخلی‌شان صفر باشد.

فضای ردیف یک ماتریس: تمام ترکیب‌های خطی بردارهای متناظر با سطر یعنی هر ردیف یک ماتریس

را یک بردار در نظر بگیریم. تمام ترکیب‌های خطی آن‌ها به ما فضای ردیف ماتریس را می‌دهد که یک زیرفضا از

Scanned with CamScanner

$\mathbb{R}^n$ است.

مثال:

$$\begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix} \begin{matrix} A \\ B \\ C \end{matrix}$$

پایه‌های قدیمی: [1 0 0] [0 1 0] [0 0 1]

سطر اول و دوم مستقل خطی اند اما سطر سوم از سطر اول و دوم مستقل نیست چون سطر اول منهای سطر دوم می‌شود سطر سوم. وقتی پایه جدیدی میسازیم باید پایه‌ها از هم مستقل باشند.

۲ تا محور پایه جدید ما را تشکیل می‌دهند: [1 2 1] [-2 -3 1]

تفکیک یعنی اینکه هر بردار را چگونه می‌توانید به صورت ترکیب خطی محورها بنویسید. ضرایب ترکیب خطی تفکیک را به ما می‌دهد. پس تفکیک A برابر [1 2 1] = به صورت [1 0] و تفکیک B برابر [0 1] و تفکیک C برابر [1 -1] خواهد شد.

ما دیدیم dimension واقعی فضای ردیفی ماتریس را پیدا کرده‌ایم. به آن Rank (رتبه) ماتریس می‌گویند

بنابراین dimension Reduction یعنی پیدا کنیم Rank (درجه / رتبه) فضای ردیفی را پیدا کنیم

یک قضیه اساسی در جبر خطی بیان می‌کند که رنگ فضای ردیفی و ستونی ماتریس ها هم برابر است.

Singular Value Decomposition (SVD)

Decomposition یعنی یک ماتریس را به صورت حاصل ضرب چند ماتریس بنویسیم

$$A_{m \times n} \sim U_{m \times r} \times \boxed{\Sigma}^{r} \times \boxed{V^T}^{n}_{r}$$

r را خودمان می‌توانیم انتخاب کنیم

Scanned with CamScanner

اگر خواهید هیچ خطایی نداشته باشد r می شود rank . (فضای ردیف dimension)

$A \approx U\Sigma V^T$ در این صورت approximation به سادگی تبدیل می شود

و می توانید r را کوچکتر از rank انتخاب کنید که در این صورت approximation داریم

U ← left singular vectors ← ماتریس mxr ← هر ردیفش متناظر با مختصات هر کدام از n

دیتا پوینت ها در فضای جدید است. ماتریس orthonormal است ← ستونها بر هم عمودند و طولشان ۱ است

Σ (Singular Values) ← یک ماتریس قطری است rxr است. عناصر روی

قطر اصلی آن مثبت اند و کوچیک می شوند از بالا به پایین. عناصر روی قطر با $\sigma_1\ \sigma_2 \dots \sigma_r$ نشان داده می شوند

خانه اول آن دارای بیشترین واریانس در بعد اول، خانه دوم دارای بیشترین واریانس در بعد دوم و ... را نشان می دهد.

V ← right Singular Vectors ← یک ماتریس nxr ← orthonormal است

V^T ← rxn ← هر ردیف این ماتریس یکی از محورهای مختصات اصلی را نشان می دهد

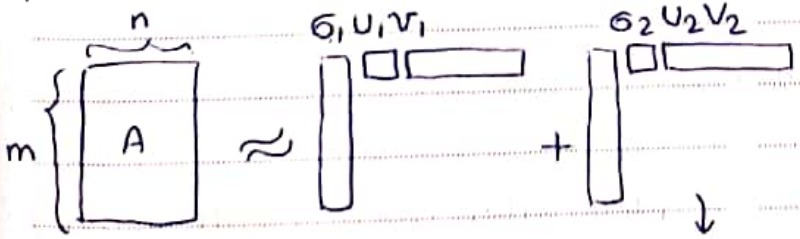
(hidden) مختصات

چون ماتریس Σ قطری است برای به دست آوردن

۲ جمعه ۱۲ ذی الحجه ۱۴۳۹ — 24 Aug 2018

حاصل ضرب این سه تا ماتریس می توانیم ستون های اول، سطرهای دوم و خانه های ماتریس قطری Σ را جداجدا در نظر بگیریم. یعنی فقط ستون اول ماتریس U را در خانه اول ماتریس Σ ضرب در سطر اول ماتریس V^T ضرب کنیم

در نتیجه یک ماتریس mxn به ما می دهد.



بیشترین سهم را در ساخت ماتریس A دارد

دومین بیشترین سهم را در ساخت ماتریس اصلی دارد

Scanned with CamScanner

ویژگی های SVD :

① برای هر ماتریس دلخواه قابل انجام است

② منحصربه‌فرد است ← Σ و U و V منحصربه‌فرد نیستند

③ ویژگی ماتریس های orthonormal :

U, V ← column orthonormal

$U^T U = I \quad V^T V = I$

④ Σ قطری است و نزولی که :

$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$

ردیف اول ماتریس V^T نشان دهنده direction اول است . برای نشان دادن هر direction ، تا عدد لازم است

ماتریس $U\Sigma$ ضعف‌ات تک‌تک مقاط را در فضای ضعف‌ات جدید نشان می‌دهد و ماتریس V^T ضعف‌ات latent factor ها را در سطح ضعف‌ات قدیمی نشان می‌دهد

تعداد سطرها و ستون‌های Σ تعداد dimension های اصلی (r) را نشان می‌دهد

تعداد واریانس دیتا در هر direction

$$\Sigma = \begin{bmatrix} 12.4 & 0 & 0 \\ 0 & 9.5 & 0 \\ 0 & 0 & 1.3 \end{bmatrix}$$

مثال از SVD در اسلایدها :

V^T ← Concept به movie را بیان می‌کند

U ← user به concept را بیان می‌کند

جلسه بیست و سوم — ۱۴۰۰/۳/۴

هر ردیف V^T ، یک direction اصلی (یک latent) را به ما نشان می‌دهد

هر کدام از σ_1 ، σ_2 و .... σ_r واریانس دیتا را در آن direction مشخص می‌کنند و در واقع شدت و قدرت آن dimension را مشخص می‌کند .

σ_i ها در واقع مقدار یکه (Singular value) ماتریس A هستند . مقادیر ویژه‌ی ماتریس $A^T A$ (ماتریس کوواریانس)

توان دو مقادیر یکه ماتریس A هستند .

ماتریس $U\Sigma$ ضعف‌ات دیتا را در فضای latent factor ها نشان می‌دهد (در واقع بردار بقویر عکور بردار بقویر عکوری) دیتا روی این ابعاد جدید را نشان می‌دهد) . ستون اول $U\Sigma$ ضعف‌ات نقاط را در dimension اول نشان می‌دهد و ...

تا این جا ما تشابه‌ی داریم و تقریبی زده‌ایم ، ابعاد واقعی دیتا را به دست آورده‌ایم . برای اینکه ما هیش بعد بیشتری

Scanned with CamScanner

داشته باشیم ابعاد متناظر با 0 های کوچک را حذف می‌کنیم. حذف آخرین عدد ماتریس Σ متناظر با حذف ستون آخر U و حذف سطر آخر V^T است.

تعداد Singular Value های غیرصفر ماتریس برابر با Rank ماتریس است (تعداد dimension های پنهان)

می‌توانیم ماتریس‌مان را با SVD تقریب بزنیم. تعداد کاهش یافته، از ضرب $U\Sigma V^T$ با یک تخمین و تقریب از ماتریس اصلی (A) به ما می‌دهد. مثلا اگر رنک ماتریس A، ۳ باشد و رنک ماتریس تخمین (A') برابر ۲ باشد و نزدیک‌ترین حالت A' این است که از بین تمام ماتریس‌های با رنک ۲ نزدیک‌ترین ماتریس به A است. نزدیکی را با Reconstruction Error می‌سنجیم که با نرم Frobenius حساب می‌شود:

$$\|M\|_F = \sqrt{\sum_{ij} M_{ij}^2} \qquad \|A-B\|_F = \sqrt{\sum_{ij} (A_{ij}-B_{ij})^2}$$

به این کار low Rank approximation می‌گویند. به دنبال تقریبی از ماتریس‌مان با رنک پایین‌تر هستیم که SVD یک جواب کلیه به ما می‌دهد. بهترین تقریب رتبه پایین را به ما می‌دهد.

energy ← حاصل جمع توان دو Singular Value ها را انرژی می‌گویند.

می‌خواهیم بیایم دعوا حل چند تا از این Singular value ها را نگه داریم و بقیه را دور بیندازیم. می‌گوییم باید مقدار انرژی SV های باقی مانده ۹۰٪ انرژی SV های اولیه باشد.

بردارهای ویژه و مقدارهای ویژه

یک ماتریس مربعی به نام M داریم. x را eigen vector می‌گوییم اگر و تنها اگر:

① x غیر صفر باشد

② عدد λ (حقیقی) وجود داشته باشد به طوری که: $Mx = \lambda x$

λ مقدار ویژه متناظر با بردار ویژه x می‌گویند.

principal eigenvalue: بین تمامی مقادیر ویژه ماتریس، مقدار ویژه‌ای که قدر مطلقش از بقیه بزرگتر است

و به بردار ویژه متناظر با آن principal eigenvector گفته می‌شود.

ماتریس M متقارن است اگر و تنها اگر $m_{ij} = m_{ji}$ برای تمام اندیس‌ها

روش پیدا کردن principal eigenvector:

① از یک بردار دلخواه اولیه استفاده می‌کنیم: x_0 (ترجیحا با نرم ۱)

② دنباله‌های مختلف x_i را آپدیت می‌کنیم.

جمعه
۱۹ ذی الحجه ۱۴۳۹
31 Aug 2018

$$x_{k+1} = \frac{Mx_k}{\|Mx_k\|} \quad \text{for } k = 0, 1, \dots$$

تعداد تکرار چند بار مرحله 2، هنگامی اتفاق می‌افتد که x تغییر چندانی ندارد

* چگونه از روی بردار ویژه (eigenvector) مقدار ویژه متناظرش را پیدا کنیم؟

$$x\lambda = Mx \rightarrow x^T x\lambda = x^T Mx \xrightarrow{x^T x = 1} \boxed{\lambda = x^T Mx}$$

$x^T x = 1$ چون نرم x برابر ۱ است هنگتی در الگوریتم آن را حساب می‌کردیم، نرمال می‌کردیم

تا این جا هر چه گفتیم برای ماتریس‌های مربعی صدق می‌کند. از این به بعد در مورد ماتریس‌های مربعی و متقارن صحبت می‌کنیم.

Scanned with CamScanner

$$M^* := M - \lambda x x^T$$

به طور مرتب eigenvalue و eigenvector حساب می‌کنیم، اثر آن را از روی ماتریس اصلی حذف می‌کنیم، ماتریس M^* بدست می‌آید. دوباره این فرآیند را روی M^* حساب می‌کنیم تا eigenvalue و eigenvector دوم برای ماتریس اصلی بدست بیاید.

eigenvector هایی که به این ترتیب به دست می‌آیند برهم عمودند

اگر تعداد iteration ها را ثابت در نظر بگیریم پیدا کردن یک بردار ویژه و مقدار ویژه $O(n^2)$ زمان می‌برد

$$A = U\Sigma V^T \Rightarrow A^T = (U\Sigma V^T)^T = (V^T)^T \Sigma^T U^T = V\Sigma U^T$$

$$A^TA = V\Sigma U^T U\Sigma V^T = V\Sigma^2 V^T$$

$U^TU = I$ ← چون ماتریس U یک ماتریس orthonormal است

Σ^2 ← همان ماتریس Σ است که عناصر روی قطر به توان 2 رسیده اند

$$A^TAV = V\Sigma^2 V^T V = V\Sigma^2$$

A^TA اصطلاحاً ماتریس کوواریانس نامیده می‌شود.

ماتریس A^TA مربعی و متقارن است.

$$\underbrace{(A^TA)}_{M} V = V\Sigma^2$$

ستون اول ↓ ستون r ام ↓

$$[MV_1\ MV_2\ MV_3 \dots MV_r] = [\sigma_1^2 V_1\ \sigma_2^2 V_2 \dots \sigma_r^2 V_r]$$

Scanned with CamScanner

$\Rightarrow MV_1 = \sigma_1^2 V_1$ $\Rightarrow$ V_1 یک بردار ویژه و σ_1^2 یک مقدار ویژه برای M است

$\Rightarrow MV_2 = \sigma_2^2 V_2$ " " " " " σ_2^2 " " " V_2

چون σ_i ها نزولی هستند V_1 بردار ویژه principal است چون بزرگترین مقدار را دارد

ماتریس A^TA را در نظر می‌گیریم. eigenVector ها و eigenvalue های آن را حساب می‌کنیم

بنابراین ماتریس V و Σ بدست می‌آید.

نکته: مقادیر ویژه‌ی ماتریس‌های متقارن همیشه غیرمنفی است پس می‌توان از آن جذر گرفت. ماتریس A^TA متقارن است پس نتیجه $\sigma_1^2 = \lambda_1$ و $\sigma_1 = \sqrt{\lambda_1}$ همیشه تعریف شده است

ستون‌های ماتریس V، eigenVector عمود برهم نرمال شده‌ی A^TA هستند.

ستون‌های ماتریس U، eigenVector های عمود برهم نرمال شده‌ی AA^T هستند پس ماتریس U را هم به همین شکل می‌یابیم

پیچیدگی محاسباتی:

$O(mn^2)$ یا $O(nm^2)$ ← هرکدام کمتر باشد

برای ماتریس‌های sparse مثل Max خیلی سریعتر می‌توان این کار را انجام داد (در O زیاد خانه‌های غیرصفر)

Scanned with CamScanner

15 جلسه بیست و چهارم - ۱۴۰۰/۳/۹

16

یک مثال از کاربرد SVD : یک بردار query داریم و می خواهیم data point هایی که شبیه کوئری ما هستند

17

را پیدا کنیم . کوئری ما : Find users that like 'Matrix' کاربرانی که دوست دارند Matrix

18

Matrix A B C D

$q = [5\ 0\ 0\ 0\ 0]$

را دوست دارند می خواهیم پیدا کنیم. می خواهیم بیتا پوینت های را پیدا کنیم که به این بردار شبیه اند. می توانیم برای پاسخ به کوئری ، بردار کوئری را با تک تک بیتا پوینت ها مقایسه کنیم و در بهترین حالت هزار اینکه بیتا ها را به فضای بچان منتقل کنیم (به dimension های اصلی و پنهان) . فضای هر کدام از بیتا پوینت ها را در این dimension های پنهان بدست بیاریم . کوئری را هم به فضای پنهان منتقل می کنیم ، فضاشان را در فضای پنهان به دست می آوریم ، بعد در این فضای پنهان شباهت کوئری را با تک تک بیتا پوینت ها می سنجیم . چگونه می توانیم فضای بردار کوئری را در فضای latent به دست آوریم ؟ ما می دانیم که ردیف های ماتریس V^T ، قدم های فضای latent را به ما نشان می دهند.

Scanned with CamScanner

می‌دانیم که: $A = U\Sigma V^T$ در نتیجه، $AV = U\Sigma$، A نشان‌دهنده مختصات نقاط ما در پایه قدیم است. مختصات نقاط در پایه قدیم ضرب در ماتریس V، مختصات نقاط در پایه جدید به ما داده است. پس برای q هم باید همین کار را انجام بدهیم. ردیف اول ماتریس A در ستون اول، ردیف دوم در ماتریس V ضرب شده (ضرب داخلی) و ردیف اول $U\Sigma$ را داده که معادل مختصات ردیف اول A در پایه جدید است.

درمورد خطی یک و بزرگ در مورد پایه‌های orthonormal وجود دارد: اینکه اگر خواهید مختصات یک نقطه دلخواه q را در این پایه حساب کنید کافی است ضرب داخلی q در پایه‌ها را حساب کنید:

$$[q.v_1 \quad q.v_2 \quad q.v_3]$$

انگار که q را داریم در ماتریس V ضرب می‌کنیم.

سوال: اگر v_1، v_2، v_3 و... (ستون‌های ماتریس V) به هم عمود بودند اما اندازه‌شان ۱ نبود، در این حالت باید چه کاری می‌کردیم؟ باید بعد از ضرب داخلی، بر اندازه ستون هم تقسیم می‌کردیم. مثلا:

$$\frac{q.v_1}{v_1.v_1}$$

سوال: فرض کنید ستون‌های V از هم مستقل خطی باشند ولی متعامد نباشند چطور؟ در این حالت با اینکه ستون‌های V دستگاه مختصات هستند اما نمی‌توانیم با تغییر در فرمول مختصات q را در این دستگاه مختصات به دست آوریم.

مثال اسلاید:

$$d = [0\ 4\ 5\ 0\ 0] \dashrightarrow [5.2\ \ 0.4]$$

$$q = [5\ 0\ 0\ 0\ 0] \dashrightarrow [2.8\ \ 0.6]$$

در این حالت شباهت کسینوسی بردار q با d_1 که یکی از دیتا پوینت‌هاست صفری شود. در فضای latent شباهت آن‌ها بزرگتر از ۱ است. منطقی این است که شباهت داشته باشند چون هر دو به فیلم‌های Sci Fi امتیاز بالا داده‌اند.

Scanned with CamScanner

مزایای SVD: ① low Rank approximation را به صورت optimal انجام می‌دهد

معایب SVD: ① پیچیدگی محاسباتی بالا $O(n^3)$ ② قابل تفسیر بودن ← explainable نیست، latent component

که به خودی خود آیندهیچ ارتباطی با سطرها و ستون‌های ماتریس دیتا ندارند

اما فضای دیتاپوینت‌ها را به صورت قابل تفسیر می‌نشیند و دوها فضای هر دوهای ما (ردیف‌های V^T) قابل تفسیر هستند.

③ کمبود sparsity ← خیلی اعضای ماتریس دنیای ما بسیار بزرگ است و خیلی از خانه‌های آن خالی هستند

بسیاری از الگوریتم‌های کهنه از sparsity ماتریس استفاده می‌کنند اما SVD استفاده نمی‌کند. ماتریس‌های بزرگ ما یعنی U و V، dense هستند.

این معایب باعث شد به سراغ تجزیه‌های دیگری برویم.

CUR Decomposition ← مشکل explainability را حل می‌کند و ماتریس‌ها بزرگ ما sparse خواهند بود

ماتریس A را به صورت حاصلضرب سه تا ماتریس می‌نویسیم: $A \approx C \cdot U \cdot R$

C ← تعدادی از ستون‌های A را با جایگذاری sample می‌کنیم و در C قرار می‌دهیم (ابعاد: $m \times c$)

R ← سطرهای A را به همین شکل می‌کنیم (ابعاد: $r \times n$) نزدما $r=c$ نیست

U ← ابعاد: $c \times r$ ← ستون‌ها و ردیف‌های سمپل شده را در نظر می‌گیریم، intersection آن‌ها را در نظر می‌گیریم، ماتریس W به دست می‌آید که $r \times c$ است. برای اینکه U به دست آید از W، شبه وارون (pseudoinverse) می‌گیریم. شبه وارون W را با W^+ نشان می‌دهیم.

وارون فقط برای ماتریس‌های مربعی که دترمینان آن غیر صفر باشد تعریف می‌شود اما شبه وارون برای

هرما ترسی تعریف می‌شود.

فرض کنیم W^+ شبه وارون W باشد. اگر تجزیه SVD ماتریس W به صورت زیر باشد:

$$W = XZY^T \quad \Rightarrow \quad W^+ = YZ^+X^T$$

$Z^+ \leftarrow$ خانه‌های صفر Z را هم تغییری نمی‌دهیم، $Z^+_{ii} = 1/Z_{ii}$

ماتریس W^+ یک ماتریس $c*r$ است.

سوال: چرا intersection می‌گیریم؟ چون ردیف‌ها و ستون‌های هم را با احتمال بیشتر انتخاب می‌کنیم در نتیجه خانه‌هایی که در وصل مشترک ردیف‌ها و ستون‌های انتخاب شده هستند خانه‌های غیر صفر هستند. در نتیجه خواهیم که ماتریس اشتراک را اینجوری داشته باشیم

سوال: چرا شبه وارون برای حالت‌هایی که وارون تعریف شده به ما وارون برای بدهد؟

$$W = XZY^T \rightarrow W^{-1} = (XZY^T)^{-1} = (Y^T)^{-1} Z^{-1} X^{-1} = \boxed{YZ^+X^T}$$

وارون و شبه وارون یکسان می‌شوند

$Y^{-1} = Y^T \rightarrow$ چون Y orthonormal است $\quad Z^{-1} = \frac{1}{Z_{ii}} \quad X^{-1} = X^T \rightarrow$ چون X orthonormal است

۳۰ جمعه ۱۱ محرم ۱۴۴۰

21 Sep 2018

حگونه سطرها و ستون‌های A را سمپل کنیم؟

معمولا uniformly at random خوب کار می‌کند. یک راه حل ← هر ستون و هر سطر را متناسب با اهمیت آن انتخاب کنیم. که در این حا اهمیت را نرم ۲ (norm 2) یا نرم ۲ به توان ۲ در نظر می‌گیریم.

مثلا اهمیت $(3\ 4\ 5)$: $3^2+4^2+5^2$

← شبه کد انتخاب ستون‌ها را اسلاید ۶

در تجزیه CUR عکس جوری‌هایی که بیست می‌آیند به هم عمود نباشند

Scanned with CamScanner

k عددی است که کاربر set می‌کند و ϵ هم به مقدار کوچک بین ۰ و ۱ است که کاربر set می‌کند. k رنک ماتریس تقریبی است که می‌خواهیم به دست آوریم. یک ماتریس A داریم که می‌خواهیم الگوریتم تقریب با مرتبه k از آن داشته باشیم. ϵ هم میزان خطایی است که می‌خواهیم داشته باشیم.

① اگر $C = O\left(\frac{k \log k}{\epsilon^2}\right)$ ③ $U = W^+$

② $r = O\left(\frac{k \log k}{\epsilon^2}\right)$

اگر ۳ شرط بالا برقرار باشد با احتمال ۹۸ درصد چنین error guarantty داریم:

$$\|A - CUR\|_F \leq (2+\epsilon)\|A - A_k\|_F$$

خطای بازسازی الگوریتم CUR — خطای بازسازی SVD

A_k ← بهترین تقریب رنک k که از ماتریس A داریم.

این نامساوی نشان می‌دهد که خطای بازسازی CUR بیتر است از SVO اما خیلی هم بهتر نیست.

ماتریس‌های بزرگ ما sparse هستند. ماتریس C اسپارس است چون بخشی از دیتاست و دیتای ما اسپارس است ماتریس R هم اسپارس است چون R هم بخشی از دیتاست.

مزایای CUR ← ① خاصیت interpretablity را دارد چون dimension های ما ردیف‌های ماتریس دیتای ما هستند. به همین دلیل dimension هایی که CUR پیدا می‌کند شامل هنوز نقاط است

عیب CUR ← ممکنه ردیف‌ها و ستون‌های تکراری داشته باشیم. ممکنه فی‌نسبت ما با بایاس پیدا کنه به سمت ردیف‌ها و ستون‌هایی که نرم بالایی دارند و رعایت کند. در عمل می‌توانیم sampling را بدون جایگذاری انجام بدهیم که سطر و ستون تکراری نداشته باشیم. در این حالت به خاطر نتایج تئوری و گارانتی خطا اومدیم با جایگذاری این کار را انجام دادیم.

SVD: $A = U \Sigma V^T$

U: sparse و کوچک — Σ: sparse و بزرگ — V: dense و بزرگ

CUR: $A = C\,U\,R$

C: sparse و بزرگ — U: dense و کوچک — R: sparse و بزرگ

Scanned with CamScanner

زمان SVD مرتبه سوم است و زمان CUR مرتبه دوم ← $O(nm)$

بهترین زمانی که برای تکمیل و محاسبه روی ماتریس باید بپردازیم به ۲ تا عامل بستگی دارد:

① اندازه ماتریس ② density ← تعداد خانه‌های غیر صفر ماتریس

اگر بخواهیم ماتریسی را به صورت بهینه پردازش کنیم باید یکی از این ۲ مورد کوچک باشد ولی در SVD

برای ماتریس U و ماتریس V^T هیچ کدام از این ۲ مورد کوچک نیست و هر دو dense و بزرگ اند.

ماتریس Σ هر دو مورد را دارد، هم کوچک است هم sparse درحالی که لازم بود هر دو را داشته باشد.

اما در CUR ماتریس C بزرگ است و sparse ← می‌توانیم از الگوریتم‌هایی که برای ماتریس‌های sparse

بهینه شده استفاده کنیم.

۲۳ جمعه ۴ محرم ۱۴۴۰

14 Sep 2018

ماتریس U هم کوچک است و dense

$C \times r$

R بزرگ و sparse است ← از الگوریتم‌های ماتریس‌های sparse استفاده می‌کنیم.

Scanned with CamScanner

مثال DBLP در اسلایده

1) Accuracy = 1 − relative sum squared errors

نسبی reconstruction error

نرم F ماتریس تفاضل را تقسیم نرم F ماتریس بیتا می‌کنیم

2) space Ratio : مقدار خانه های غیر صفر ماتریس decomposition

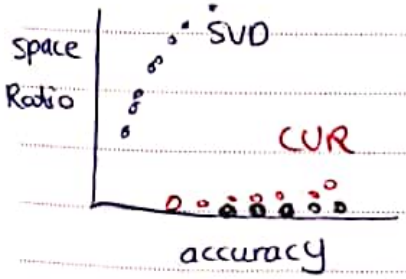
مقدار خانه های غیر صفر ماتریس وردی

هرچقدر این نسبت بالاتر باشه یعنی ماتریس های حاصل از تجزیه denseتر هستند

دقت های بیشتری نیاز داریم.

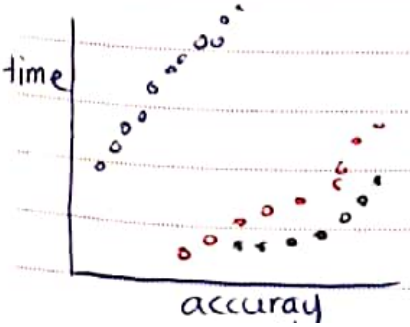
3) Cpu time

در SVD هرچقدر اجازه بدهیم Rank ماتریس تجزیه بالاتر باشد accuracy بیشتری داریم - در عوض از رتبه در SVD را با افزایش Rank انجام داده ایم . accuracy بیشتر شده و space Ratio هم بیشتر شده . space ای که نیاز داریم خیلی زیاد خواهد بود



ولی در CUR اینگونه نیست نقاط مشکل CUR بدون حاشیه گذاری است یعنی اجازه ندهیم ستون ها و سطرهای تکراری انتخاب شوند

space Ratio خیلی کنترلده رشد میکنه با افزایش accuracy . چون ماتریس های خروجی ما sparse هستند حالت بدون حاشیه گذاری CUR کمتر کاربرد دارد



در SVD با افزایش Rank ماتریس تجزیه شده، وقت زیادی میبره و زمان اجرا هم بیشتری میشود خیلی بسریع

ولی در CUR با اجازه دادن اینکه مقدار سطر و ستون های بیشتری داشته باشیم accuracy بالا می‌رود ولی زمان اجرا با شیب کمتری افزایش می‌یابد

Scanned with CamScanner

جلسه بیست و پنجم - ۱۴۰۰/۳/۱۱

سیستم‌های توصیه‌گر (Recommendor systems)

مثلا شما در آمازون به سری آیتم‌ها را جست و جو می‌کنید یا لایک می‌کنید. بعد آمازون به سری اقلام دیگر را به شما پیشنهاد می‌دهد.

یا مثلا در سایت‌های موزیک. شما به سری موزیک‌های خاص را گوش می‌کنید بر حسب موزیک‌هایی که گوش کردید سامانه میاد به سری موزیک‌های دیگه به شما پیشنهاد میده. یا مثلا به سری کلیپ‌هایی را می‌بینید و کلیپ‌های دیگری که به شما پیشنهاد داده می‌شود. یوتوب گوگل نیوز بر حسب چیزهایی که انتخاب کرده‌اید و خوانده‌اید به سری چیزهای دیگه به شما نشون داده می‌شه.

اینکه این Recommendation ها چگونه ایجاد می‌شوند موضوع این جلسه از درس است.

Scanned with CamScanner

سه نوع Recommendation داریم:

① یه سری توصیه‌های گلچین شده که همشون دارن می‌شن صرف‌نظر از رفتاری که کاربر از خودش نشون داده

② سایت‌های فروش میان یه سری aggregation های ساده ای انجام میدهند. مثلا ۱۰ تا آیتمی که بیشتر خریداری شده به همه‌ی userها پیشنهاد می‌دهند یا مثلا ۱۰ تا آیتمی که جدیدتر اضافه شده‌اند.

③ سیستم های بزرگ با توجه به سابقه کاربر، پیشنهادهای مخصوص هر کاربر را به او نشان می‌دهند.

Formal Model

X ← مجموعه مشتری ها (کاربرها)

S ← مجموعه آیتم ها (اقلام، فیلم، موزیک ها و...)

می‌خواهیم یک utility function را یاد بگیریم: $u: X \times S \rightarrow R$ (مجموعه نمره‌ها)

این تابع یک زوج مرتب که از یک user و یک آیتم تشکیل شده را ورودی می‌گیرد و یک امتیاز (نمره) را تخمین می‌زند. آن آیتمی که بیشترین امتیاز را بگیرد به کاربر پیشنهاد می‌دهیم. utility Matrix ← سطر: user ها ستون: آیتم‌ها

کاربرها در مورد یه سری از آیتم‌ها امتیاز داده‌اند. utility function با توجه به ویژگی‌های کاربر و آیتم‌ها امتیازی را برای آیتم‌هایی که نمره‌اش را نمی‌دانیم تخمین می‌زند.

۲ تا مرحله اصلی برای پیدا کردن امتیازها:

① score های اولیه را چگونه پیدا کنیم؟

جمعه ۶
۱۸ محرم ۱۴۴۰
28 Sep 2018

Scanned with CamScanner

② چگونه از روی score هایی که داریم، score هایی که نمی دانیم را حدس بزنیم؟

ما علاقه مند هستیم جاهایی که score بالا دارند را حدس بزنیم چون آن ها را می خواهیم پیشنهاد دهیم

③ چگونه ارزیابی کنیم سیستم ما خوب کاری می کند یا نه؟

1 نحوه ی جمع آوری score های اولیه

Explicit ← از کاربرها بخواهیم که امتیاز بدهند (در عمل خوب کار نمی کند چون کاربر علاقه ای ندارد)

② crowdsourcing ← به کاربرها پول بدهیم که فیلم ببینند

Implicit ← ① از روی action های کاربر درمیاوریم، مثلا کلیپ را تا آخر ببیند نمره بالا بدهیم و اگر فقط اول کلیپ را دید نمره پایین بدهیم

2

از مشخصات کاربرها، آیتم ها... استفاده کنیم و Utility Function را یاد بگیریم

چالش ← ماتریس U، اسپارس است، دنیای آموزشی کمی داریم

cold start داریم ← به کاربر یا به آیتم تازه وارد می شود سطر و ستون متناظر آن در U خالی است

و این بعضی از الگوریتم ها را دچار مشکل می کند

به طور کلی ۳ دسته الگوریتم Recommendation داریم:

① Content_based ← از ویژگی ها، محتواها یا content کاربرها و آیتم ها استفاده می کنند برای توصیه کردن

Scanned with CamScanner

② Collaborative (تعاملی) ← از تصمیمات کاربرهای دیگه برای توصیه کاربر خاص به آیتم خاص استفاده می‌کنند

③ Latent Factor based ← شباهت داده‌ها را کاهش میدهد. سعی می‌کند Factor های پنهان موجود در دیتا را استخراج کند و از آن برای توصیه استفاده کند.

۱ روش های مبتنی بر محتوا

ایده اصلی ← هر آیتم یک محتوا دارد (Feature Vector)

شما می‌خواهید به User X پیشنهاد آیتم بدهیم. این User از قبل به سری action داشته. با آیتم‌هایی

رابطه کاربر X پیشنهاد می‌دهید که کاربر X به آیتم‌هایی مشابه آن، قبلا نظر مثبت نشان داده است. مثلا به

کاربری که می‌خواهد فیلم ببیند، فیلم‌هایی را پیشنهاد می‌دهید که شبیه فیلم‌هایی باشد که قبلا به آن ۴ امتیاز بالا داده.

هر کاربر یک پروفایل دارد که براساس action هایش آن را آپدیت می‌کنیم. معمولا محتوا را با Feature Vector مدل می‌کنیم

چگونه Feature های مهم را انتخاب کنیم؟

مثلا برای Text از TF_IDF استفاده می‌کنیم.

$$IDF_i = \log \frac{N}{n_i}$$

f_{ij} ← تعداد دفعاتی که کلمه i در داکیومنت j رخ می‌دهد N ← تعداد کل داکیومنت‌ها

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{ij}} \leftarrow f_{ij} \text{ نرمال شده ی}$$

n_i ← تعداد داکیومنت‌هایی که کلمه i در آن اتفاق می‌افتد. هرچقدر تعداد داکیومنت‌هایی که ترم i در آن

اتفاق می‌افتد بیشتر باشد اهمیت ترم i برای داکیومنت j کم می‌شود.

Scanned with CamScanner

$$IDF_i = \log \frac{N}{n_i}$$

$$\text{TF_IDF score} : w_{ij} = TF_{ij} \times IDF_i$$

در نهایت کلمه‌هایی که TF_IDF بالایی دارند را به همراه خود این وزن‌ها به عنوان Feature Vector داکیومنت انتخاب می‌کنیم

فرض کنید user profile را ساخته‌ایم. حالا چگونه پیشنهاد را بدهیم؟

الان یک Feature Vector برای user و یک Feature Vector برای آیتم داریم. هرچقدر شباهت این دو بردار بیشتر باشد، احتمال بیشتری تعلق می‌گیرد. مثلا از شباهت کسینوسی می‌توان استفاده کرد:

user profile ← x
item profile ← i

$$u(x,i) = \cos(x,i) = \frac{x \cdot i}{\|x\| \cdot \|i\|}$$

چون تعداد user ها و آیتم‌ها زیاد است

و نمی‌توان برای تک تک آن‌ها شباهت را

۱۳ جمعه ۲۵ محرم ۱۴۴۰
5 Oct 2018

حساب کنیم به عنوان پیش پردازش از LSH

استفاده می‌کنیم و Recommandation هایی که برای user خواهیم داشت فقط از بالکت مربوطه

خودش خواهد بود. (user های مشابه را داخل یک بالکت قرار می‌دهیم، item های مشابه را هم)

مزایای روش های content_based :

① ارزیابی utility استفاده‌ای نمی‌کند بنابراین در گیر چالش‌های مربوط به ساختن و اسپارس بودن این ماتریس نمی‌شود

Scanned with CamScanner

۲) اینکه اگر کاربری سلیقه خیلی خاصی هم داشته باشد باز هم می‌تواند پیشنهادهای خاص خودش را به او بدهد چون برای هر کاربر پروفایل داریم

۳) می‌تواند آیتم‌های به ندرت استفاده شده و غیر محبوب را پیشنهاد بدهد چون فقط به تشابهات کاربر و آیتم توجه دارد. با نظرات کاربران در مورد آیتم‌ها کاری ندارد. یه آیتم ممکنه غیر محبوب باشه اما تشابهش با کاربر زیاد باشه او نو توصیه می‌کنه.

۴) explainable است ← سیستم می‌تواند توضیح بدهد که چرا این آیتم را به این کاربر پیشنهاد داده است

۵) first-rator problem ندارد، اینکه یه آیتم اول کار باید یه سری score داشته باشه تا در فرآیند توصیه شروع بکنه، این الگوریتم این مشکل را ندارد.

مزایای روش‌های content_based

۱ خیلی دقت‌ها بیاد پایین بردارهای ویژگی سخت است. مثلا شما یه سری عکس دارید و کاربر به آن‌ها امتیازی دهد. با یک سری روش‌هایی توان بردارهای ویژگی برای تصویر استخراج کرد اما explainability نقض می‌شود

۲ کاربر دقیق پروفایلش ساخته می‌شه که یک action ای داشته باشد. برای کاربری که تازه وارد شده نمی‌توان آیتمی را پیشنهاد داد چون هنوز پروفایلی براش ساخته نشده.

۳ خیلی specialized عمل می‌کند. به صورت تکراری، کاربر به سری عمل با توجه به علاقه‌مندی‌ها و سلیقه کنونی‌اش ایراد می‌دهد و بر اساس آن به پروفایلی ساخته می‌شود و توصیه‌ها دقیق‌تر می‌شود و کاربر عکس العمل‌هایش را می‌بیند و...

به کاربر این فرصت داده نمی‌شود تا آیتم‌های متفاوت در اختیارش قرار بگیرد تا interest های خود را نشان بدهد

همچنین تفاوت کاربرهای دیگر در اختیار کاربر قرار نمی‌گیرد. ممکنه یه آیتم شباهت زیادی به کاربر داشته باشد اما تفاوت‌ها در مورد آن بد باشد در این صورت نباید چنین آیتمی را پیشنهاد بدهیم. مثلا ممکنه Fake News باشه یا فیلم بی ارزش باشد.

Scanned with CamScanner

روش‌های content-based در دوسه‌سال اخیر دوباره بدلیل دو دلیل، مورد توجه قرار گرفته‌اند. چون

① روش‌های Representation learning پیشرفت زیادی داشته‌اند ← Feature Vector های خوبی تولید شده

② روش یافتن شباهت دو تا آیتم هم پیشرفت‌های خیلی خوبی داشته است ← به صورت یک مسئله رگرسیون ول‌ی می‌شود و شباهت بردارها یاد گرفته می‌شود.

ول‌ی مشکلی دارند که explainable نیستند.

2 روش‌های Collaborative Filtering

اساس این روش: یک user x و یک آیتم i داریم که خانه مربوط به آن خالی است، می‌خواهیم حدس بزنیم

گروهی به نام N که شامل k تا user است که این user ها به آیتم i نمره داده باشند و اینکه از نظر بردار نمره شبیه‌ترین باشند به user X، انتخاب می‌کنیم. سپس نمرات این k تا user به آیتم i را با هم aggregate (تجمیع) می‌کنیم و این عددی می‌شود تخمین ما از نمره X به آیتم i. علت اینکه کار می‌کند، چون userهایی که از نظر بردار نمره شبیه user ما هستند یعنی برای آیتم‌های x که مثل u user عمل کرده‌اند پس برای آیتم خاص i هم مثل user x عمل خواهند کرد.

معیارهای شباهت:

① Jacard Similarity

$$Sim(x,y) = \frac{|r_x \cap r_y|}{|r_x \cup r_y|} = \frac{2}{5}$$

$r_x = [1, -, -, 1, 3]$ $\quad r_x = \{1, 4, 5\}$

$r_y = [1, -, 2, 2, -]$ $\quad r_y = \{1, 3, 4\}$

مشکل: دو مقادیر score ها را در نظر نمی‌گیرد در حالی که خیلی مهم است.

② Cosine Similarity → در این جا خانه‌های که امتیاز ندارد را 0 در نظر می‌گیریم

$$Sim(X,Y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{\|r_x\| . \|r_y\|}$$

$r_x = \{1, 0, 0, 1, 3\}$

$r_y = \{1, 0, 2, 2, -\}$

مشکل این روش: کم‌ترین تاثیر را جا‌هایی می‌گذارد که ما یک missing score داشته باشیم اما منفی

Scanned with CamScanner

11 Oct 2018

سیت. چون ممکنه کاربرهای مادر مورد یک آیتم missing score داشته باشند اما واقعا اگر به آن آیتم امتیاز بدهند، شبیه باشند. وقتی باید امتیاز پایین داشته باشیم که کاربر امتیاز پایین و دیگری امتیاز بالا بدهد یا cos similarity اینگونه عمل می‌کند. مثلا شباهت او را با کمتر از او در نظر می‌گیرد. بنابراین باید دارایم چیزه داشته باشیم برای عدم شباهت.

③ pearson correlation coefficient → cos تا تغییر اعمال می‌کند روی شباهت

یک زیربردار از دو بردار در نظر می‌گیریم ← S_{xy} ← مجموعه آیتم‌هایی که هم x و هم y برای آن‌ها نظر داده

(فقط آیتم‌های مشترک را در نظر می‌گیریم) ← تغییر اول (بقیه آیتم‌ها را در نظر نمی‌گیریم)

شباهت r_x و r_y با فقط در آیتم‌های S_{xy} می‌سنجیم

تغییر دوم ← یک نرمال سازی

۲۰ جمعه ۲ صفر ۱۴۴۰

12 Oct 2018

هر خانه را معنای میانگین عزات آن برداری کنیم

← میانگین کل نه فقط میانگین آیتم‌های S_{xy}

(ممکنه بعضی از خانه‌ها منفی بشوند)

$$sim(x,y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}}$$

هر زمان یک خانه خاص یک آیتم‌s خاص کمترین دارد Contribution را در معیار شباهت دارد وقتی که در ترم‌های صورت کسر یکی از آن‌ها منفی و دیگری مثبت باشد

مثلا r_{xs} از میانگین نره‌های x خیلی پایین‌تر باشد و r_{ys} از میانگین نره‌های y خیلی بالاتر باشد و خیلی متفاوت عمل کرده اند برای آیتم‌s.

وقتی که هر دو از میانگین پایین‌تر یا بالاتر نره بدهند تاثیر مثبت روی نره شباهت دارد و درست هم هست.

Scanned with CamScanner

جلسه بیست و ششم - ۱۴۰۰/۳/۱۸

مثالی از معیار pearson در اسلایدها

فرض کنیم که ما معیار شباهت را اعمال کرده‌ایم و همسایگی را پیدا کرده‌ایم. مانیه خانه خاص (x, i) را در نظر داریم. (خانه‌ی مربوط به user x و item i). مجموعه‌ی N از همسایه‌های ردیف x ام که k تا عضو دارد پیدا می‌کنیم به طوری که این همسایه‌ها ۲ شرط داشته باشند. اول اینکه خانه i ام‌شان پر باشد، دوما بین آن‌هایی که خانه i ام‌شان پر است تردیدترین و شبیه‌ترین هستند به بردار rating x. بعد از روی این خانه‌هایی که پیدا کردیم، score خانه (x, i) به این صورت محاسبه می‌شود:

میانگین امتیازهای این k تا همسایه ←

$$r_{xi} = \frac{1}{K} \sum_{y \in N} r_{yi}$$

اما بهتر است که از میانگین وزن‌دار استفاده کنیم:

$$r_{xi} = \frac{\sum_{y \in N} S_{xy} \cdot r_{yi}}{\sum_{y \in N} S_{xy}}$$

← iterate می‌کنیم روی همسایه‌های x و به هرکدام وزنی برابر با شباهت y و x می‌دهیم. (y همسایه x است)

سوال: تا الان از User_User collaborative filtering استفاده می‌کردیم، آیا می‌توانیم از item_item استفاده کنیم؟ فرض کنید می‌خواهیم مقدار خانه (x, i) را تخمین بزنیم. به جای اینکه همسایه‌های x را پیدا کنیم بیاییم همسایه‌های i را پیدا کنیم. بله قابل انجام است. بیاییم برای item i، k تا همسایه پیدا می‌کنیم. این همسایه‌ها باید دو تا شرط داشته باشند: خانه x ام‌شان نمره داشته باشه، دوما از بین همه‌ی آن‌هایی که خانه x ام‌شان نمره دارد این‌ها شبیه‌ترین باشند به آیتم i. بیاییم این k تا همسایه را در مجموعه N قرار می‌دهیم

Scanned with CamScanner

در اینجا عزی خانه‌ی x_i را به این صورت به دست می آوریم:

$$r_{xi} = \frac{\sum_{j \in N(i;x)} S_{ij} \cdot r_{xj}}{\sum_{j \in N(i;x)} S_{ij}}$$

میانگین وزن دار روی همسایه‌های آیتم i. وزن هر همسایه را برابر با شباهت آن با آیتم در نظر می گیریم.

مثال را اسلاید

مدل عامل خودمان را به Collaborative Filtering local (محلی) است

$$r_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} S_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} S_{ij}}$$

یک baseline estimator پیدا می کنیم برای نمره من که براساس نمرات کلی و کلی موجود در utility Matrix کار می کند و به همسایگی خاص user ها و آیتم ها کاری ندارد. یک تخمین سطح بالا است از خانه (x,i)

baseline estimate for r_{xi} : $\boxed{b_{xi} = \mu + b_x + b_i}$

بعد میایم baseline رو با هم جمع می کنیم و تخمین خاص من را بدست می آوریم

Collaborative Filtering جمع می کنیم فقط این تفاوت را دارد که score هر خانه در بردار همسایه را (r_{xj}) منهای baseline score اش می کنیم. b_{xj} ← baseline Estimator مربوط به خانه‌ی x_j

μ ← میانگین کل نمرات واقعی موجود در ماتریس utility

b_x ← اینکه user x چقدر نمراتش اختلاف دارد با μ

b_i ← نمرات واقعی آیتم i چقدر اختلاف دارد از میانگین کل (μ)، میانگین نمرات موجود برای آیتم‌ها را می گیریم و حاصل را منهای μ می کنیم

$\boxed{b_x = (\text{avg. rating of user } x) - \mu}$

یعنی user کلا نمرات بالایی میدهد و خب به آیتم نهم نمره بالایی می دهد به همین خاطر b_x را در نظر می گیریم. یا آیتم یعنی کلا نمراتش بالا است احتمالش زیاد که x هم بیش تر نمره بالایی بدهد به همین خاطر b_i را هم در نظر می گیریم

بعد بین User_User ، item_item ، روش دوم بهتر عمل می کند و جواب می دهد چون آیتم‌ها معمولا یک هدف دارند اما User ها سلیقه ها و هدف های مختلف دارند.

مزیت روش های Collaborative Filtering نیازبه Feature ندارد. در حقیقت بردار به rating

Scanned with CamScanner

های Feature نشسته است و این امر دامنه استفاده اش را گسترده می کند

عیب: ① cold_start دارد ← کاری که تازه وارد شده هیچ rating ای ندارد. نمی شه براش شباهت تعیین کرد و در هیچ شباهتی هم شرکت نمی کند.

② sparsity ← ماتریس utility خیلی اسپارس است در نتیجه بردارهای ما هم خیلی اسپارس هستند. در نتیجه تعداد زیادی از خانه ها را در pearson حذف می کنیم و دقت الگوریتم پایین می آید

③ First rater ← همان مشکل ① هست فقط برای آیتم ها. آیتمی که جدید اضافه می شود بردار نمره ندارد در نتیجه همسایه پیدا کردن براشون سخته. یا مثلا آیتم های خیلی خاص user های کمی بهشون نمره دادن و وقتی استراک می گیریم تهی می شود.

④ popularity bias ← آیتم های پرطرفدار در خیلی از همسایگی ها در شباهت ها می افتند، صرف نظر از اینکه شباهتشون زیاد باشه یا نه در نتیجه به سمت آیتم های پرطرفدار بایاس داریم.

۲۷ جمعه ۹ صفر ۱۴۴۰ 19 Oct 2018

در عمل برای حل مشکلات این الگوریتم، میایم الگوریتم های Recommendation را با هم ترکیب می کنیم

مثلا الگوریتم های content_based می تواند بیاید مشکل cold_start را حل کند وقتی user جدید اضافه می شود برای تعریف کردن همسایگی اش، content را در نظر بگیریم یا برای آیتم جدید که اضافه می شود ابتدا بردفایلش را در نظر بگیریم

Scanned with CamScanner

ارزیابی الگوریتم‌های Recommendation

معمولا یک دیتاست به شکل ماتریس داریم و ماتریس به صورت ضمنی است چون ابعادش خیلی بالاست. یک نسخه از ماتریس به صورت test_data به شما داده می‌شود. باید فرض کنید دیتاهای موجود در آن هیچ تست را ندارید و با الگوریتم خود آن‌ها را حدس بزنید.

معیارهای ارزیابی: یک معیار خیلی رایج ← RMSE

$$RMSE = \sqrt{\frac{1}{N}\sum_{xi}(r_{xi} - r^*_{xi})^2}$$

r_{xi} ← مقدار پیش‌بینی شده امتیاز کاربر x به آیتم i

r^*_{xi} ← مقدار واقعی امتیاز کاربر x به آیتم i

precision at top 10: ۱۰ درصد از خانه‌هایی که score بالا دارند حدس می‌زنیم و با مقدار واقعی overlap اش رو حساب می‌کنیم

Rank correlation برای خانه‌های تخمین زده شده یک ranking می‌سازید و براساس مقدار واقعی نشان هم یک ranking دیگر و این دو تا ranking را با هم مقایسه می‌کنید

یا اینکه می‌توانید به صورت یک مسئله classification به مسئله Recommendation نگاه کنید و ارزیابی را انجام دهید به صورت یک مسئله classification باشد. برچسب می‌زنید که آیتم i به user x مربوط هست یا نه. اگر مربوط هست توصیه شود. به ازای هر جفت (x و i) یک لیبل تعیین می‌کنید. مثلا از accuracy و Roc می‌توانید استفاده کنید برای ارزیابی آن. همین طور از coverage هم می‌توانید استفاده کنید. مثلا اینکه الگوریتم ببیند برای چند درصد از خانه‌ها می‌تواند حدسی را تخمین بزند. (تخمین زدن معنیش می‌تونه این باشه که خطا از یه حدی کمتر باشه)

coverage ← درصد خانه‌هایی که خطای تخمین الگوریتم برای آن‌ها از یه حدی کمتره

هرکدام از این معیارها ممکنه مشکلاتی را در عمل موجب شوند. مثلا در مورد accuracy

و برای خودمون prediction diversity داشته باشیم به این معنا که می‌خواهیم برای هر user به حداقلی Recommendation

Scanned with CamScanner

درست داشته باشیم. ممکنه الگوریتمی داشته باشیم که کلا accuracy پایینی نسبت به الگوریتم 2 داشته باشد ولی برای هر user میاد یکی دو تا آیتم را به درستی Recommend می‌کند. اما الگوریتم 2 که accuracy بالاتری دارد برای بیشتری از user ها Recommendation هایش درسته ولی برای یه تعدادی هم همه‌اش غلطه. الگوریتم 1 بهتر است چون در نهایت user یکی دو تا از Recommendation ها را انتخاب می‌کند.

* prediction Context ← برای آیتم‌ها هم باید diversity داشته باشد وابنطور نباشد که برای یه سری از انواع خاص آیتم‌ها accuracy خیلی بالایی نشان بدهد و برای یه سری از آیتم‌های دیگه کلا performance پایینی داشته باشد. برای همه آیتم‌ها باید میزان قابل قبولی از accuracy از خودش نشان بدهد.

order of prediction: ترتیب روی top ها خیلی مهم چون در نهایت خانه‌هایی که score بالا دارند به کاربر نشان داده می‌شوند. یه نسخه تغییر یافته‌ای از RMSE وجود دارد که میزان خطا را متناسب با رتبه در نظر می‌گیرد. اگر رتبه بالا باشد خطای آن را خیلی بیشتر نشان می‌دهد نسبت به رتبه پایین.

پیچیدگی زمانی Complexity برای Collaborative Filtering

فایده جوابی می‌خواهیم KNN ای انجام بدهیم. فرض کنید تعداد آیتم‌ها را ثابت در نظر بگیریم و بیاییم user_user collaborative filtering ای انجام بدهیم. اگر بخواهیم k تا همسایه نزدیک برای یک user پیدا کنیم پیچیدگی زمانی ما چقدر است؟

$O(k \cdot |x|)$ — برای یک user

اگر بخواهیم به صورت exact حل کنیم از $O(n)$ بهتر نمی‌شود — تعداد user ها = $|x|$

در نتیجه می‌توانیم از LSH استفاده کنیم برای کم کردن زمان. بیاییم آیتم‌هامون رو داخل یه سری باکت بیندازیم. امیدی که در یک باکت اند شبیه در نظر گرفته می‌شوند.

اگر از LSH استفاده نکنیم برای هر user همه‌ی بردار، $O(k \cdot |x|^2)$

Scanned with CamScanner

27 Oct 2018

* روش های جدید برای Recommendation

8 سیستمی که در مسابقه Netflix برنده شد ← Bellkor

Bellkor Recommender system

10 در سه سطح تخمین خانه ها را انجام دادند:

11 ① Global (کلی یا عمومی) ← از base Line Estimator استفاده کردند

12 ② Regional ← از latent factor ها استفاده کردند

13 ③ local ← از Collaborative Filtering بهبود داده شده استفاده کردند

14 Global

فرض کنید فردی می خواهد به فیلمی نمره بدهد و ما می خواهیم یک تخمین از این score داشته باشیم

15 کل میانگین نمرات : 3.7

16 تعداد امتیازهای این شخص به اندازه 0.5 از میانگین بالاتر است

17 این فرد آدم سختگیری بوده و امتیازهایی که داده به اندازه 0.2 از میانگین پایین تر بوده

18 تخمین عمومی ما (baseLine estimator) : $3.7 + 0.5 - 0.2 = 4$

Local Neighborhood (CF/NN)

حال اگر از Collaborative Filtering استفاده کنیم که روش local است و همسایگی فیلم را در نظر میگیرد: فرض کنید اندازه همسایگی ما 1 باشد و فیلمی که این فرد به آن نمره داده و از نظر بردار نزدیک

Scanned with CamScanner

شبیه به این فیلم است، انتخاب می‌کنیم. نمره این فیلم این فرد به این فیلم 3.8 بوده. درنتیجه دوتا Score پیدا کردیم. یکی Global دیگری هم Local.

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} S_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} S_{ij}}$$

$\mu + b_x + b_i$ ↑

وزن‌های S_{ij} که به چه صورت پیدا می‌کنیم

می‌تواند میزان شباهت بین آیتم i و j باشد ولی معیار دقیقی نیست. گام اولی که پیشنهاد دهنده‌های این روش برداشتند این بود که سعی کردند این وزن‌ها را یادبگیرند با استفاده از روش‌های آموزش که داریم.

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij} (r_{xj} - b_{xj})$$

w_{ij} ← میزان interaction بین آیتم i و آیتم j
این وزن بستگی به user x ندارد.

برای یادگیری وزن‌ها یک ایده کلی این است که یک objective Function تعریف کنیم و سعی می‌کنیم آن را minimize کنیم و وزن‌ها را یادبگیریم. سعی می‌کنیم معیار ارزیابی‌مان که RMSE بود را min کنیم:

$$\frac{1}{|R|} \sqrt{\sum_{(i;x) \in R} (\hat{r}_{xi} - r_{xi})^2}$$

برای ساده‌تر کردن تابع هدف می‌توانیم $\frac{1}{|R|}$ را حذف کنیم چون ثابته و در الگوریتم هم می‌توانیم حذف کنیم بدون اینکه تاثیری روی وزن‌ها داشته باشه. درنهایت از معیاری که به نام SSE استفاده می‌کنیم

Scanned with CamScanner

$$SSE = \sum_{(i,x)\in R} (\hat{r}_{xi} - r_{xi})^2$$

مجموعه‌ای که در دیتای آموزشی دارای score است

$\hat{r}_{xi}$ همون RMSE هم می‌شود. به جای min شدن SSE را min کنیم که مجزیه min شدن

می‌خایت می‌خواهیم

می‌توانیم فرمولش را قرار دهیم که به دست آوردیم. تابع هدف همان:

$$J(W) = \sum_{x,i} \left(\left[b_{xi} + \sum_{j\in N(i;x)} w_{ij} (r_{xj} - b_{xj}) \right] - r_{xi} \right)^2$$

با min کردن این تابع می‌خواهیم وزن‌های w_{ij} را به دست آوریم

یک روش رایج برای minimize کردن تابع، گرادیان نزولی است: ① مشتق تابع J را نسبت به وزن‌ها می‌گیریم.

11 جمعه ۲۳ صفر ۱۴۴۰

2 Nov 2018

② از یک مقدار دلخواه برای w شروع می‌کنیم

مشاهده‌ی وزن‌ها را ⊥ قرار می‌دهیم. بعد از خارج شده می‌شویم. در هر تکرار بردار وزن‌ها را به این صورت آپدیت می‌کنیم: $w = w - \nabla f(w)$

تا قدر ادامه می‌دهیم تا به همگرایی برسیم.

$$\partial_w J = \left[\frac{\partial J(w)}{\partial w_{ij}}\right] = 2\sum_{x,i} \left(\left[b_{xi} + \sum_{k\in N(i;x)} w_{ik} (r_{xk} - b_{xk}) \right] - r_{xi} \right) (r_{xj} - b_{xj})$$

for $j \in \{N(i;x), \forall i, \forall x\}$ else $\frac{\partial J(w)}{\partial w_{ij}} = 0$

Scanned with CamScanner

به ازای یک x خاص برای آیتم خاص i، همسایگی آیتم i را حساب می‌کنیم (wها) و برای آن wهایی که در همسایگی آیتم i هستند به ازای این x خاص، مشتق را حساب می‌کنیم که برابر با مقدار به دست آمده شود. برای بقیه wها که داخل همسایگی $N(i;x)$ نیستند مشتق برابر صفر می‌شود.

w_{new} ← بردار وزن در تکرار $t+1$ ام

w_{old} ← بردار وزن در تکرار t ام

while $|w_{new} - w_{old}| > \varepsilon_0$

$w_{old} = w_{new}$

$w_{new} = w_{old} - \eta \cdot \nabla w_{old}$

گام learning (مقدار ثابت) — مشتق تابع در w_{old}

برای همگرایی با مقدار تکرارهای ثابت را در نظر می‌گیریم یا یک حد آستانه برای $|w_{new} - w_{old}|$

در مسابقه‌ای که Net Flix برگزار کرده بود هدف این بود که مقدار RMSE از 0.95 به 0.85 برسد

- Global Average = 1.129 → یعنی فقط از μ استفاده کنیم
- User Avg : 1.06 → b_x را هم دخالت بدهیم — بدیهی b_x را هم دخالت بدهیم
- Movie Avg : 1.05 → b_i " " " b_i " "
- Netflix : 0.95
- 0.94 → basic collaborative filtering (item-item)
- 0.91 → CF + Biases + learned weights
- ↓ 0.85

Scanned with CamScanner

مدل Latent Factor

یک ماتریس دیتا داریم (R) که هر ردیف یک آیتم و هر ستون یک user را نشان می‌دهد

این ماتریس را تجزیه می‌کنیم و به شکل حاصل ضرب دو تا ماتریس می‌نویسیم:

$$R \approx QP^T$$

ماتریس Q ← ماتریس فیچرات جدید دیتای ما در دستگاه فیچرات جدید ما.

در این ماتریس متناظر با هر دیتا یک ردیف داریم و هر ردیف فیچرات دیتای ما در Hidden Factor ما است.

ماتریس P^T ← هر ردیف آن متناظر فیچرات hidden dimension های فضای اصلی دیتا

می‌توانیم R را تقریب بزنیم با: QP^T.

$$Q = U\Sigma \quad P^T = V^T$$

یا $Q = U \quad P^T = \Sigma V^T$

سوال: بعضی از خانه‌های ماتریس R خالی است

چگونه آن را تجزیه کنیم؟

می‌توانیم در گام اول یک مقدار اولیه مثلا ۰ بذاریم و بعد در فرآیند decomposition، طوری این‌ها رو بهینه کنیم که Reconstruction error کمترین باشد

جلسه بیست و هفتم - ۱۴۰۰/۳/۲۳

Latent Factor Models

گفتیم که بعضی از خانه‌های R خالی هستند، فرض کنید $R \approx Q \cdot P^T$

تجزیه‌ی R به $Q \cdot P^T$ را از یک طریق به دست آورده‌ایم. چگونه خانه‌های خالی R را پر کنیم؟

خانه (i,x) ماتریس R می‌شود ضرب i سطر ام Q در ستون x ام P^T. در واقع ماتریس

حاصل ضرب $Q \cdot P^T$ تخمین و تقریبی از R می‌شود.

$$\hat{r}_{ix} = q_i \cdot p_x = \sum_f q_{if} \cdot p_{xf}$$

(مثال از Q و P^T در اسلایدها)

$$A = U \Sigma V^T \quad R = Q P^T$$

$$A = R, \; Q = U, \; P^T = \Sigma V^T$$

سوال: چگونه این Decomposition را به دست آوریم؟

یه راه استفاده از SVD هست. خوبی SVD ← best low rank approximation را به ما می‌دهد

$$\min_{U,V,\Sigma} \sum_{ij \in A} (A_{ij} - [U \Sigma V^T]_{ij})^2$$ ← خطای Reconstruction

↳ در واقع همان خطای SSE هست و خود SSE هم وابسته است به RMSE.

در نهایت معیار ارزیابی ما برای Recommendor system خطای RMSE است که SVD دارد تخمین کار را برای ما می‌دهد. SVD دارد برای F ماتریس تفاضل را minimize می‌کند که در حقیقت همان SSE هست و RMSE را کم می‌کند روی دیتای آموزشی و اگر فرض کنیم هر دیتای ما از train به test می‌رویم تغییری پیدا می‌کند پس می‌توانیم امیدوار باشیم که min کردن RMSE روی دیتای آموزشی روی دیتای تست هم خطای RMSE را min می‌کند. این یک نکته خوب در مورد SVD است که در حقیقت از این ناشی می‌شود که SVD به best low Ran approximation می‌دهد.

فقط یه مشکلی که داریم اینه که ماتریس R دارای missing value است.

Scanned with CamScanner

عیب SVD ← پیچیدگی زمانی بالایی دارد (مرتبه ۳) — accuracy خوبی دارد ولی چون R دارای خانه‌های خالی است استفاده از SVD خیلی خوب نیست

بنابراین از روش‌های دیگری برای تجزیه استفاده می‌کنیم. درواقع به مسئله به چشم یک مسئله optimization نگاه می‌کنیم. خانه‌هایی از ماتریس R را که داریم به عنوان دیتای آموزشی درنظر می‌گیریم و سعی داریم که پارامترها را یاد بگیریم. پارامترهامون ماتریس‌های Q و P^T هستند. به گونه‌ای یاد می‌گیریم که objective function ما min شود.
درواقع می‌خواهیم خطا روی داده‌های آموزشی (خانه‌هایی از R که مقدارش را می‌دانیم) کم‌ترین مقدار شود.

$$\min_{P,Q} \sum_{(i,x)\in R} (r_{ix} - q_i \cdot p_x)^2 \qquad \hat{r}_{ix} = q_i \cdot p_x$$

یک ابرپارامتر داریم ← تعداد latent factor ها (تعداد ستون‌های ماتریس Q یا تعداد سطرهای ماتریس P)

هرچقدر کمتر باشد خطای بیشتری روی دیتای آموزشی داریم

۲۵ جمعه ۸ ربیع الاول ۱۴۴۰ — 16 Nov 2018

هرچقدر K را بزرگ‌تر بگیریم مسئله over fitting به وجود می‌آید و الگوریتم برای دیتای تست بد کار می‌کند.

K را خیلی کوچک درنظر می‌گیرند و برای حل مسئله overfitting ترم‌های Regularization اضافه می‌کنند:

$$\min_{P,Q} \sum_{\text{training}} (r_{ix} - q_i p_x)^2 + \left[\lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2\right]$$

λ_1 و λ_2 دو تا ابرپارامتر هستند که ما ست می‌کنیم. اهمیت هرکدام از ترم‌های Regularization را نشان می‌دهند
درحقیقت با تعریف ترم Regularization سعی داریم ماتریس Q و P که پیدا می‌کنیم تا حدامکان sparse باشند و Rank شان پایین باشد. (یک جریمه تعریف می‌کنیم و force می‌کنیم ترم P و Q پایین بیاید درنتیجه sparse شود و رنکش پایین بیاید)
بنابراین با حل این مسئله optimization پارامترهامون که ماتریس‌های P و Q هستند یادمی‌گیریم.

Scanned with CamScanner

چگونه می‌توانیم P و Q بهینه را بیابیم؟

برای حل این مسئله optimization، می‌توان از روش gradient descent

استفاده کنیم. (الگوریتم گرادیان نزولی در اسلایدها)

مشتق objective function را نسبت به P و Q باید حساب کنیم.

$$\nabla q_{jf} = \sum_{i,x} -2(r_{ix} - q_i \cdot p_x) p_{xf} + 2\lambda_2 q_{jf}$$

نکته: مشتق گرفتن از ماتریس یعنی مشتق تک‌تک عناصر آن.

مشکل روش گرادیان نزولی ← برای اینکه یک iteration برویم باید کل دیتای آموزشی را scan کنیم و کاست زمانی دارد.

راه‌حل ← از stochastic gradient descent استفاده کنیم. به‌صورت تصادفی گرادیان نزولی را انجام می‌دهد. مثلاً به جای اینکه بگیم گرادیان را برای همه خانه‌ها انجام بدهیم، بعضی از خانه‌های P یا Q را انتخاب گرادیان بگیریم

به حالت از stochastic Gradient descent این صورت هست که گرادیان نسبت به تمامی متغیرها را در هر iteration انجام می‌دهیم. مثلاً در هر iteration نسبت به Q ممکنه انجام بدهیم در هر iteration نسبت به P.

به روشی که برای گرادیان پیشنهاد شد: ما میایم روی تک‌تک دیتاهای training در هر گام گرادیان نزولی، iteration انجام بدهیم. ما میایم به دیتای آموزشی خاص (x, i) برای خواندن در یک گام می‌رویم جلو. در بعد بعدی را می‌خوانیم در یک گام می‌رویم جلو.

GD:

$$q_{jf} \leftarrow q_{jf} - \eta \left[\sum_{i,x} -2(r_{ix} - q_i \cdot p_x) p_{xf} + 2\lambda_2 q_{jf} \right]$$

SGD:

$$q_{jf} \leftarrow q_{jf} - \mu \left[-2(r_{ix} - q_i \cdot p_x) p_{xf} + 2\lambda_2 q_{jf} \right]$$

بعد از خواندن هر دیتای آموزشی یک update روی ماتریس Q و P به صورت بالا انجام می‌دهید ↑ به جای اینکه یک دور کامل دیتای آموزشی را بخوانید و یک آپدیت روی ماتریس Q و یک آپدیت روی ماتریس P انجام بدهید چون این فرآیند تصادفی است ممکنه از تابع هدف دور و نزدیک شویم ولی مستقیم به سمت نقطه optimum نرویم ولی سریع انجام می‌شود.

Scanned with CamScanner

Stochastic به صورت نوبتی عمل می‌کند اما در نهایت نتیجه خوبی می‌دهد و خیلی سریع‌تر عمل می‌کند

دوتا loop داریم: ① until convergance ② به روب دیگه کد در آن نیتاهای آموزش را می‌خوانیم و به ازای هر نیتای آموزش یک step در گرادیان برمیداریم.

خود latent factor به‌تنهایی نتوست حداکثر نتیجه‌ای که در مسابقه Netflix خواسته شده بود را به دست آورد. به همین دلیل می‌خواهیم Baseline estimator را وارد کنیم.

baseLine Estimator چگونه کار می‌کند؟ ← تخمین خیلی $\mu + b_x + b_i$

b_x ← میزان اختلافی که user x از میانگین نمرات دارد

b_i ← میزان اختلافی که نمرات آیتم i از میانگین نمرات دارد

می‌خواهیم baseline را با یک روش user_movie interaction که همان latent factor است ادغام کنیم

اگر بخواهیم baseline estimator را در latent factor بیاوریم: b ← بردار

حالت اول:

این ۳ تا رو داریم و معلومه

$$\hat{r}_{ix} = \underbrace{\mu + b_x + b_i}_{\text{بردار } b \text{ مربوط به user}} + \underbrace{q_i \cdot p_x}_{\text{بردار } b \text{ مربوط به آیتم}} \quad \text{latent factor ها براساس estimation}$$

q و p ای که در این حالت یادگرفته می‌شود sparse تر است. در تابع هدف منفی μ و b_x و b_i هم اضافه می‌شود

حالت دوم: حالت دیگری هم برای وارد کردن baseline estimator در latent factor پیشنهاد شده.

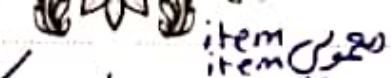
هرچقدر بتونیم بیشتر یاد بگیریم بهتر است. در این روش بردار b را از مقادیر پیش فرض گرفته‌ایم استفاده نکنیم بیایم اینا رو هم یاد بگیریم و به صورت پارامتر روش نگاه کنیم. تابع هدف این شکلی میشه:

$$\min_{b_x, b_i, Q, P} \sum_{(x,i) \in R} (r_{ix} - (\mu + b_x + b_i + q_i p_x))^2 + \left(\lambda_1 \sum_i \|q_i\|^2 + \lambda_2 \sum_x \|p_x\|^2 + \lambda_3 \sum_x \|b_x\|^2 + \lambda_4 \sum_i \|b_i\|^2\right)$$

ماتریس P و Q را با استفاده از SVD مقداردهی اولیه می‌کنیم و scoreهایی که نداریم را صفر قرار می‌دهیم. ماتریس b مربوط به user ها و آیتم‌ها را هم مقداردهی اولیه می‌کنیم با تعریف اریبی‌های اصلی‌شان.

این روش RMSE را نسبت به حالتی که از b های ثابت استفاده کنیم کاهش می‌دهد.

Scanned with CamScanner



در روش Collaborative Filtering پارامترهایی که میدی گیریم وزن‌ها هستند. چگونه می‌توانیم مقدار پارامترهامون رو زیاد کنیم؟ مثلا با افزایش اندازه‌ی همسایگی (k تا همسایه در نظر می‌گرفتیم برای هر آیتم، k را اگر زیاد کنیم مقدار وزن‌ها زیادی شود) راه دیگر افزایش تعداد users ها (افزایش اندازه جدول rating)، هر چقدر وزن‌ها بیشتر باشد هزینه بیشتر و فرآیند آموزش کند تر و خطا کمتری شود

در روش Basic latent factor پارامترهای ما ماتریس های P و Q هستند و باید تک تک عناصر آن ها را تخمین بزنیم افزایش پارامترها ← با افزایش تعداد latent factor ها (تعداد ستون های ماتریس Q و سطرهای ماتریس P^T)

در روش latent factor + baseline + learn ← بهترین performance را در مقایسه با دو روش قبلی دارد RMSE به میزان قابل توجهی بهبود یافته. برای افزایش مقدار پارامترها هم مشابه حالت قبلی باید اندازه P و Q را زیاد کنیم. تعداد ستون‌های ماتریس Q که تعداد سطرهای P هست را زیاد کنیم.

subspace Embedding

انجام سازی یا sketching ← به جای اینکه الگوریتم‌هامون رو بزرگ کنیم، دیتا مون رو کوچیک کنیم و یک الگوریتم معمولی روی آن پیاده کنیم. پس ما یک تبدیل داریم که باید یک سری ویژگی‌های خاصی داشته باشد. مثلا داده‌ها را کوچیک می‌کنیم و الگوریتمی روی آن اعمال می‌کنیم و جواب به دست می‌آوریم. باید یک گارانتی داشته باشیم که این جواب به جواب اصلی روی داده‌های زیاد، نزدیک باشد و یک تقریب خوبی از آن باشد.

(باید error guaranty داشته باشیم)

Scanned with CamScanner

جلسه بیست و هشتم - ۱۴۰۰/۳/۲۵

8

Sketching برای Linear Regression:

9

least square Linear Regression چیست؟ فرض کنید یک ماتریس A داریم که mxn است

10

یک بردار b هم داریم که mx1 است. این دو تا دیتای ورودی ما هستند. هر دیتا پوینت ما یک ردیف از ماتریس

11

A است به علاوه‌ی یک عدد از بردار b (خانه متناظر در بردار b). در رگرسیون خطی می‌خواهیم یک تابع یاد

12

بگیریم که ورودی آن یک بردار دیتر گی mتایی است (مثلا یک ردیف از ماتریس A) و خروجی اش یک عدد است

13

مثلا مقدار متناظر با این ردیف در بردار b. این تابع قرار است یک نگاشت (mapping) برقرار کند بین

14

این بردار دیتر گی mتایی و این تک خونه. مقادیر b ما را با توجه به Feature Vector به ما بدهد.

15

درحقیقت mتا example برای آموزش (train) داریم. به طور کلی مسئله‌ی یادگیری این تابع

16

رگرسیون نامیده می‌شود.

17

18 در Linear Regression فرض می‌کنیم خروجی ما که بردار b هست یک ترکیب خطی از فیچرهای ماست

فرض می‌کنیم مقادیر ماتریس A عددی هست و مقادیر بردار b هم عددی هست

یه بردار x داریم که وزن‌های فیچرها را نشان می‌دهد. (n فیچرها ← $A_1\ A_2 \dots A_n$)

رگرسیون خطی حداقل مربعات least square Linear Regression

در این رگرسیون، بردار x اینگونه پیدا می‌شود:

Scanned with CamScanner

x ای را پیدا می‌کنیم به طوری که $A.x$ را نزدیک‌ترین نقطه به بردار b می‌کند:

$$\text{Argmin}_x \|A.x - b\|_2 \rightarrow \text{norm } 2$$

در حقیقت این جا Ax یک ترکیب خطی از ستون‌های ماتریس A است:

$$A.x = x_1 A_1 + x_2 A_2 + \dots + x_n A_n$$

به عبارت دیگر: از بین تمام ترکیب خطی‌های ستون‌های A (فضای ستونی ماتریس A) به دنبال column space

این هستیم که نزدیک‌ترین نقطه به b باشد.

برای این کار از توان ۲ می‌رسانیم: $\text{argmin}_x \|A.x - b\|_2^2$

← رنک ستونی یک ماتریس ← dimension فضای ستونی - فضای ستونی، تمام ترکیب‌های خطی ستون‌های ماتریس

← به طور مشابه رنک (رتبه یا مرتبه) برای فضای سطری هم تعریف می‌شود (فضای ردیفی ماتریس)

← بُعد فضای ستونی با بُعد فضای ردیفی با هم برابر هستند

← ماتریس A، full rank است اگر و تنها اگر تمام ستون‌هایش مستقل خطی باشند (به عبارت دیگر اگر n تا ستون دارد، تعداد بُعدهای فضای ستونی آن هم n باشد)

تجزیه SVD ماتریس A: $A = U.\Sigma.V^T$

نرم ۲ بردار: $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$

وارون ماتریس A اگر و تنها اگر حداقل یکی از شرط‌های زیر برقرار باشد: $A^{-1}.A = I$ $A.A^{-1} = I$

Scanned with CamScanner

اگر ماتریس مربعی نباشد و یا مربعی باشد و دترمینان آن صفر باشد، از شبه وارون استفاده می‌کنیم که بر

اساس تجزیه SVD تعریف می‌شود. شبه وارون ماتریس A:

$V.\Sigma^{\dagger}.U^{T}$

↳ شبه وارون Σ

(در صورتی که تجزیه SVD برای ماتریس A بصورت $U\Sigma V^{T}$ باشد)

راه حل:

$$x = \underbrace{A^{\dagger}}_{\text{شبه وارون } A}.b$$

مزیت راه حل ← closed form داریم. مثلا اگه از نرم ۱ استفاده کنیم فرم بسته ندارد

عیب ← زمانبر است چون لازم است SVD را بیابیم و SVD هم از مرتبه ۳ (cubic) است

$\min(O(nm^{2}), O(mn^{2}))$

بنابراین دنبال جوابی هستیم که تقریبی از جواب بهینه به ما بدهد بطوری که

این تقریب: اولا نمایش به میزان قابل توجهی بهتر باشد، دوما دارای خطای آن کرانی وجود داشته باشد یعنی خطا در بدترین حالت هم بینهایت و خارج از کنترل نباشد. این تقریب به این صورت تعریف می‌شود:

$$\|A.x' - b\|_{2}^{2} \leq (1+\epsilon)\min_{x}\|A.x - b\|_{2}^{2} \qquad \epsilon \in (0,1)$$

ϵ عدد خیلی کوچکی است در نتیجه میزان خطا محدود

جمعه
۲۲ ربیع الاول ۱۴۴۰

30 Nov 2018

قابل حل است. پس ما می‌خواهیم مسئله optimization

را به صورت exact حل کنیم و به صورت approximate می‌خواهیم حل کنیم. زمان سریعتری دارد و خطای کمی دارد.

transformation های اعمال می‌کنیم روی A و b، بعد مسئله را روی ماتریس و بردار حاصل به صورت بهینه حل می‌کنیم. چون دنیای ما کوچیک شده زمان زیادی نمی‌برد، حتی اگر مسئله را به صورت optimal حل کنیم.

۲ تا مدل را بررسی می‌کنیم:

① subsampled Randomized Hadamard

② count sketch

توضیح ①:

Hadamard Matrix: یک ماتریس مربعی که تعداد سطر و ستون‌هایش توانی از ۲ است.

$n = 2^{k}$

$\underbrace{n \times n}$

Scanned with CamScanner

$H[i,j] = \frac{(-1)^{\langle i,j \rangle}}{\sqrt{n}}$ ← عبارت $\langle i,j \rangle$ را در نظر گرفته ضرب داخلی می‌کنیم

عبارت دودویی i و j را پیدا می‌کنیم و آن را به صورت بردار در نظر می‌گیریم و ضرب داخلی می‌کنیم

برای ساختن ماتریس Hadamard رابطه بازگشتی هم داریم:

$$H_k = \begin{bmatrix} H_{k-1} & H_{k-1} \\ H_{k-1} & H_{k-1} \end{bmatrix}$$

از یک ماتریس ۲×۲ شروع می‌کنیم: $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ به همین صورت بازگشتی ماتریس‌های بزرگتر را می‌سازیم ماتریس Hadamard دوبعدی

subsampled randomized Hadamard transfom

این تبدیل از حاصل ضرب سه تا ماتریس تشکیل شده که یا از سمت چپ در A یا ضرب می‌شود

$$P.H.D$$

ماتریس $D \leftarrow m \times m$ ، قطری و مربعی است، عناصر روی قطر اصلی یا $+1$ هستند یا -1 با احتمال یکسان. بقیه خانه‌ها صفر هستند. یک ماتریس مربعی $m \times m$ می‌سازیم که عناصر روی قطر اصلی با احتمال ۵۰٪ برابر $+1$ و با احتمال ۵۰٪ برابر -1 هستند. ماتریس $H \leftarrow$ ماتریس Hadamard $m \times m$

ماتریس $P \leftarrow r \times m$ ، r تعداد سمپل‌هایی که می‌خواهیم بگیریم. در هر ردیف ماتریس P دقیقاً یک مقدار غیر صفر داریم. برای هر ردیف از ماتریس P یکی از ستون‌ها را به صورت رندوم انتخاب کرده و آن را برابر $P[i,j] = \sqrt{m}/\sqrt{r}$ قرار می‌دهیم.

Uniformly at Random

نکته: Sampling در واقع نوعی Sketching (خلاصه سازی) است با این تفاوت که در Sampling فقط کمتر از داده‌ها را برمی‌داریم اما در Sketching تغییراتی را هم روی داده‌ها می‌دهیم.

CountSketch

یک ماتریس CountSketch داریم به نام S که در هر ستونش یک مقدار غیر صفر دارد دقیقاً. در هر ستون یکی از ردیف‌ها را به صورت رندوم انتخاب می‌کنیم و آن را با احتمال ۵۰٪ برابر $+1$ یا -1 قرار می‌دهیم

$S_{k \times m}$ (k×m)

کیفیت ماتریس اسپارس است و تعداد عناصر غیر صفرش m تاست

$$k = O\left(\frac{n^2}{\epsilon^2}\right)$$

Scanned with CamScanner

یک ویژگی خوبی که countSketch دارد این است که ضرب S.A به صورت efficient انجام می‌گیرد.

معمولا برای ضرب ماتریس از حلقه‌های تودرتو استفاده می‌شود که $O(n^3)$ می‌شود اما در این جا ماتریس S.A از مرتبه nonzero element به گونه‌ای است که ضرب count sketch می‌شود

تعداد خانه‌های غیر صفر ماتریس A که ماتریس A هم چون ماتریس sparse است.

گام‌های Sketching:

① ابتدا یک ماتریس sketch تولید کنید (countSketch یا P.H.D) و آن را S می‌نامیم

② ماتریس و بردار دیتا را از S عبور می‌دهیم یعنی SA و Sb را حساب می‌کنیم

③ مسئله least square regression را برای S.A و S.b حل می‌کنیم به صورت optimal:

$$\arg\min_{x'} \| (S.A).x' - S.b \|_2^2$$

جواب مسئله: $(S.A)^{\dagger}.S.b$

می‌خواهیم error guarantee داشته باشیم. در حالت استفاده از P.H.D می‌خواهیم تعداد ردیف‌های ماتریس P را چقدر داشته باشد (r) برای اینکه error guarantee ما برقرار باشد را پیدا کنیم.

در count sketch هم مقدار k را باید تعیین بکنیم.

(در اسلایدها فرمول‌هایی از مقدار r و k ارائه شده که برای امتحان لازم نیست)

Scanned with CamScanner

Generalized least Squre Linear Regression (رگرسیون خطی حداقل مربعات تعمیم یافته)

به جای بردار b، یک ماتریس B داریم که $m \times n'$ است. یعنی هر datapoint ما از یک ردیف از ماتریس A تعریف شده و همین طور یک ردیف از ماتریس B

هدف: n' تا تابع یا mapping یاد بگیریم. تابع اول یک mapping از Feature vector های ما به ستون اول ماتریس B برقراری کند. تابع دوم هم همینطور...

مثل این است که به جای اینکه یک بردار b داشته باشیم n' تا بردار b داشته باشیم. راه حل اینکه هر کدوم از ستون های B را در نظر بگیریم و رگرسیون بزنیم.

$$\underset{X}{\operatorname{argmin}} \|A.X - B\|_F^2 \qquad X \in R^{n \times n'}$$

Forbenius norm

مشکل این راه حل: پیچیدگی زمانی در n' خطی می شود

(یک یک ستون ها را در نظر بگیریم و حل کنیم)

به جای اینکه تک تک ستون ها را در نظر بگیریم و مسئله را حل کنیم می خواهیم efficient تر این کار را انجام بدهیم از Affine embedding استفاده می کنیم.

۱۶ جمعه
۲۹ ربیع الاول ۱۴۴۰
7 Dec 2018

Affine Embedding

یک $S \in$ Affine Embedding یک ماتریس S است که این ۳ ویژگی را باید داشته باشد:

① بتونه ضرب ماتریسی رو به صورت بهینه و تقریبی انجام بدهد. فرض کنید دو تا ماتریس C و D دارید.

باید این ویژگی را داشته باشه:

$$\|C.S^T.S.D - C.D\|_F$$

باید عدد کوچکی باشد با احتمال زیاد

می خواهید به عنوان تقریبی از CD، CS^TSD را حساب بکنید. این تقریب باید خوب باشد

Scanned with CamScanner

② به ازای هر بردار دل خواه x، Ax را در نظر بگیرید یک بردار است. اگر این بردار را از S عبور

دهید و نرم ۲ بگیرید باید این شرط برقرار باشد:

$$\|S.Ax\|_2^2 = (1 \pm \epsilon) \|A.x\|_2^2$$

③ نرم یک ماتریس را حفظ کند:

$$\|SC\|_F^2 = (1 \pm \epsilon) \|C\|_F^2$$

دو تا صدایی که معرفی کردیم دیدگی های مدل affine را هم دارند فقط ممکنه به مقدار اندازه‌شان

بیشتر باشد وقتی که به عنوان affine استفاده می‌شوند.

الگوریتم:

① ابتدا یک affine Embedding به نام S تولید می‌کنیم (مثلا CountSketch, PHD)

② ماتریس A و B را از S عبور می‌دهیم: SA , SB

③ بعد این مسئله را حل می‌کنیم به صورت optimal:

$$\text{argmin}_{x'} \|(SA)x' - SB\|_F^2$$

error guarantee $\rightarrow 1+\epsilon$

که جوابش تقریبی از جواب اصلی است.

Scanned with CamScanner